

Welcome

Thank you for choosing Freenove products!

How to Start

When reading this, you should have downloaded the ZIP file for this product.
Unzip it and you will get a folder containing tutorials and related files. Please start with this PDF tutorial.

- ! Unzip the ZIP file instead of opening the file in the ZIP file directly.
- ! Do not move, delete or rename files in the folder just unzipped.

Get Support

Encounter problems? Don't worry! Refer to "TroubleShooting.pdf" or contact us.

When there are packaging damage, quality problems, questions encountering in use, etc., just send us an email. We will reply to you within one working day and provide a solution.

support@freenove.com

Attention

Pay attention to safety when using and storing this product:

- This product is not suitable for children under 12 years of age because of small parts and sharp parts.
- Minors should use this product under the supervision and guidance of adults.
- This product contains small and sharp parts. Do not swallow, prick and scratch to avoid injury.
- This product contains conductive parts. Do not hold them to touch power supply and other circuits.
- To avoid personal injury, do not touch parts rotating or moving while working.
- The wrong operation may cause overheat. Do not touch and disconnect the power supply immediately.
- Operate in accordance with the requirements of the tutorial. Fail to do so may damage the parts.
- Store this product in a dry and dark environment. Keep away from children.
- Turn off the power of the circuit before leaving.

About

Freenove provides open source electronic products and services.

Freenove is committed to helping customers learn programming and electronic knowledge, quickly implement product prototypes, realize their creativity and launch innovative products. Our services include:

- Kits for learning programming and electronics
- Kits compatible with Arduino®, Raspberry Pi®, micro:bit®, ESP32®, etc.
- Kits for robots, smart cars, drones, etc.
- Components, modules and tools
- Design and customization

To learn more about us or get our latest information, please visit our website:

<http://www.freenove.com>

Copyright

All the files provided in the ZIP file are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). You can find a copy of the license in the ZIP file.



It means you can use these files on your own derived works, in part or completely. But not for commercial use.

Freenove® brand and logo are trademarks of Freenove Creative Technology Co., Ltd. Must not be used without permission.



Other registered trademarks and their owners appearing in this document:

Arduino® is a trademark of Arduino LLC (<https://www.arduino.cc/>).

Raspberry Pi® is a trademark of Raspberry Pi Foundation (<https://www.raspberrypi.org/>).

micro:bit® is a trademark of Micro:bit Educational Foundation (<https://www.microbit.org/>).

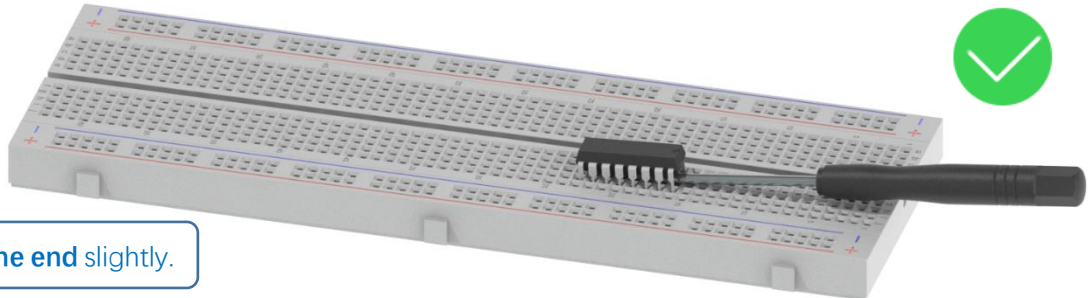
ESPRESSIF® and ESP32® are trademarks of ESPRESSIF Systems (Shanghai) Co., Ltd (<https://www.espressif.com/>).

Remove the Chips

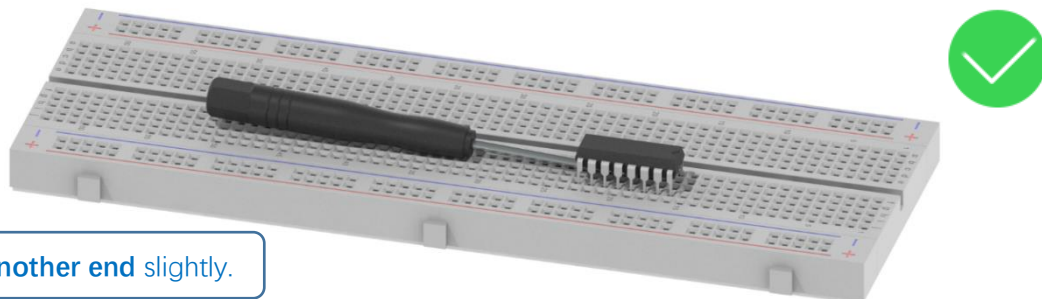
Some chips and modules are inserted into the breadboard to protect their pins.

You need to remove them from breadboard before use. (There is no need to remove GPIO Extension Board.)

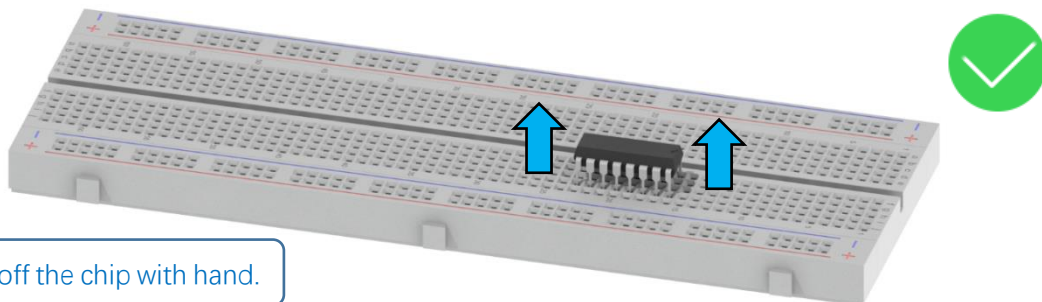
Please find a tool (like a little screw driver) to handle them like below:



Step 1, lift **one end** slightly.

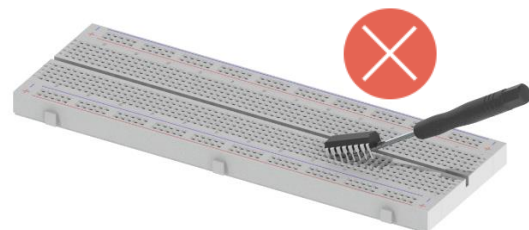
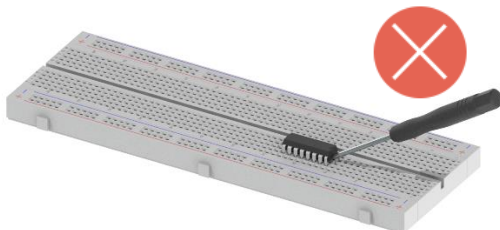


Step 2, lift **another end** slightly.



Step 3, take off the chip with hand.

Avoid lifting one end with big angle directly.



Contents

Welcome.....	1
Remove the Chips.....	3
Contents	4
Prepare	5
ESP32-WROVER	6
0.3 Installing CH340 (Important).....	9
0.4 Burning Micropython Firmware (Important).....	19
0.5 Testing codes (Important)	24
0.6 Thonny Common Operation.....	31
0.7 Note	36
Chapter 1 LED (Important).....	38
Project 1.1 Blink.....	38
Chapter 2 Bluetooth	47
Project 2.1 Bluetooth Low Energy Data Passthrough.....	47
Chapter 3 WiFi Working Modes	57
Project 3.1 Station mode	57
Project 3.2 AP mode	61
Project 3.3 AP+Station mode.....	65
Chapter 4 TCP/IP	69
Project 4.1 As Client.....	69
Project 4.2 As Server.....	80
Chapter 5 Camera Web Server	85
Project 5.1 Camera Web Server	85
What's next?.....	96
End of the Tutorial	96

Prepare

ESP32 is a micro control unit with integrated Wi-Fi launched by Espressif, which features strong properties and integrates rich peripherals. It can be designed and studied as an ordinary Single Chip Microcontroller (SCM) chip, or connected to the Internet and used as an Internet of Things device.

ESP32 can be developed both either with C/C++ language or micropython language. In this tutorial, we use micropython. With Micropython is as easy to learn as Python with little code, making it ideal for beginners. Moreover, the code of ESP32 is completely open-source, so beginners can quickly learn how to develop and design IOT smart household products including smart curtains, fans, lamps and clocks.

We divide each project into four parts, namely Component List, Component Knowledge, Circuit and Code. Component List helps you to prepare material for the experiment more quickly. Component Knowledge allows you to quickly understand new electronic modules or components, while Circuit helps you understand the operating principle of the circuit. And Code allows you to easily master the use of ESP32 and its accessory kit. After finishing all the projects in this tutorial, you can also use these components and modules to make products such as smart household, smart cars and robots to transform your creative ideas into prototypes and new and innovative products.

In addition, if you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com

ESP32-WROVER

ESP32-WROVER has launched a total of two antenna packages, PCB on-board antenna and IPEX antenna respectively. The PCB on-board antenna is an integrated antenna in the chip module itself, so it is convenient to carry and design. The IPEX antenna is a metal antenna derived from the integrated antenna of the chip module itself, which is used to enhance the signal of the module.

PCB on-board antenna

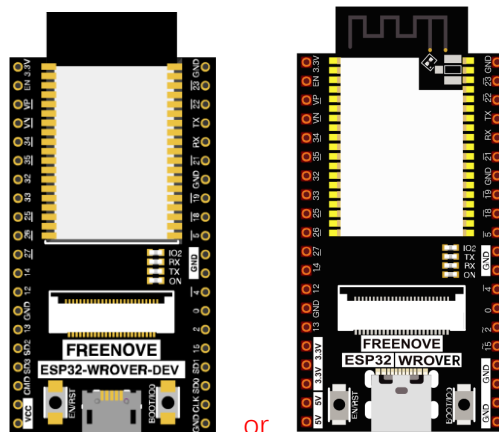


IPEX antenna



In this tutorial, the ESP32-WROVER is designed based on the PCB on-board antenna package.

ESP32-WROVER

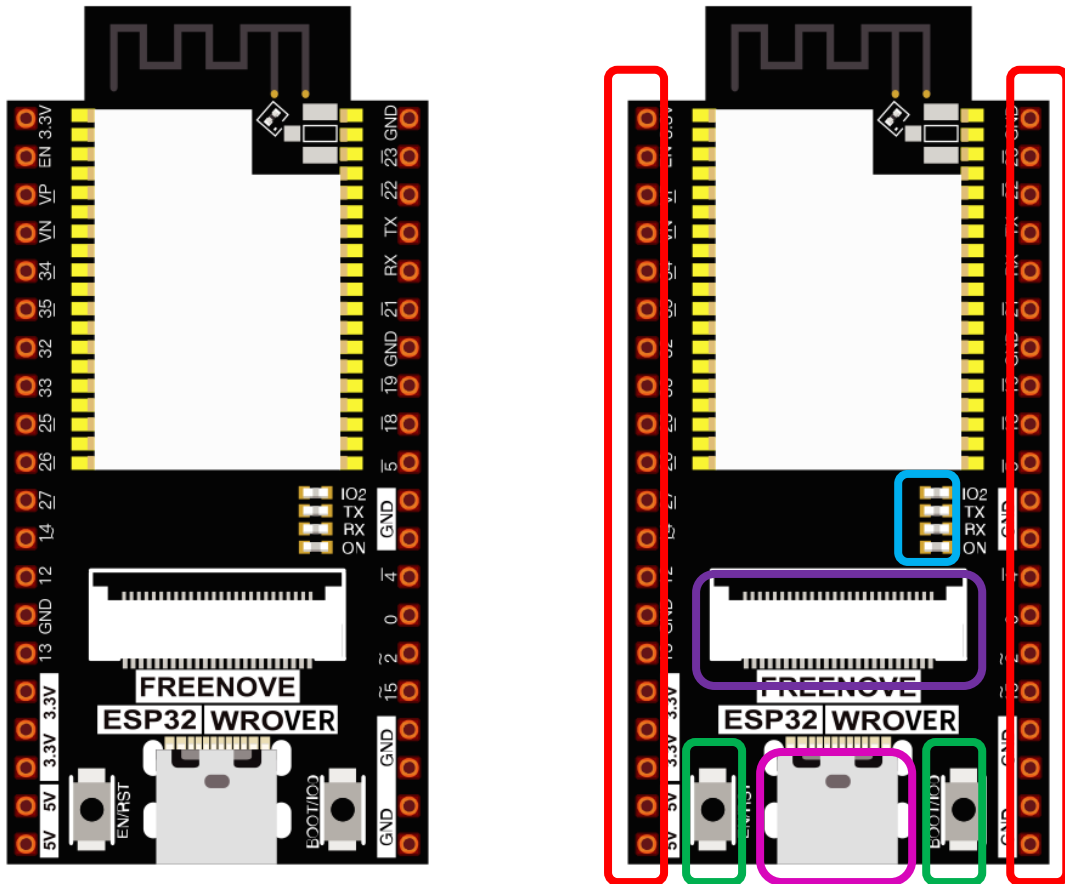


or

The version on the left is no longer mass-produced, and we mainly maintain the version on the right. Please note that there are many pirated versions of the ESP32 WROVER that look very similar to the version on the left. None of them will carry our logo and Freenove font.

We do not sell pirated ESP32 WROVER, nor do we provide after-sales service for pirated.

The hardware interfaces of ESP32-WROVER are distributed as follows:



Compare the left and right images. We've boxed off the resources on the ESP32-WROVER in different colors to facilitate your understanding of the ESP32-WROVER.

Box color	Corresponding resources introduction
	GPIO pin
	LED indicator
	Camera interface
	Reset button, Boot mode selection button
	USB port

Name	No.	Type	Function
GND	1	P	Ground
3V3	2	P	Power supply
EN	3	I	Module-enable signal. Active high.
SENSOR_VP	4	I	GPIO36, ADC1_CH0, RTC_GPIO0
SENSOR_VN	5	I	GPIO39, ADC1_CH3, RTC_GPIO3
IO34	6	I	GPIO34, ADC1_CH6, RTC_GPIO4
IO35	7	I	GPIO35, ADC1_CH7, RTC_GPIO5
IO32	8	I/O	GPIO32, XTAL_32K_P (32.768 kHz crystal oscillator input), ADC1_CH4, TOUCH9, RTC_GPIO9
IO33	9	I/O	GPIO33, XTAL_32K_N (32.768 kHz crystal oscillator output), ADC1_CH5, TOUCH8, RTC_GPIO8
IO25	10	I/O	GPIO25, DAC_1, ADC2_CH8, RTC_GPIO6, EMAC_RXD0
IO26	11	I/O	GPIO26, DAC_2, ADC2_CH9, RTC_GPIO7, EMAC_RXD1
IO27	12	I/O	GPIO27, ADC2_CH7, TOUCH7, RTC_GPIO17, EMAC_RX_DV
IO14	13	I/O	GPIO14, ADC2_CH6, TOUCH6, RTC_GPIO16, MTMS, HSPICLK, HS2_CLK, SD_CLK, EMAC_TXD2
IO12 ¹	14	I/O	GPIO12, ADC2_CH5, TOUCH5, RTC_GPIO15, MTDI, HSPIQ, HS2_DATA2, SD_DATA2, EMAC_TXD3
GND	15	P	Ground
IO13	16	I/O	GPIO13, ADC2_CH4, TOUCH4, RTC_GPIO14, MTCK, HSPID, HS2_DATA3, SD_DATA3, EMAC_RX_ER
SHD/SD2 ²	17	I/O	GPIO9, SD_DATA2, SPIHD, HS1_DATA2, U1RXD
SWP/SD3 ²	18	I/O	GPIO10, SD_DATA3, SPIWP, HS1_DATA3, U1TXD
SCS/CMD ²	19	I/O	GPIO11, SD_CMD, SPICSS0, HS1_CMD, U1RTS
SCK/CLK ²	20	I/O	GPIO6, SD_CLK, SPICLK, HS1_CLK, U1CTS
SDO/SD0 ²	21	I/O	GPIO7, SD_DATA0, SPIQ, HS1_DATA0, U2RTS
SDI/SD1 ²	22	I/O	GPIO8, SD_DATA1, SPID, HS1_DATA1, U2CTS
IO15	23	I/O	GPIO15, ADC2_CH3, TOUCH3, MTDO, HSPICSS0, RTC_GPIO13, HS2_CMD, SD_CMD, EMAC_RXD3
IO2	24	I/O	GPIO2, ADC2_CH2, TOUCH2, RTC_GPIO12, HSPWP, HS2_DATA0, SD_DATA0
IO0	25	I/O	GPIO0, ADC2_CH1, TOUCH1, RTC_GPIO11, CLK_OUT1, EMAC_TX_CLK
IO4	26	I/O	GPIO4, ADC2_CH0, TOUCH0, RTC_GPIO10, HSPHD, HS2_DATA1, SD_DATA1, EMAC_TX_ER
NC1	27	-	-
NC2	28	-	-
IO5	29	I/O	GPIO5, VSPICSS0, HS1_DATA6, EMAC_RX_CLK
IO18	30	I/O	GPIO18, VSPICLK, HS1_DATA7
IO19	31	I/O	GPIO19, VSPIQ, U0CTS, EMAC_TXD0
NC	32	-	-
IO21	33	I/O	GPIO21, VSPHD, EMAC_TX_EN
RXD0	34	I/O	GPIO3, U0RXD, CLK_OUT2
TXD0	35	I/O	GPIO1, U0TXD, CLK_OUT3, EMAC_RXD2
IO22	36	I/O	GPIO22, VSPWP, U0RTS, EMAC_TXD1
IO23	37	I/O	GPIO23, VSPID, HS1_STROBE
GND	38	P	Ground

Notice:

- GPIO12 is internally pulled high in the module and is not recommended for use as a touch pin.
- Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and SCS/CMD, namely, GPIO6 to GPIO11 are connected to the SPI flash integrated on the module and are not recommended for other uses.

For more information, please visit: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf

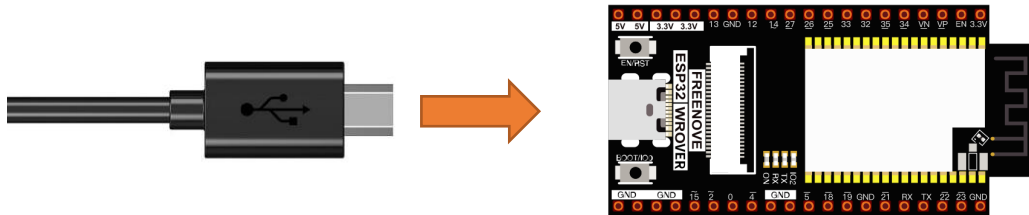
0.3 Installing CH340 (Important)

ESP32 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

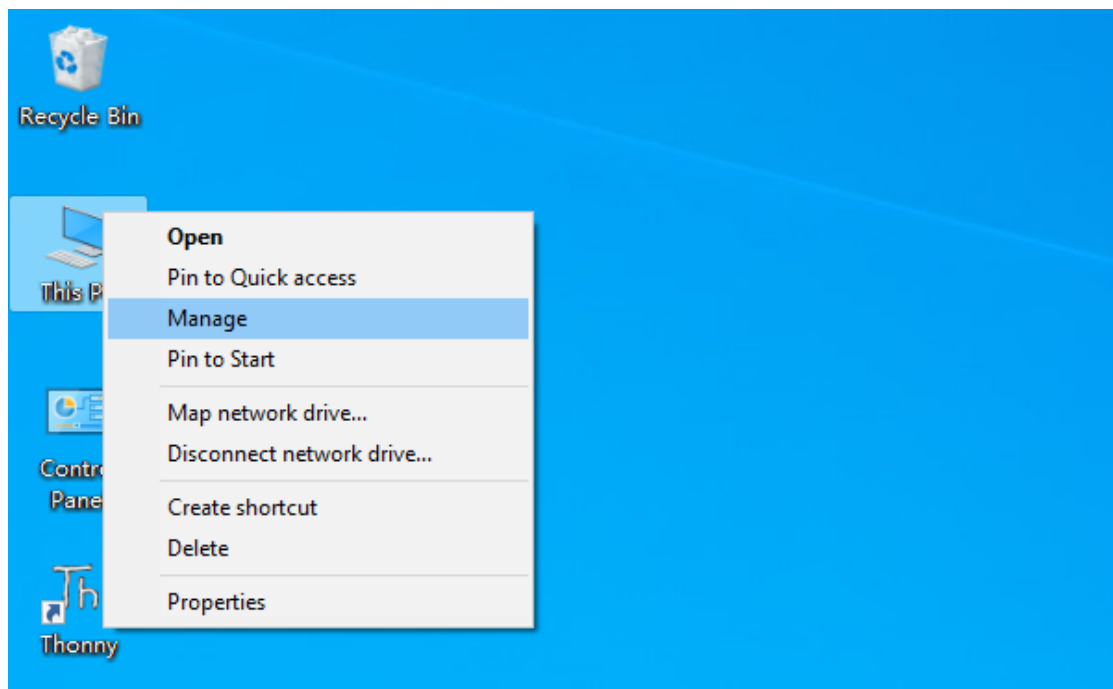
Windows

Check whether CH340 has been installed

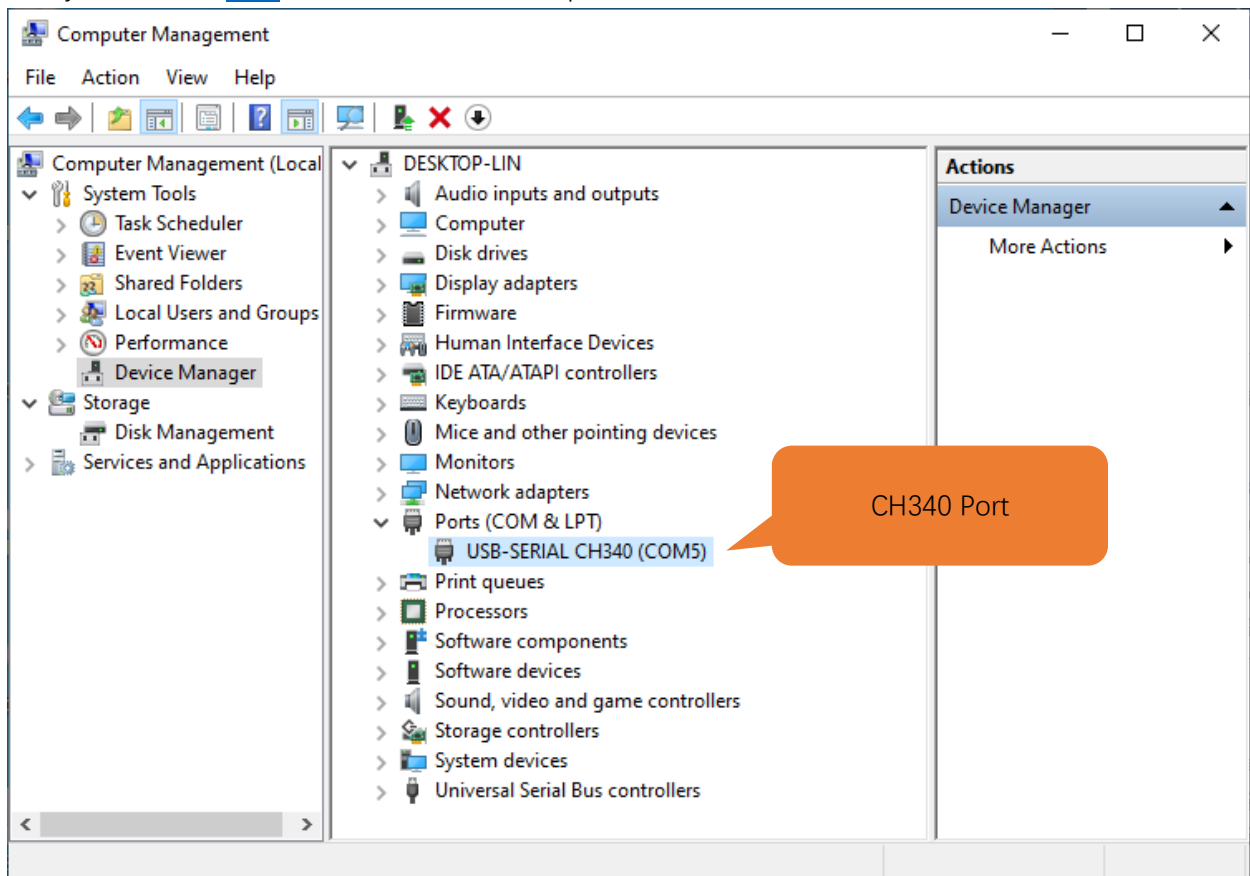
1. Connect your computer and ESP32 with a USB cable.



2. Turn to the main interface of your computer, select "This PC" and right-click to select "Manage".



3. Click “Device Manager”. If your computer has installed CH340, you can see “USB-SERIAL CH340 (COMx)”. And you can click [here](#) to move to the next step.



Installing CH340

- First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

index / search / search CH340

All (14)

Downloads (7)

Products (4)

Application (2)

Video (1)

News (0)

keyword CH340

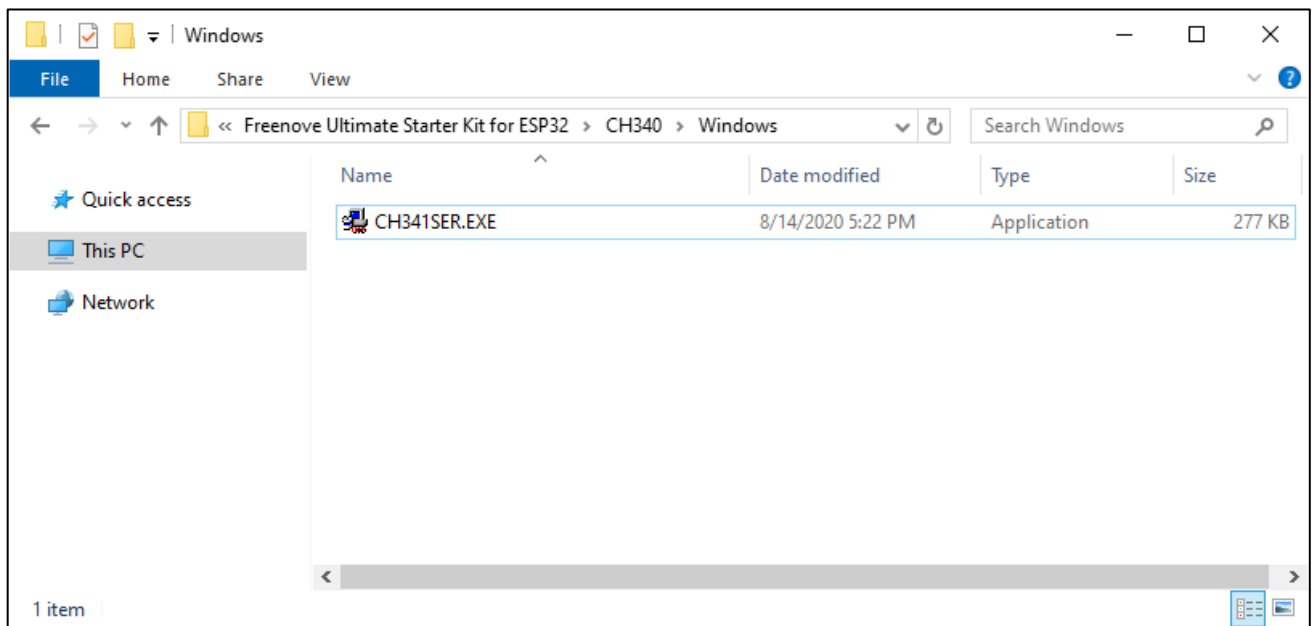
Downloads(7)

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library, File (Host Driver), App Demo Example (USB to UART Den...	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05
Others			
PRODUCT_GUIDE.P...	Electronic selection of product selection manual, please refer to related product technical manual for more technical information.	1.4	2018-12-29
InstallNoteOn64...	Instructions for the driver after 18 years of August cannot be installed under some 64-bit WIN7 (English)	1.0	2019-01-10

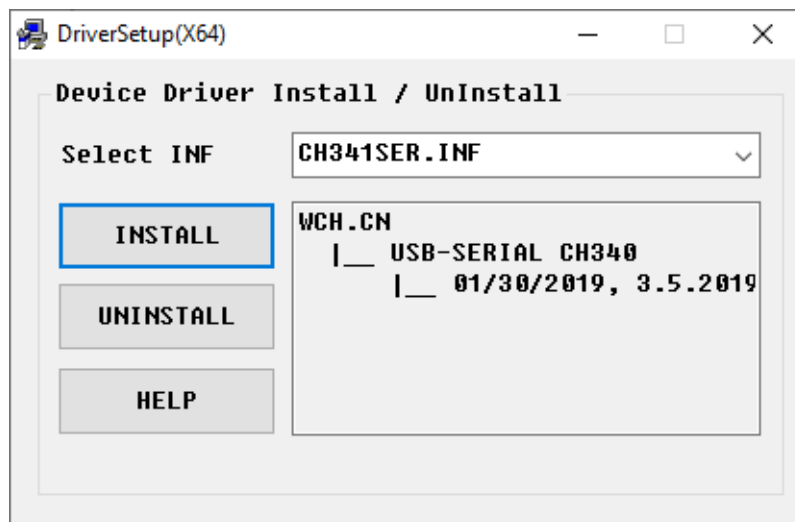
You can also open “Freenove_ESP32_WROVER_Board/CH340”, we have prepared the installation package.

Name	Date modified	Type	Size
Linux	8/14/2020 5:24 PM	File folder	
MAC	8/14/2020 5:23 PM	File folder	
Windows	8/14/2020 5:23 PM	File folder	

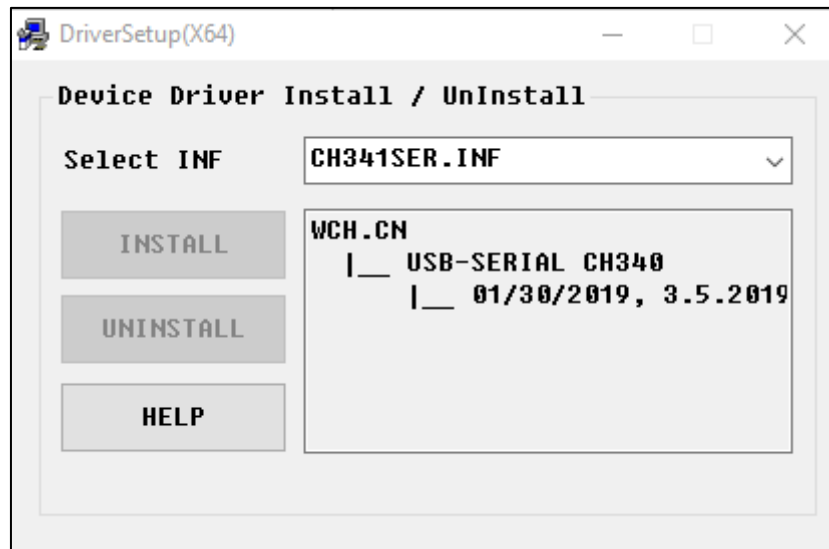
2. Open the folder "Freenove_ESP32_WROVER_Board/CH340/Windows/ch341ser"



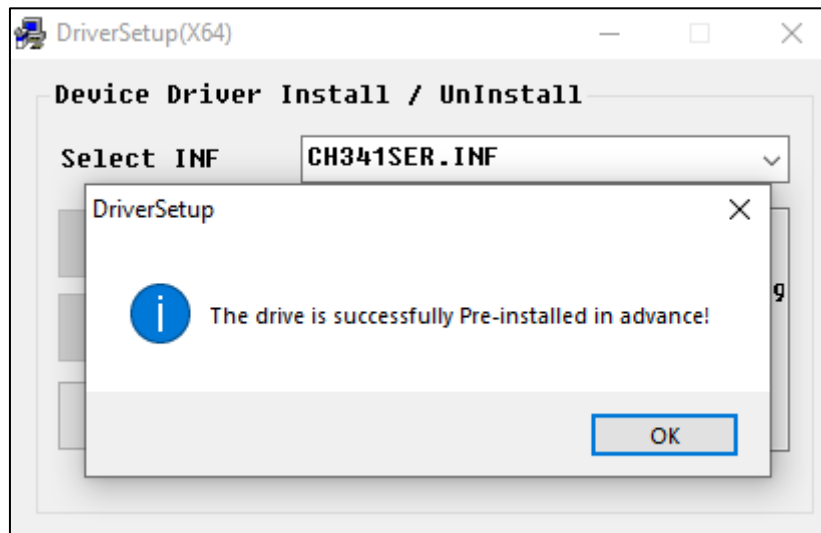
3. Double click "CH341SER.EXE".



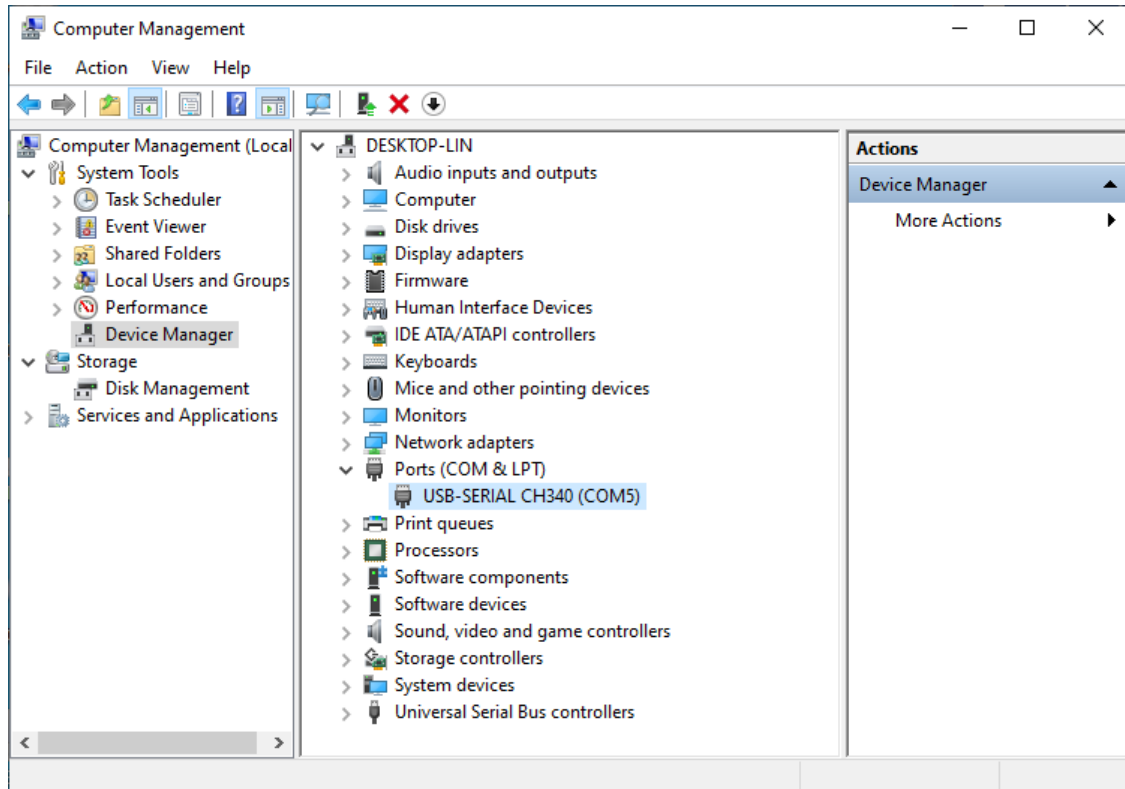
4. Click "INSTALL" and wait for the installation to complete.



5. Install successfully. Close all interfaces.



6. When ESP32 is connected to computer, select "This PC", right-click to select "Manage" and click "Device Manager" in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

MAC

First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

The screenshot shows the WCH website with a search bar containing 'ch340'. The left sidebar lists categories: All (14), Downloads (7), Products (4), Application (2), Video (1), and News (0). The main content area shows 'keyword ch340' and 'Downloads(7)'. A table lists the following files:

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library file (Java Driver), App Demo Example, and Android Demo SDK.	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05

Orange callout boxes highlight the following files:

- Windows: CH341SER.EXE
- Linux: CH341SER_LINUX...
- MAC: CH341SER_MAC.ZI...

If you would not like to download the installation package, you can open “Freenove_ESP32_WROVER_Board/CH340”, we have prepared the installation package.

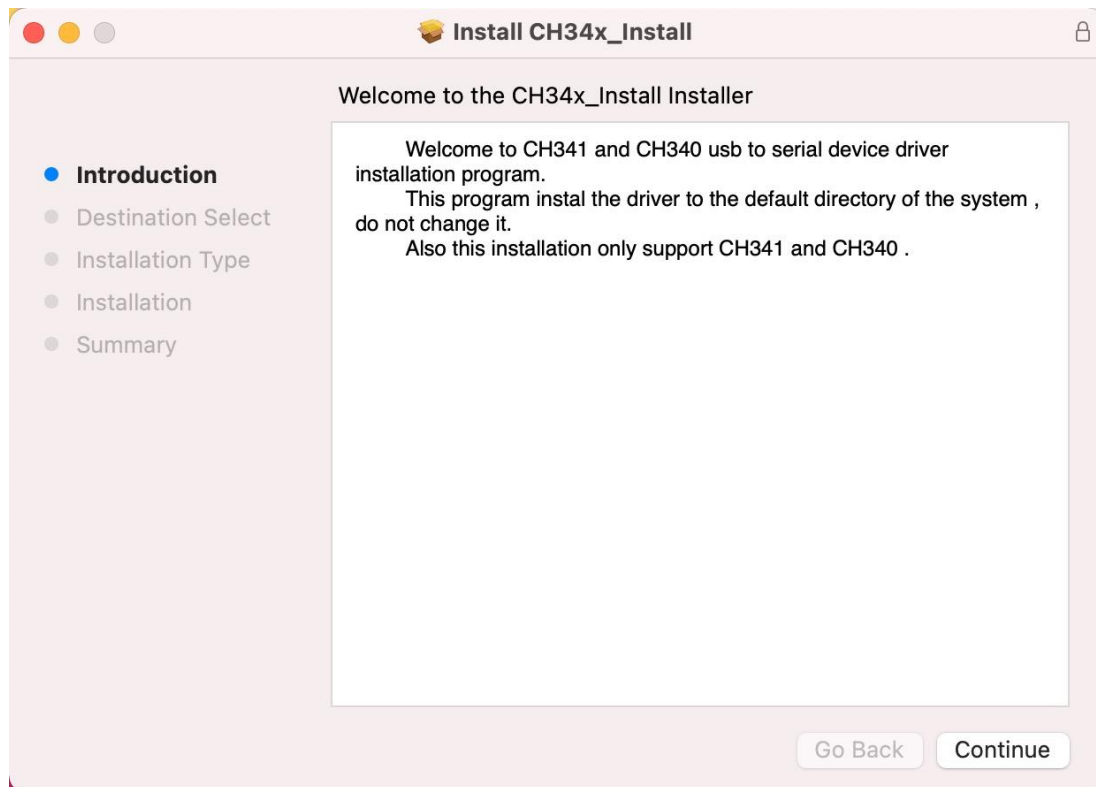
Second, open the folder “Freenove_ESP32_WROVER_Board/CH340/MAC/”

The screenshot shows a macOS Finder window titled 'MAC'. The left sidebar shows the 'Favorites' section with 'Downloads' selected. The main pane displays the following files:

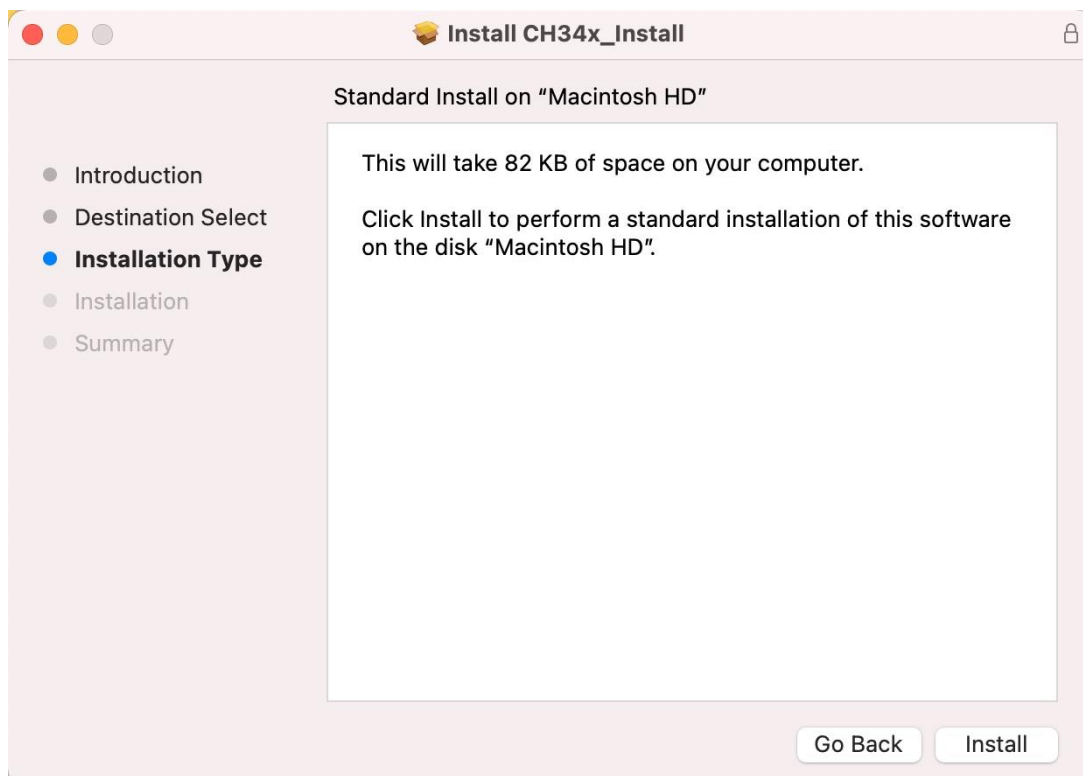
Name	Modified	Size	Kind
CH34x_Install_V1.5.pkg	Dec 8, 2016 at 2:00 PM	26 KB	Installer package
ReadMe.pdf	Dec 8, 2016 at 4:09 PM	146 KB	Adobe...ocument

An orange callout box points to the 'CH34x_Install_V1.5.pkg' file with the text 'Run it.'.

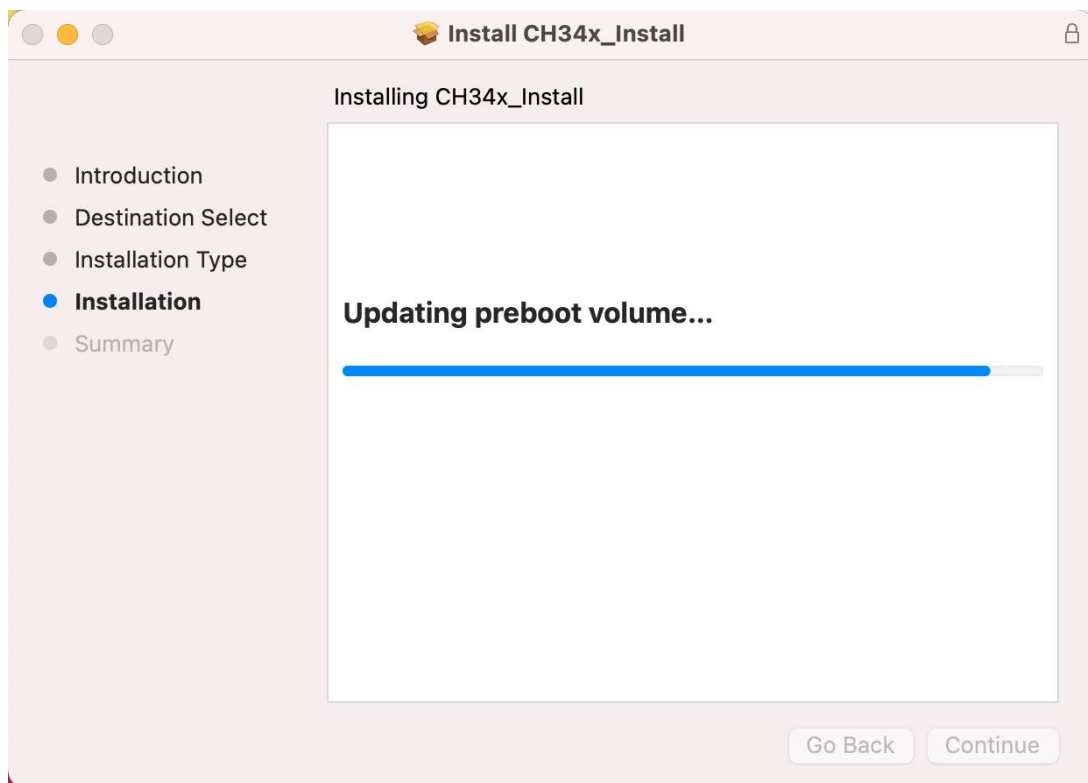
Third, click Continue.



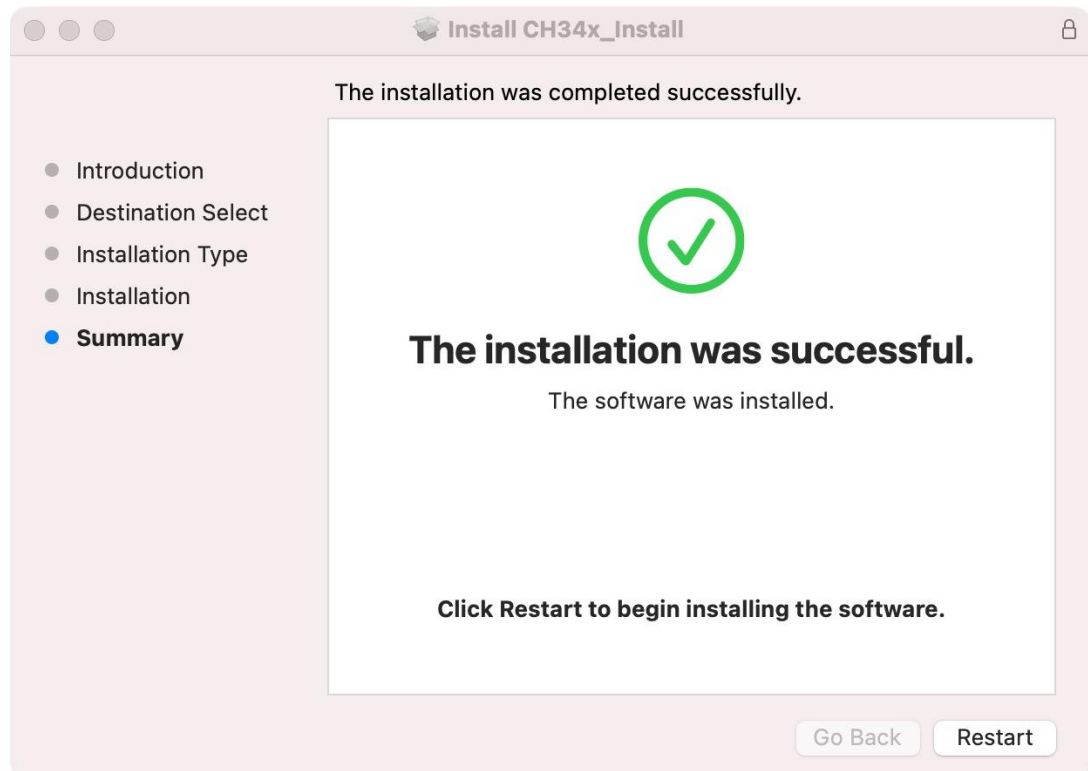
Fourth, click Install.



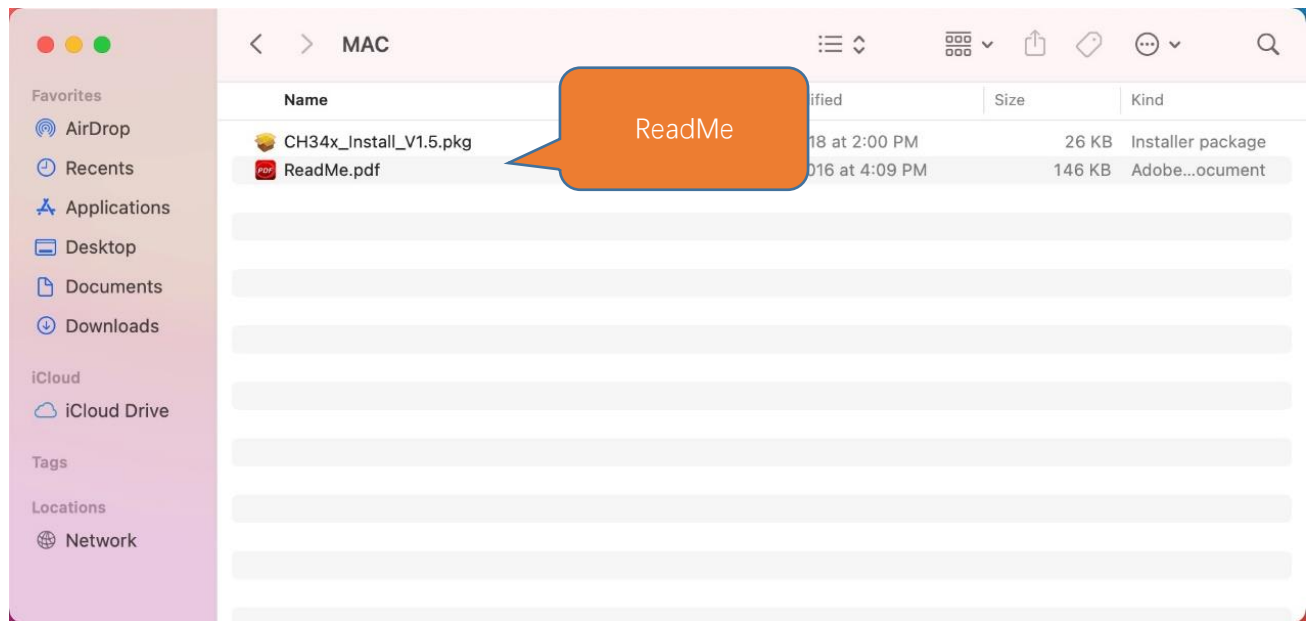
Then, waiting Finish.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.



0.4 Burning Micropython Firmware (Important)

To run Python programs on ESP32, we need to burn a firmware to ESP32 first.

Downloading Micropython Firmware

Official website of microPython: <http://micropython.org/>

Webpage listing firmware of microPython for ESP32: <https://micropython.org/download/esp32spiram/>

Firmware

Releases

v1.18 (2022-01-17) .bin [.elf] [.map] [Release notes] (latest)
v1.17 (2021-09-02) .bin [.elf] [.map] [Release notes]
v1.16 (2021-06-23) .bin [.elf] [.map] [Release notes]
v1.15 (2021-04-18) .bin [.elf] [.map] [Release notes]
v1.14 (2021-02-02) .bin [.elf] [.map] [Release notes]
v1.13 (2020-09-02) .bin [.elf] [.map] [Release notes]
v1.12 (2019-12-20) .bin [.elf] [.map] [Release notes]

Nightly builds

v1.18-382-g014912daa (2022-04-27) .bin [.elf] [.map]
v1.18-377-geb9674822 (2022-04-26) .bin [.elf] [.map]
v1.18-370-g28e7e15c0 (2022-04-22) .bin [.elf] [.map]
v1.18-366-gef1c2cdab (2022-04-21) .bin [.elf] [.map]

Firmware (Compiled with IDF 3.x)

Releases

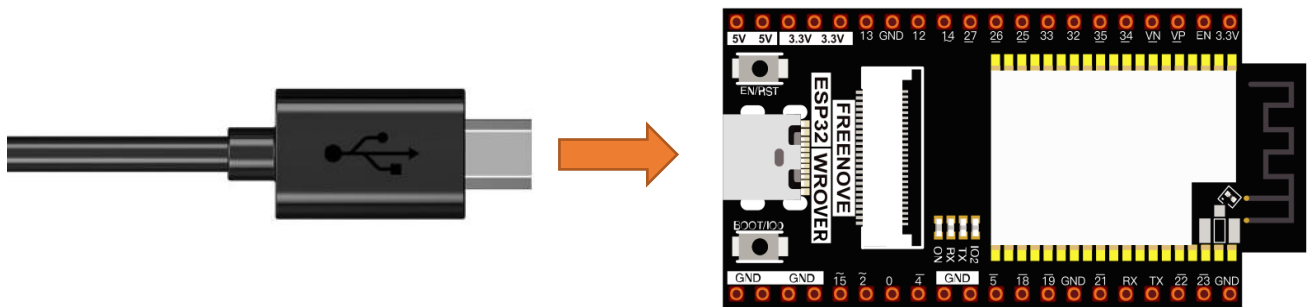
v1.14 (2021-02-02) .bin [.elf] [.map] [Release notes] (latest)
v1.13 (2020-09-02) .bin [.elf] [.map] [Release notes]
v1.12 (2019-12-20) .bin [.elf] [.map] [Release notes]
v1.11 (2019-05-29) .bin [.elf] [.map] [Release notes]
v1.10 (2019-01-25) .bin [.elf] [.map] [Release notes]
v1.9.4 (2018-05-11) .bin [.elf] [.map] [Release notes]

Firmware used in this tutorial is **esp32spiram-20220117-v1.18.bin**

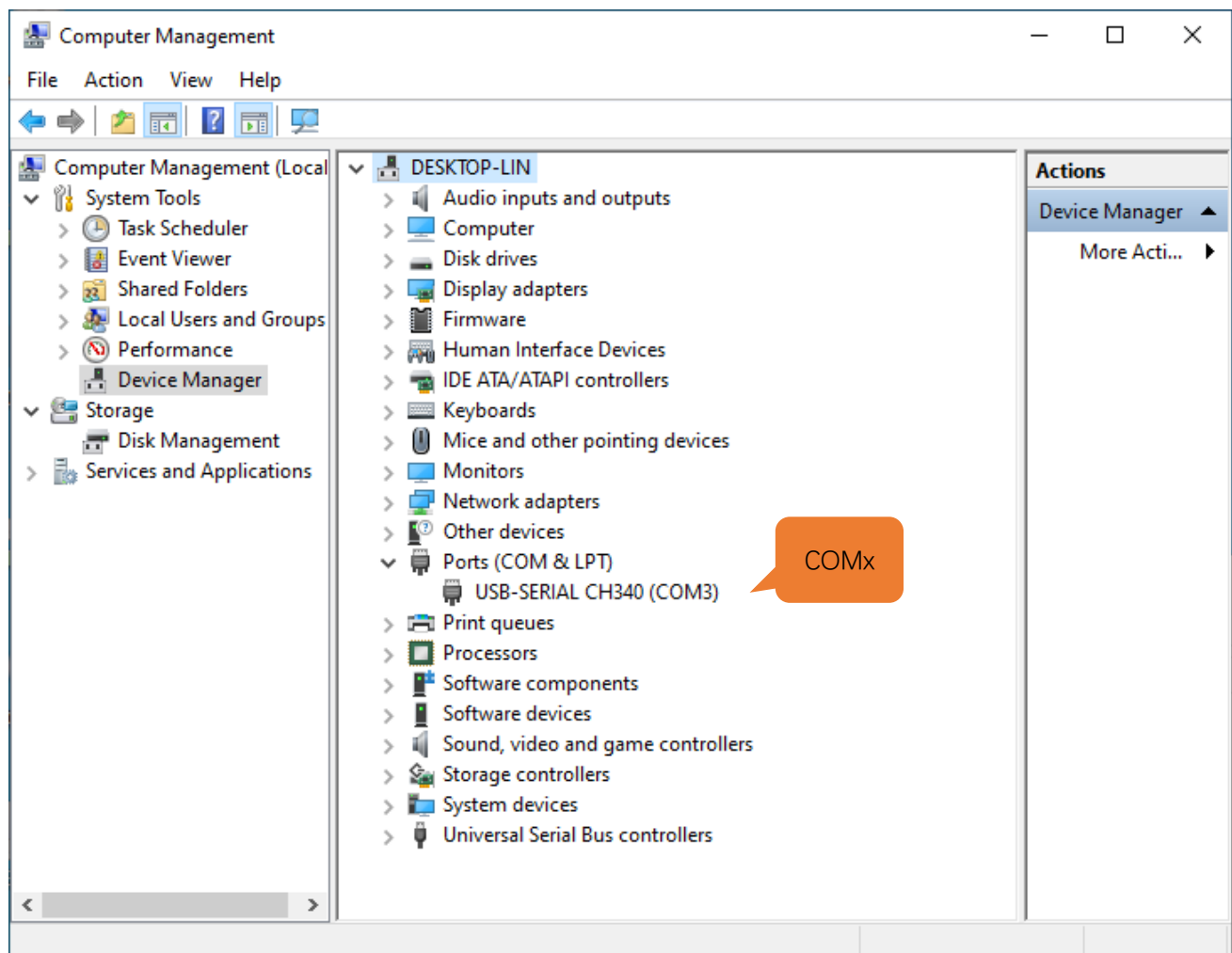
This file is also provided in our data folder "**Freenove_ESP32_WROVER_Board/Python/Python_Firmware**".

Burning a Micropython Firmware

Connect your computer and ESP32 with a USB cable.

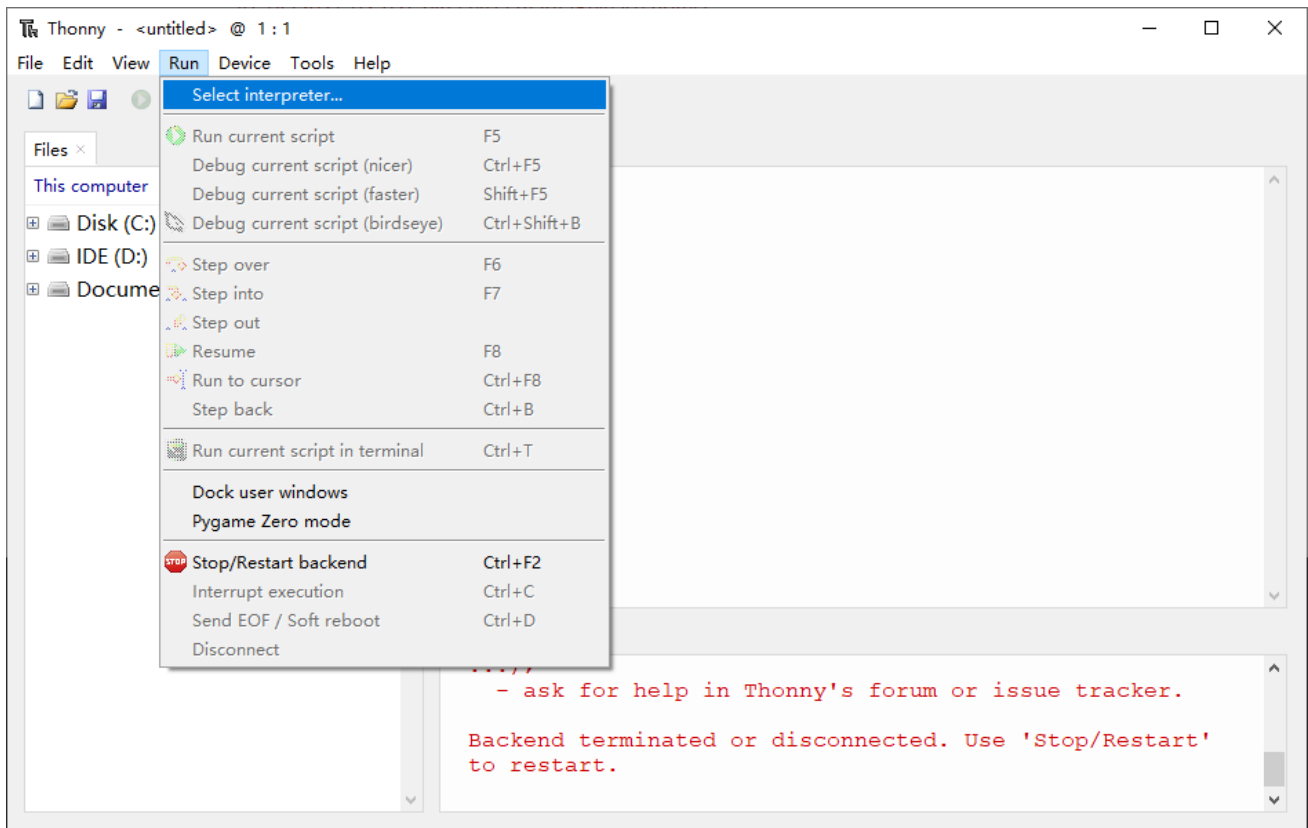


Make sure that the driver has been installed successfully and that it can recognize COM port correctly. Open device manager and expand "Ports".

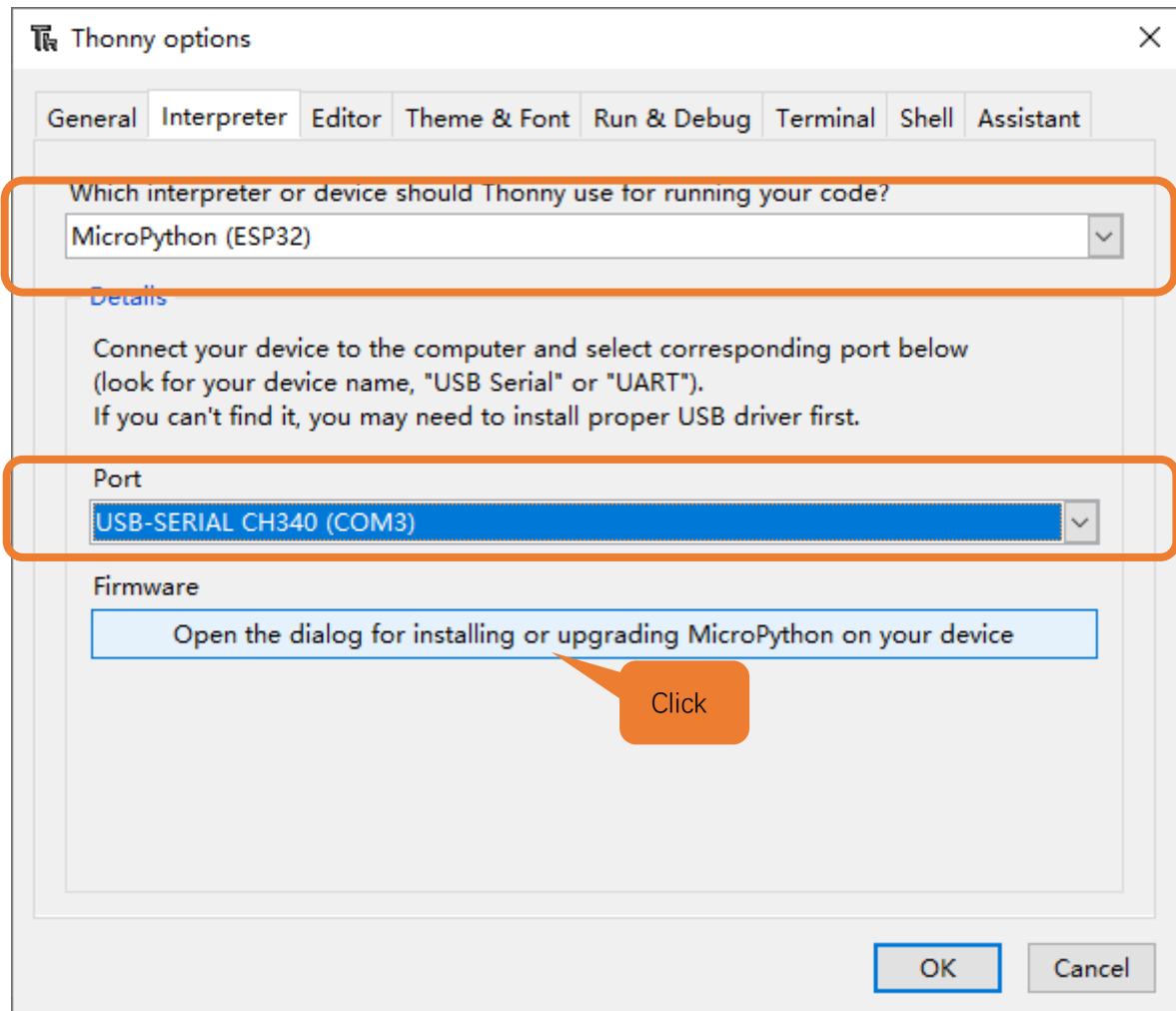


Note: the port of different people may be different, which is a normal situation.

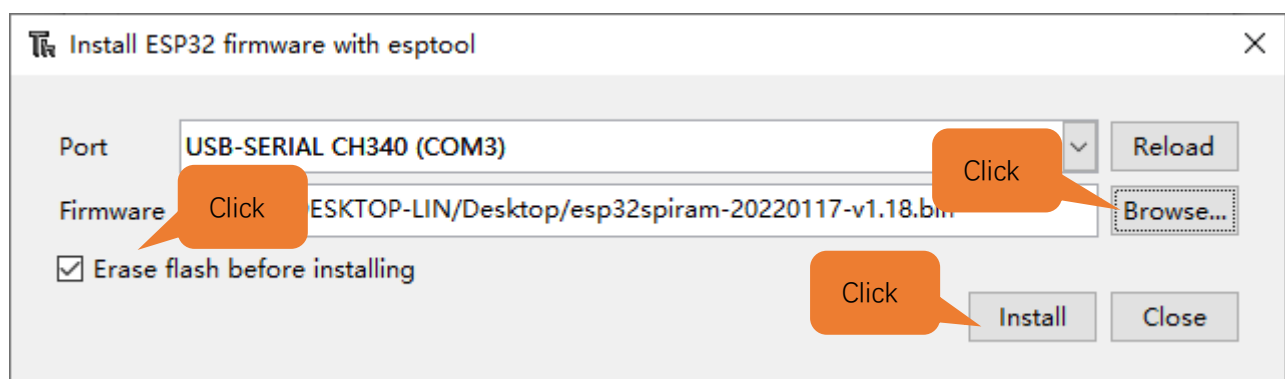
1. Open Thonny, click “run” and select “Select interpreter...”



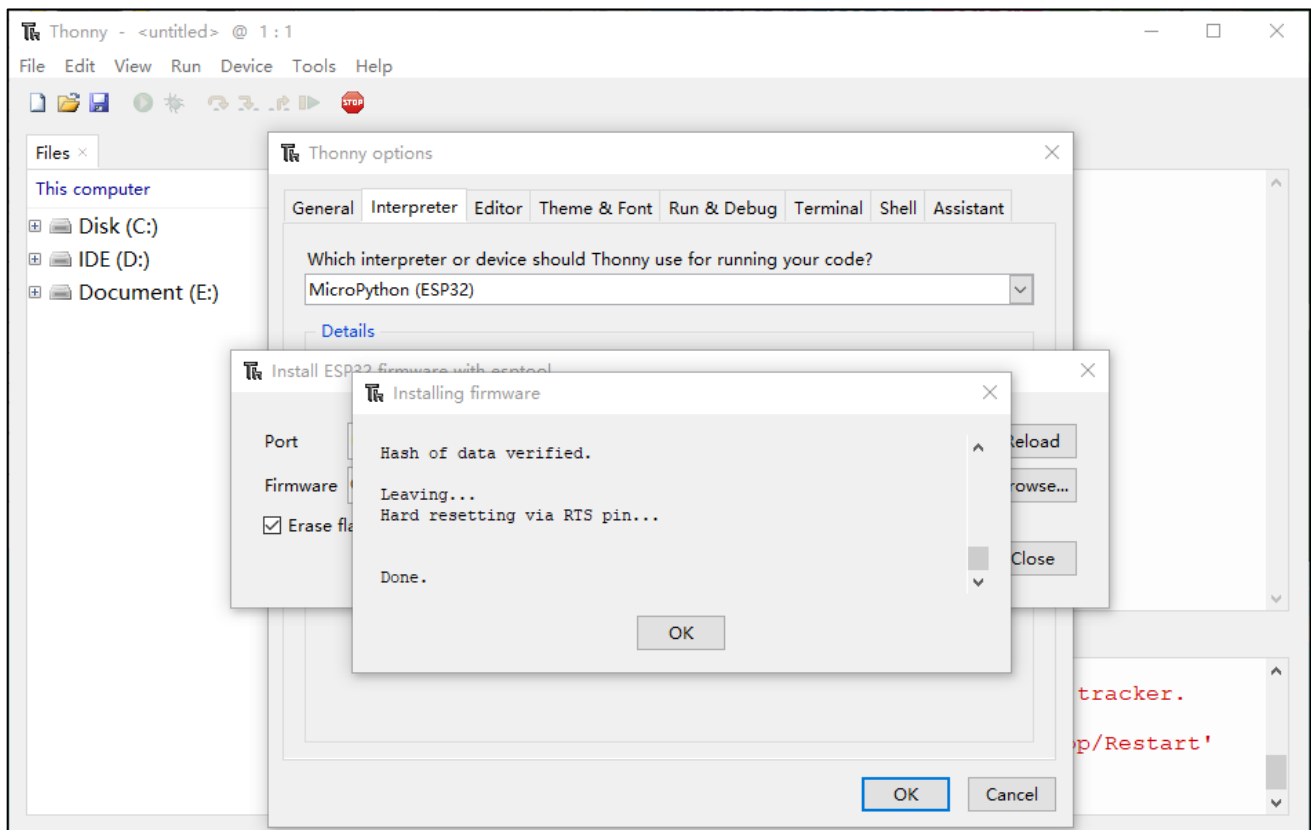
2. Select “Micropython (ESP32)”, select “USB-SERIAL CH340 (COM4)”, and then click the long button under “Firmware”.



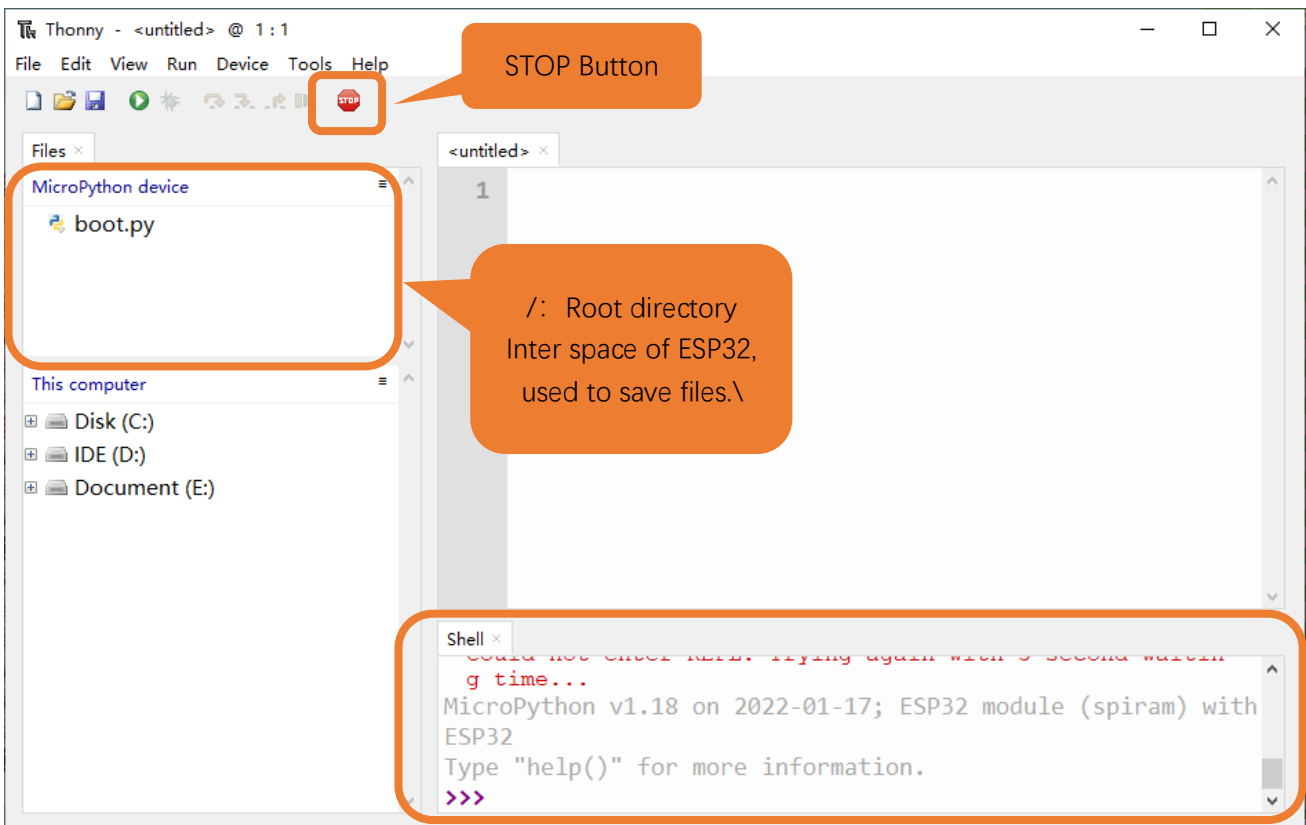
3. The following dialog box pops up. Select “USB-SERIAL CH340 (COM3)” for “Port” and then click “Browse...”. Select the previous prepared microPython firmware “**esp32spiram-20220117-v1.18.bin**”. Check “Erase flash before installing” and click “install” to wait for the prompt of finishing installation.



4. Wait for the installation to be done.



5. Close all dialog boxes, turn to main interface and click "STOP". As shown in the illustration below

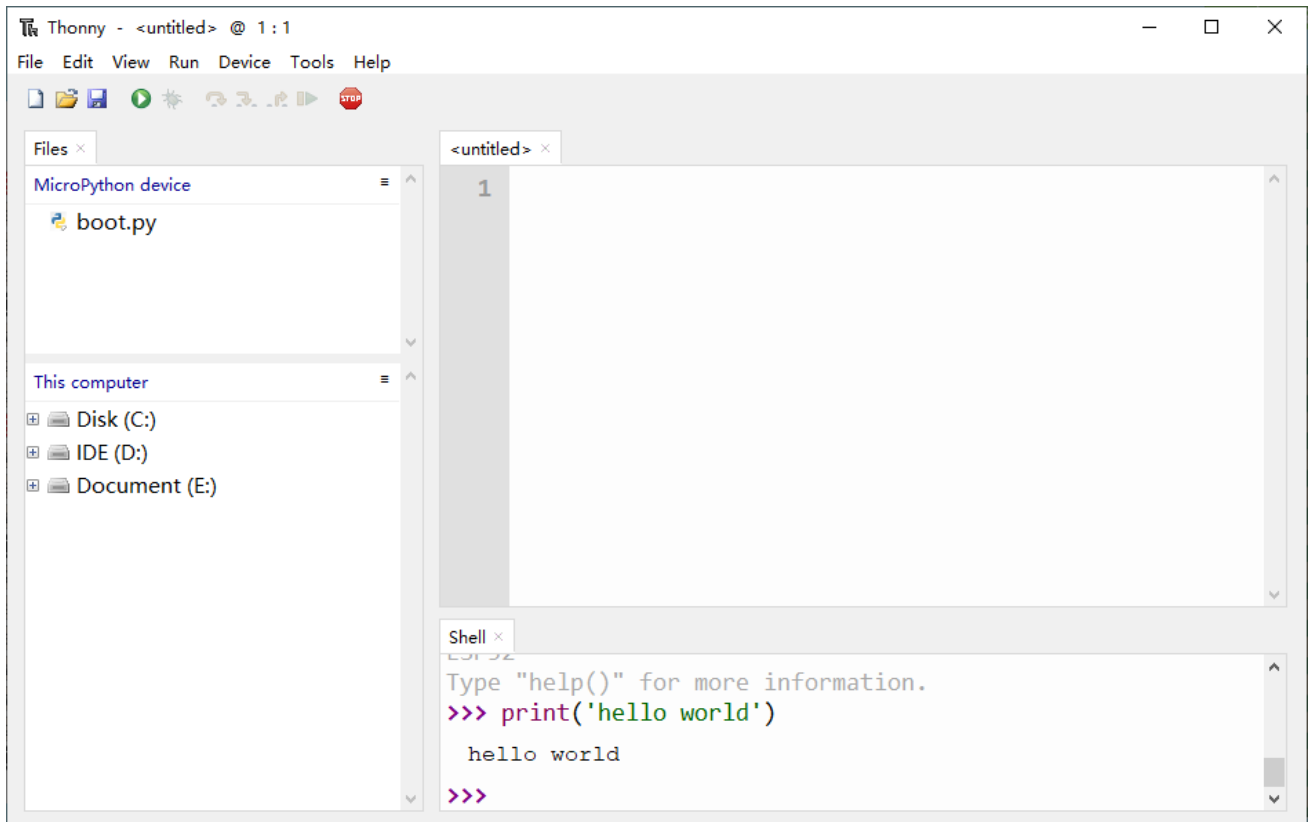


6. So far, all the preparations have been made.

0.5 Testing codes (Important)

Testing Shell Command

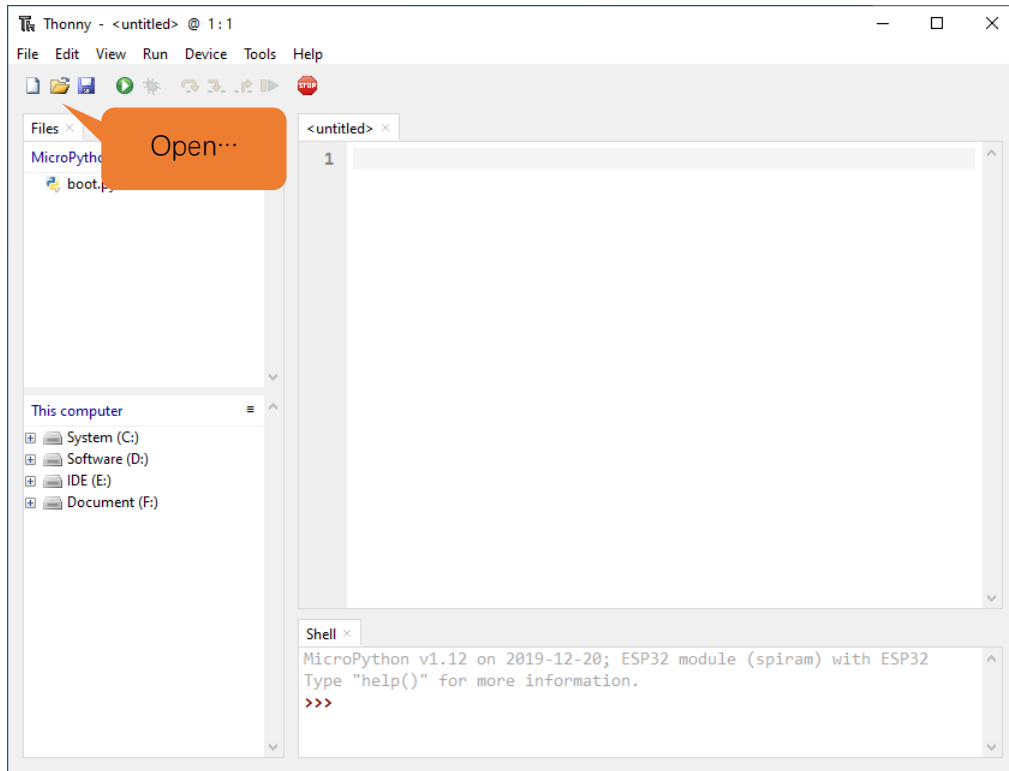
Enter "print('hello world')" in "Shell" and press Enter.



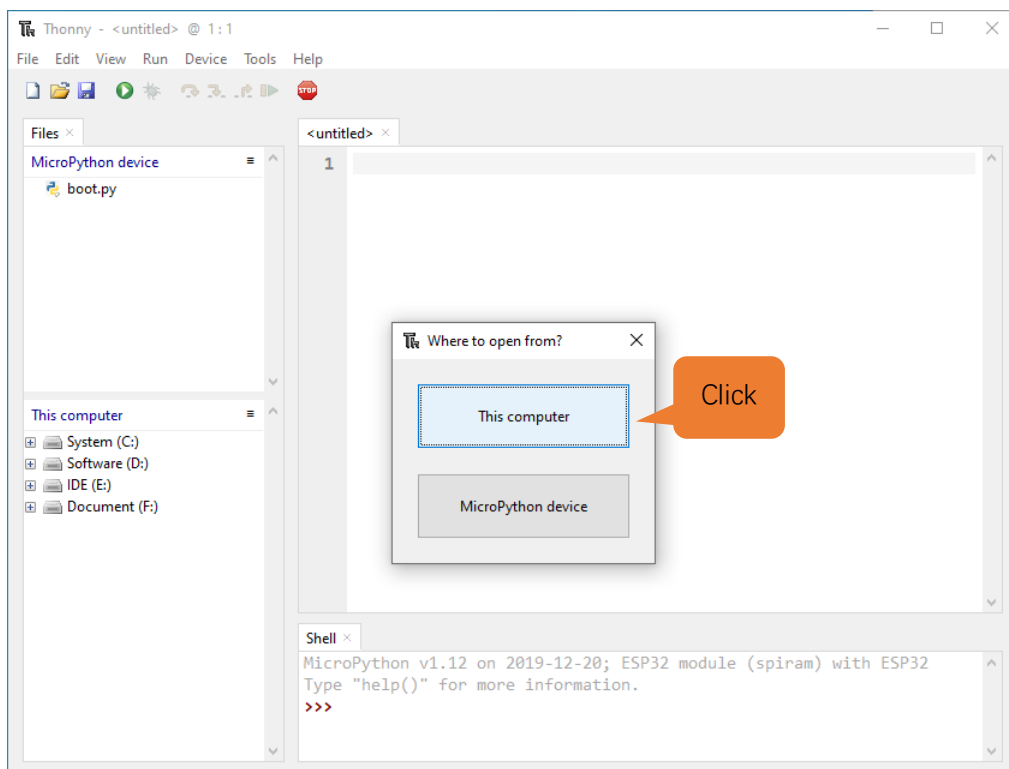
Running Online

ESP32 needs to be connected to a computer when it is run online. Users can use Thonny to write and debug programs.

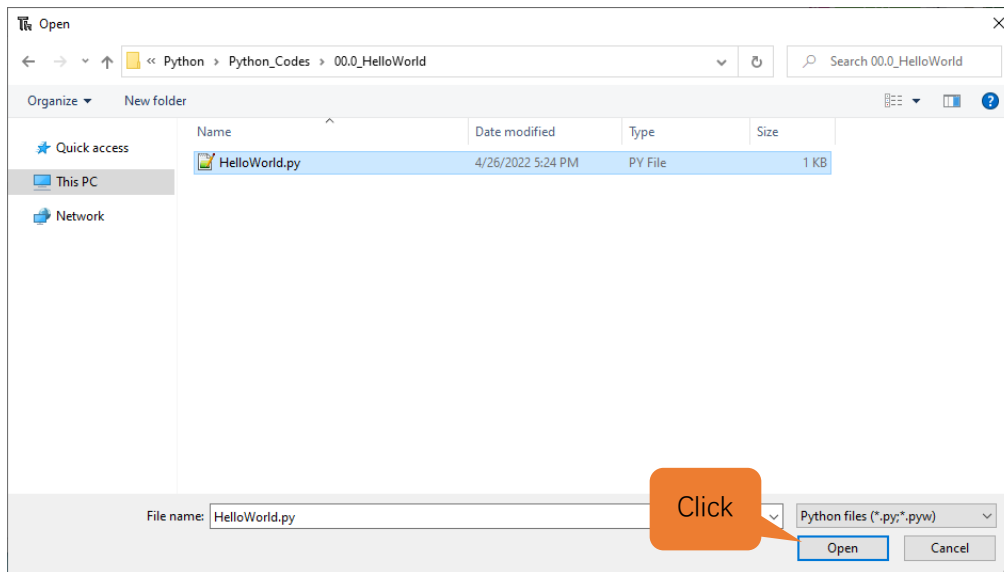
1. Open Thonny and click “Open…”.



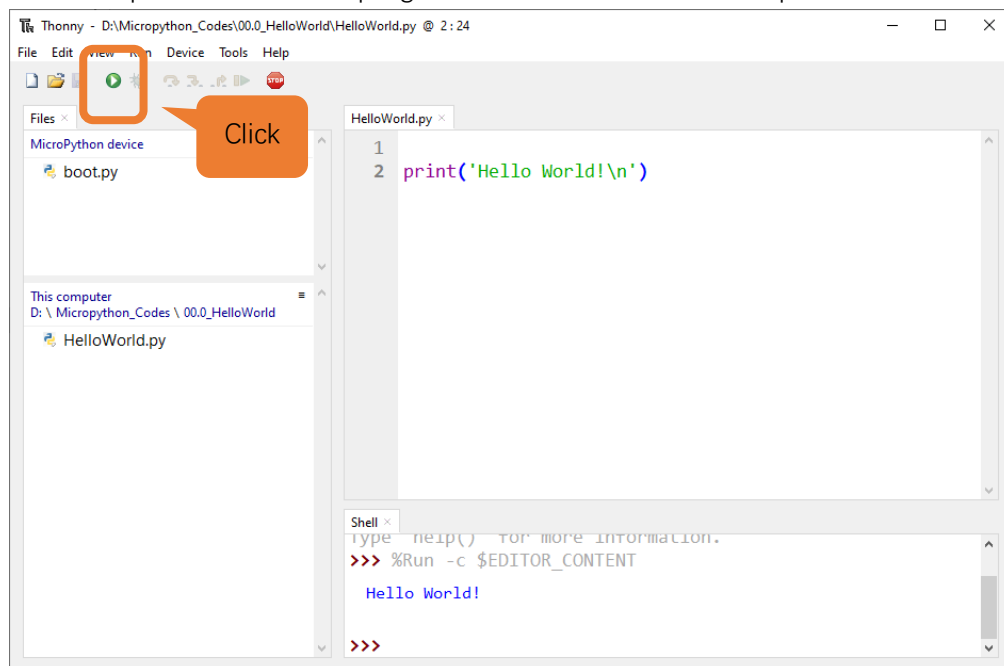
2. On the newly pop-up window, click “This computer”.



In the new dialog box, select “HelloWorld.py” in “Freenove_ESP32_WROVER_Board/Python/Python_Codes/00.0_HelloWorld” folder.



Click “Run current script” to execute the program and “Hello World” will be printed in “Shell”.

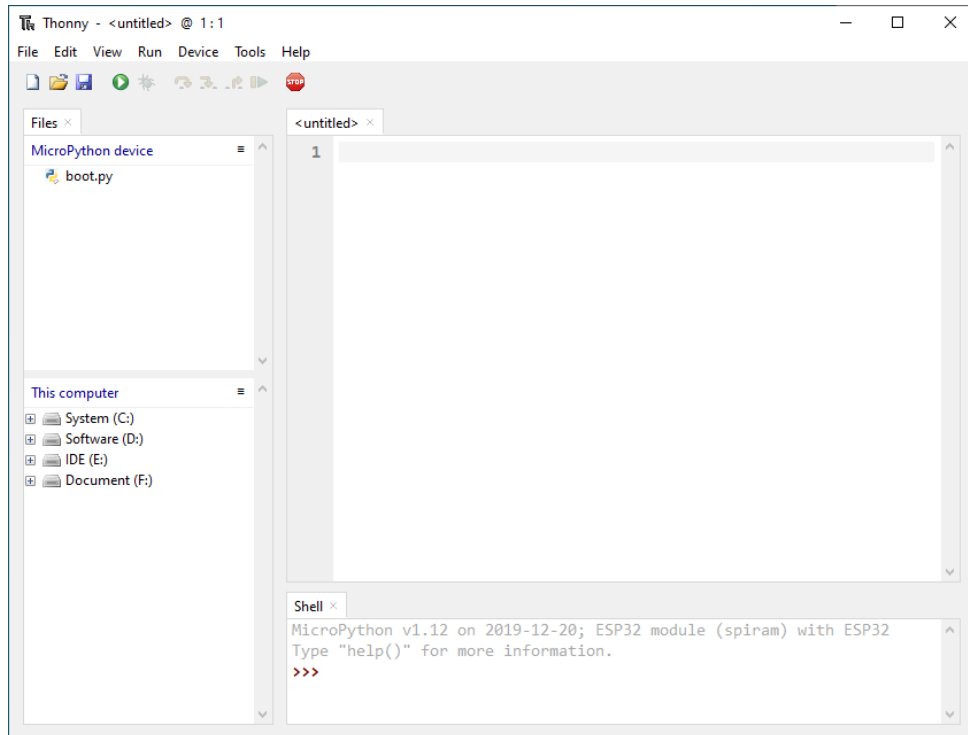


Note: When running online, if you press the reset key of ESP32, user's code will not be executed again. If you wish to run the code automatically after resetting the code, please refer to the following [Running Offline](#).

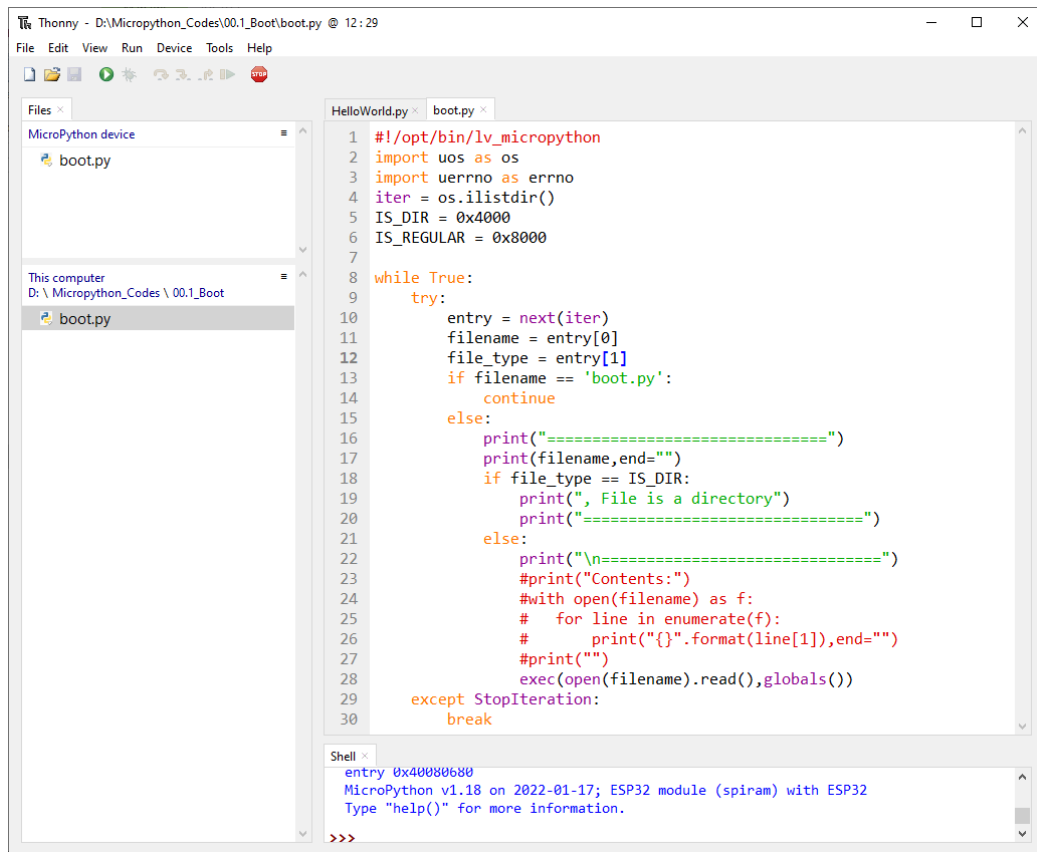
Running Offline (Importance)

After ESP32 is reset, it runs the file `boot.py` in root directory first and then runs file `main.py`, and finally, it enters “Shell”. Therefore, to make ESP32 execute user’s programs after resetting, we need to add a guiding program in `boot.py` to execute user’s code.

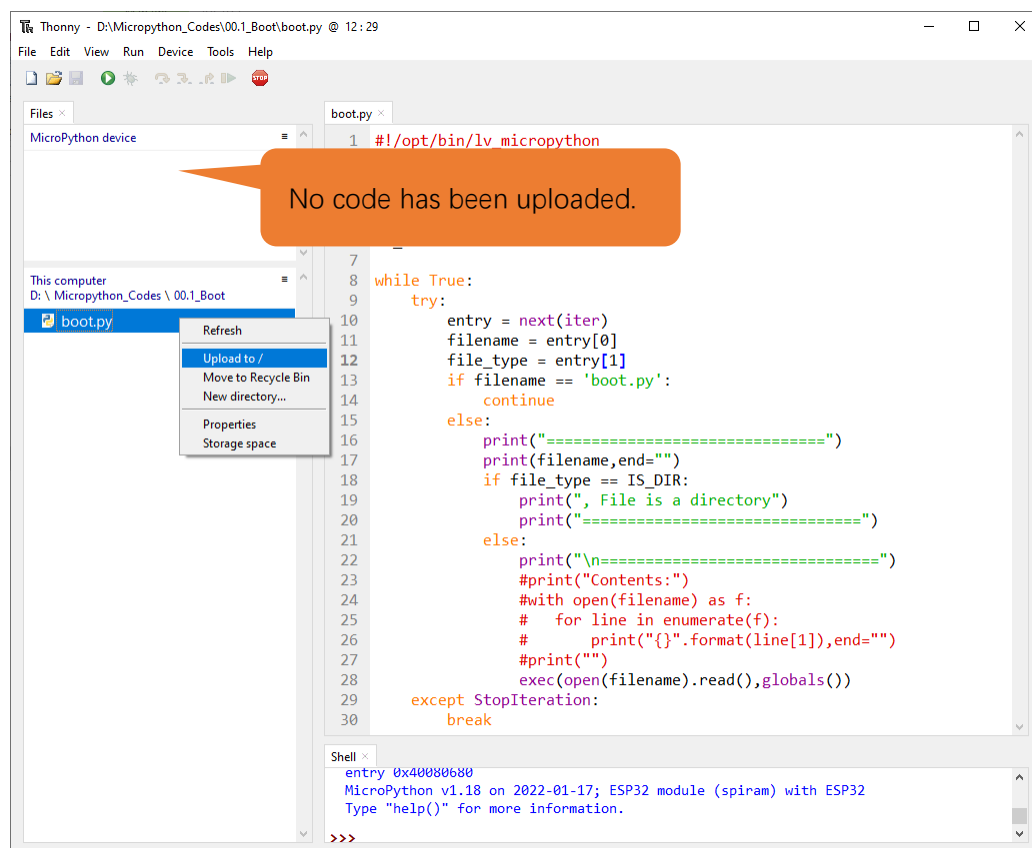
1. Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”. Open “Thonny”。



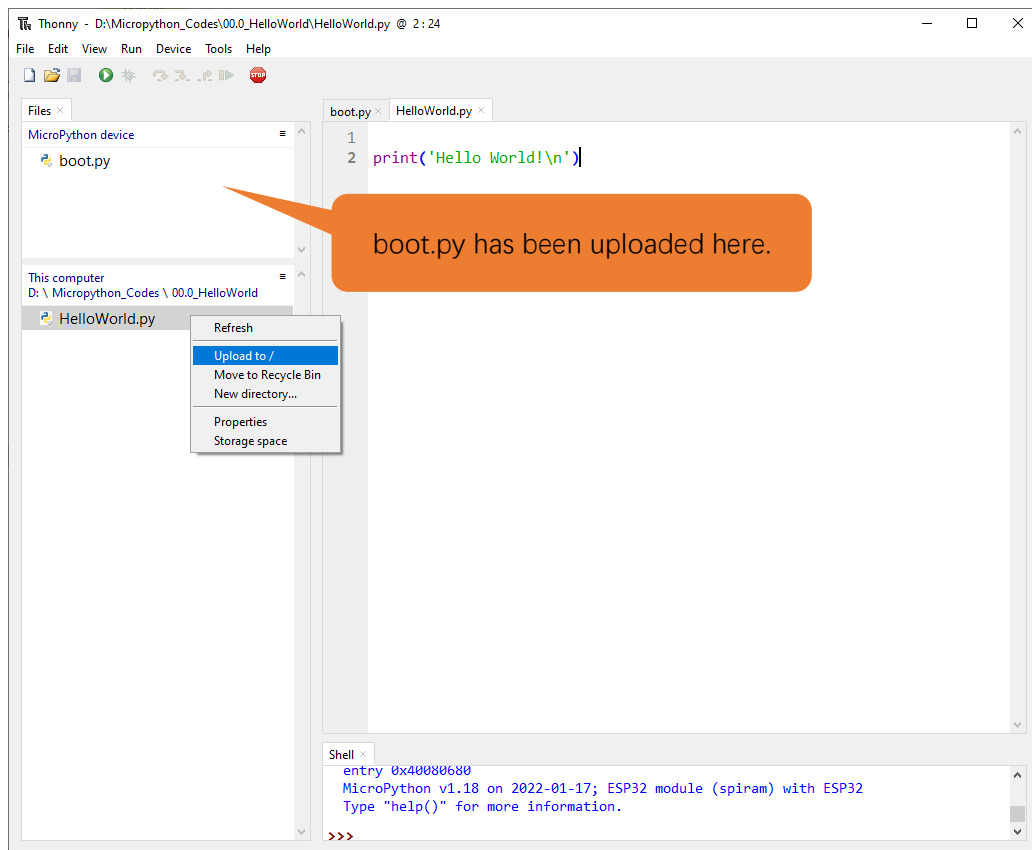
2. Expand “00.1_Boot” in the “Micropython_Codes” in the directory of disk(D), and double-click `boot.py`, which is provided by us to enable programs in “MicroPython device” to run offline.



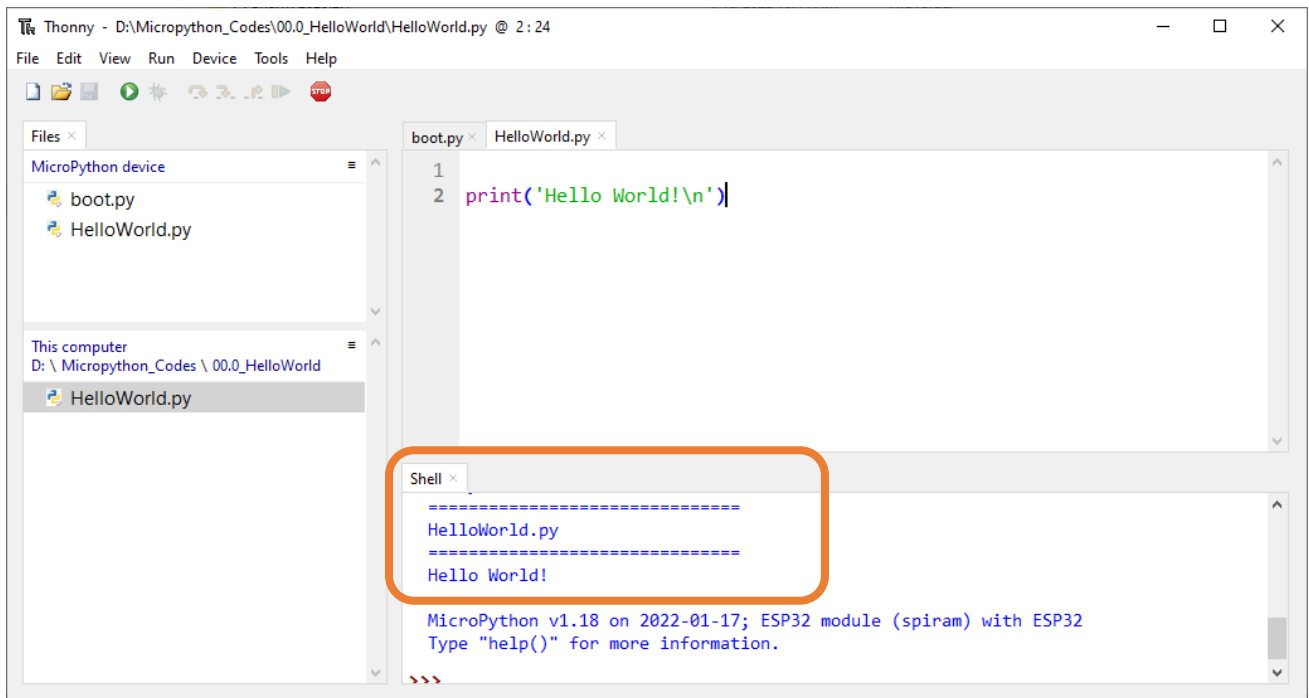
If you want your written programs to run offline, you need to upload boot.py we provided and all your codes to "MicroPython device" and press ESP32's reset key. Here we use programs 00.0 and 00.1 as examples. Select "boot.py", right-click to select "Upload to /".



Similarly, upload “HelloWorld.py” to “MicroPython device”.



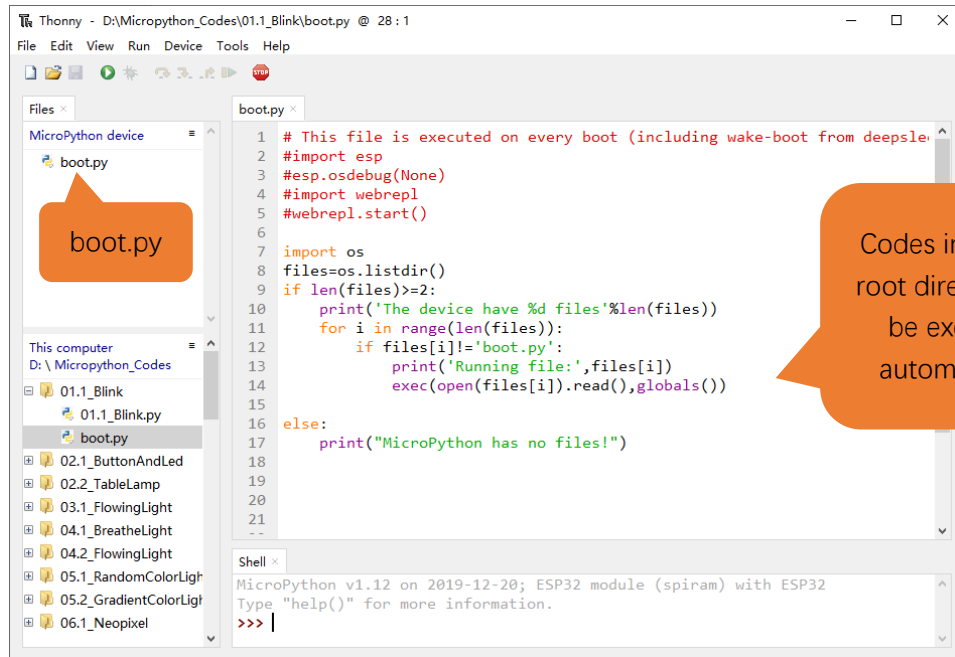
3. Press the reset key and in the box of the illustration below, you can see the code is executed.



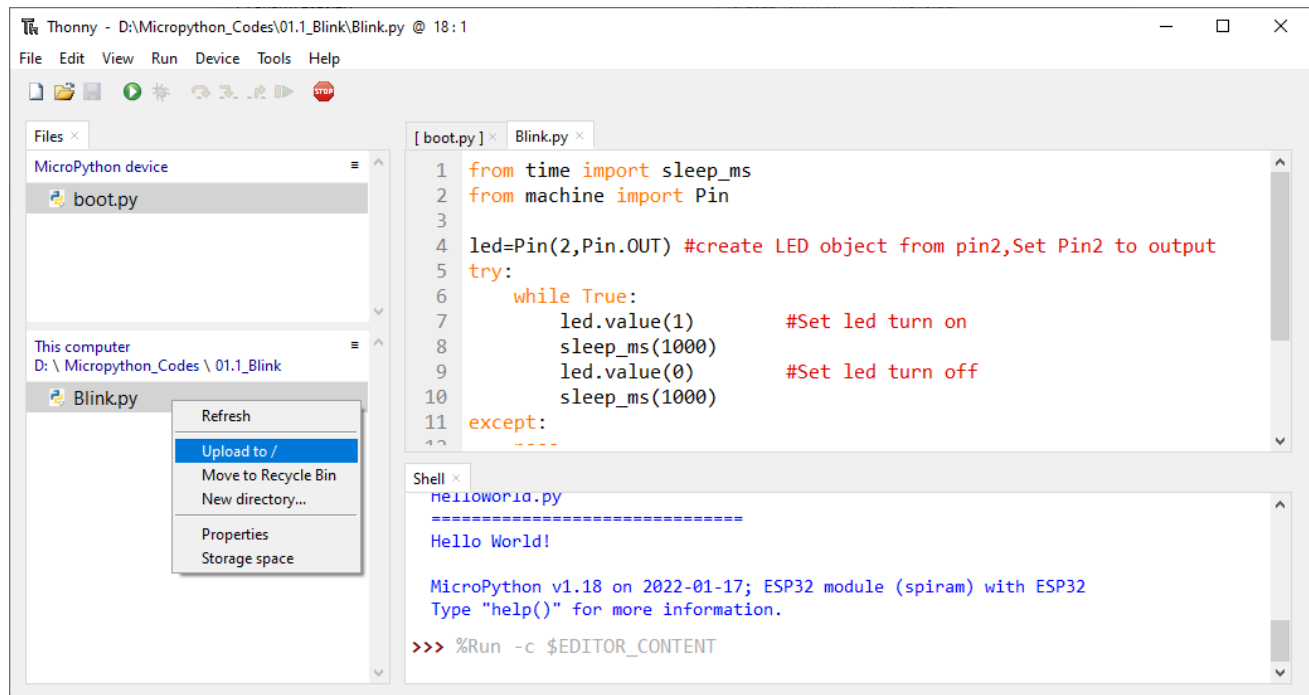
0.6 Thonny Common Operation

Uploading Code to ESP32

Each time when ESP32 restarts, if there is a “boot.py” in the root directory, it will execute this code first.

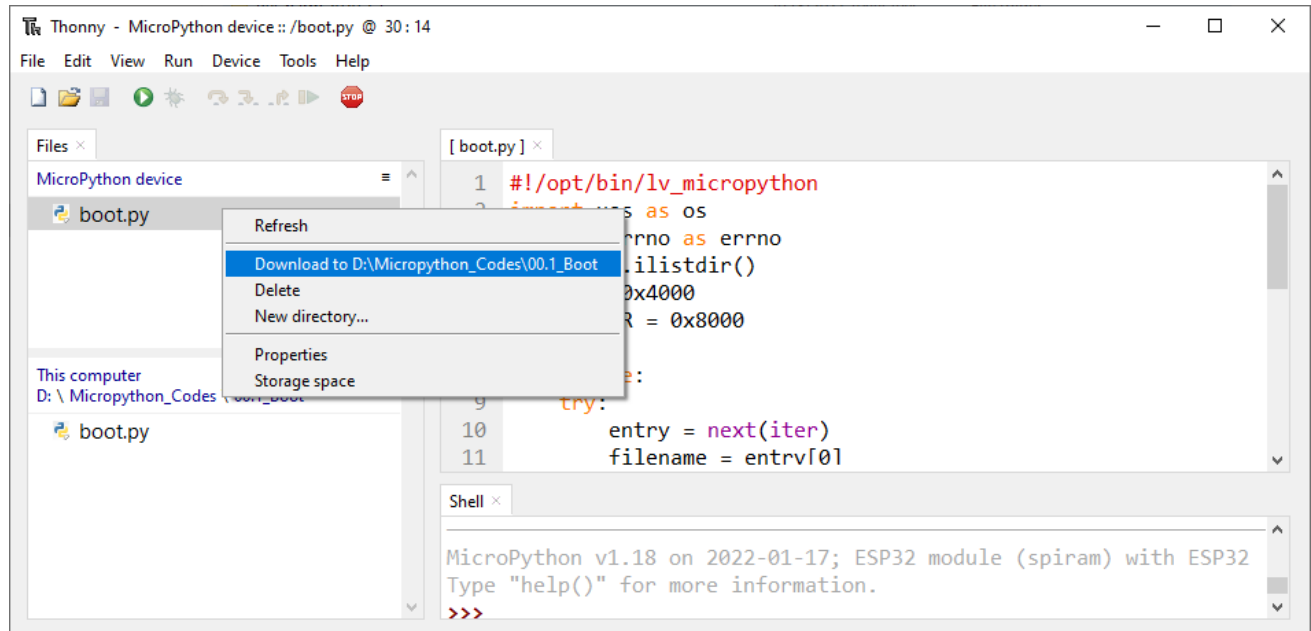


Select “Blink.py” in “01.1_Blink”, right-click your mouse and select “Upload to /” to upload code to ESP32's root directory.



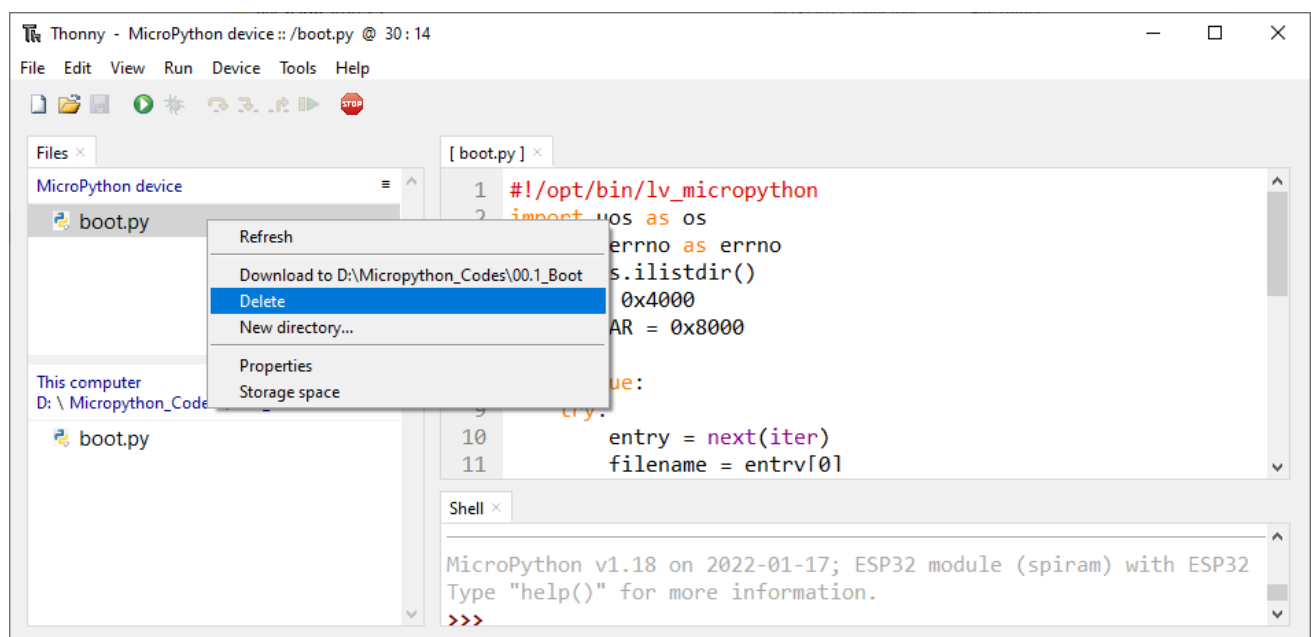
Downloading Code to Computer

Select “boot.py” in “MicroPython device”, right-click to select “Download to ...” to download the code to your computer.



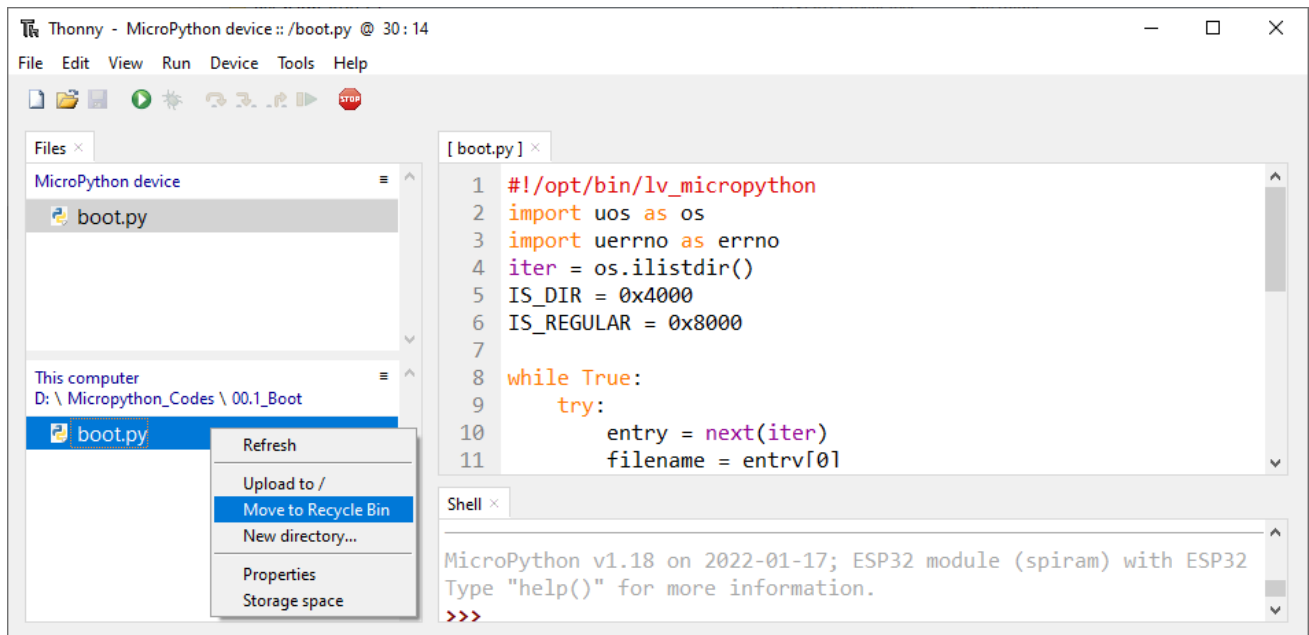
Deleting Files from ESP32's Root Directory

Select “boot.py” in “MicroPython device”, right-click it and select “Delete” to delete “boot.py” from ESP32's root directory.



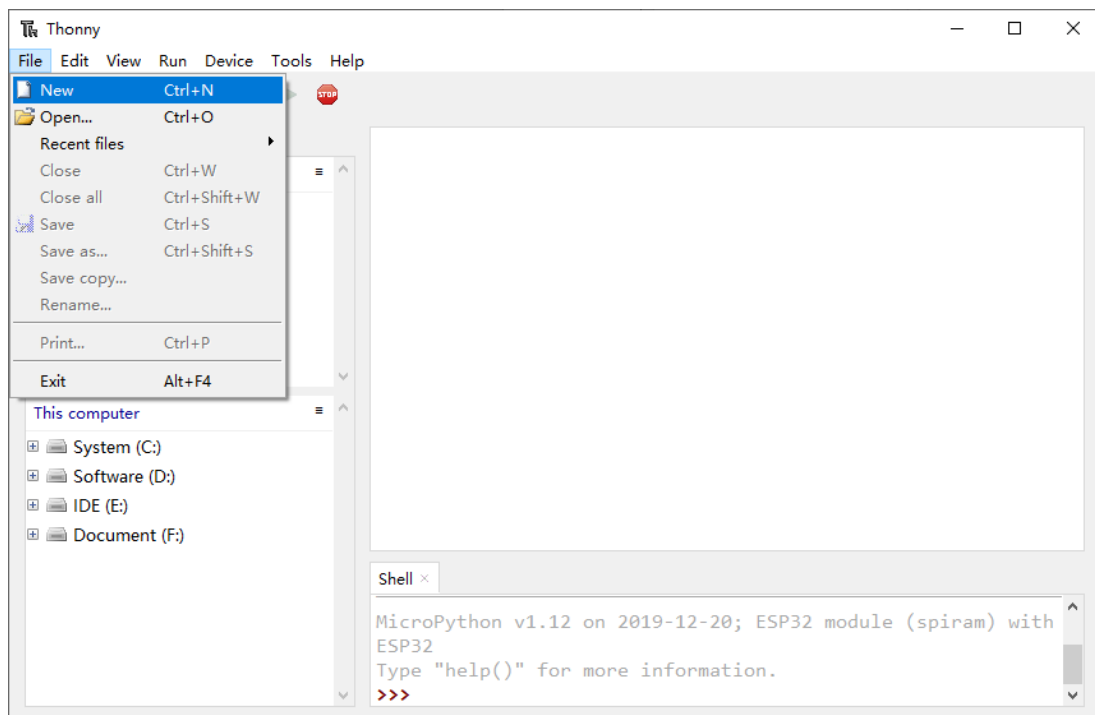
Deleting Files from your Computer Directory

Select “boot.py” in “00.1_Boot”, right-click it and select “Move to Recycle Bin” to delete it from “00.1_Boot”.

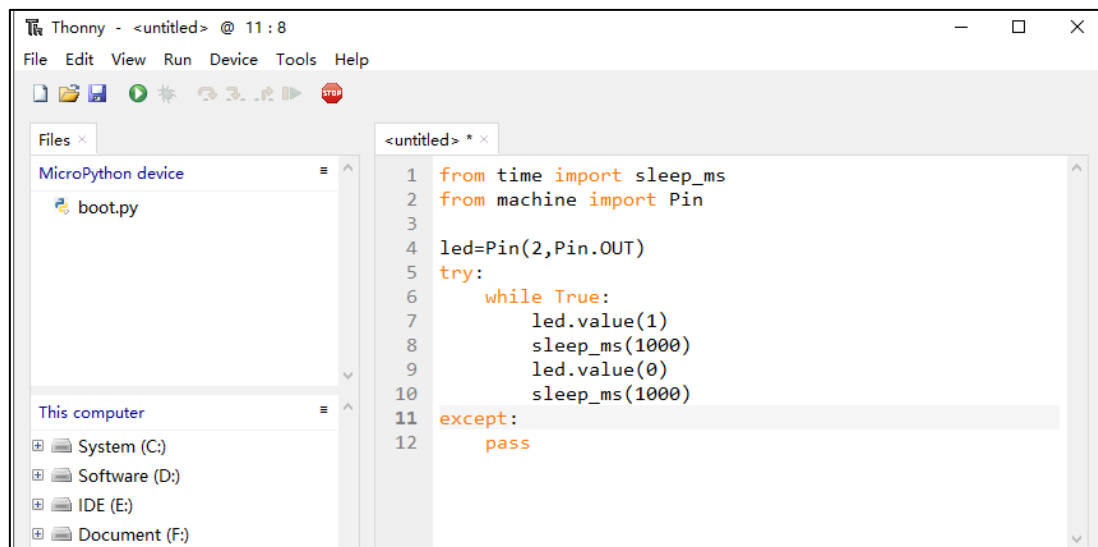


Creating and Saving the code

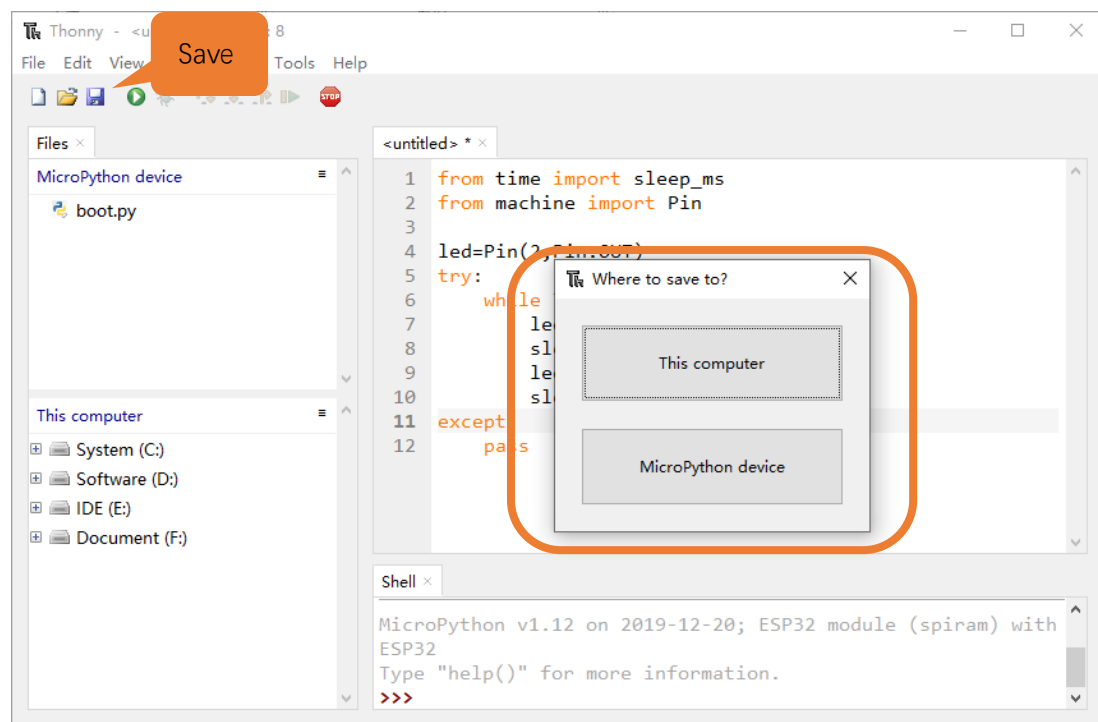
Click “File”→“New” to create and write codes.



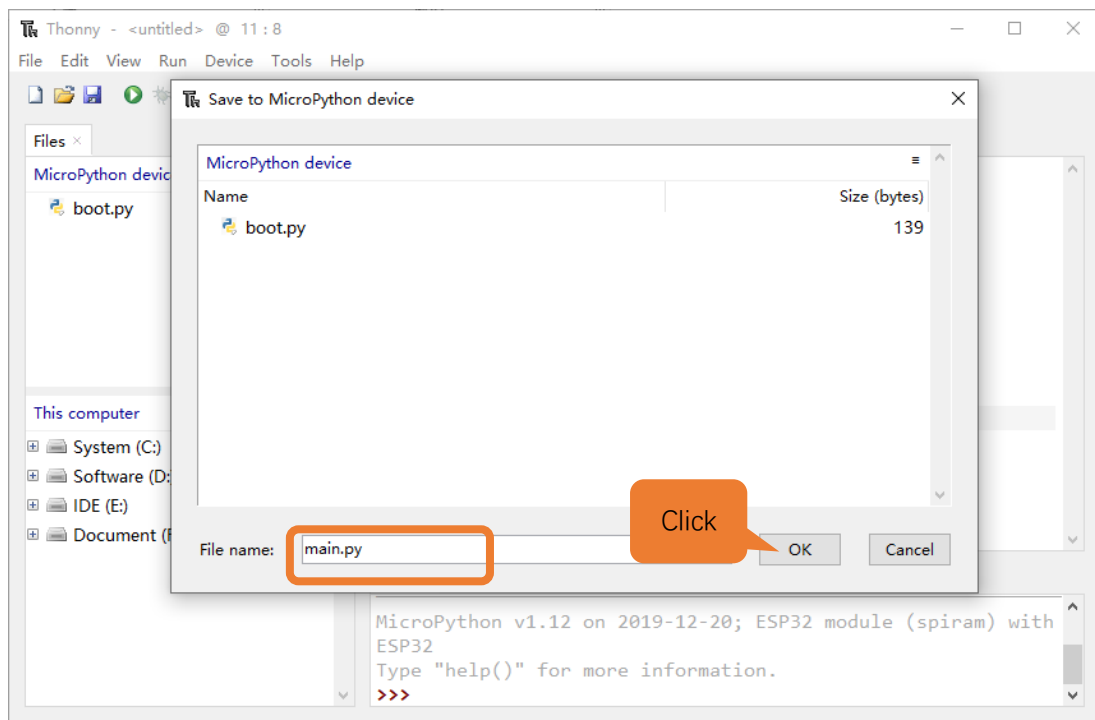
Enter codes in the newly opened file. Here we use codes of “01.1_Blink.py” as an example.



Click “Save” on the menu bar. You can save the codes either to your computer or to ESP32-WROVER.



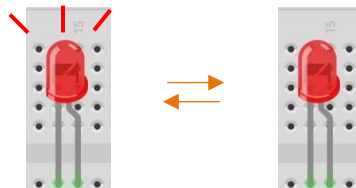
Select "MicroPython device", enter "main.py" in the newly pop-up window and click "OK".



You can see that codes have been uploaded to ESP32-WROVER.



Disconnect and reconnect USB cable, and you can see that LED is ON for one second and then OFF for one second, which repeats in an endless loop.



0.7 Note

Though there are many pins available on ESP32, some of them have been connected to peripheral equipment, so we should avoid using such pins to prevent pin conflicts. For example, when downloading programs, make sure that the pin state of Strapping Pin, when resetting, is consistent with the default level; do NOT use Flash Pin; Do NOT use Cam Pin when using Camera function.

Strapping Pin

The state of Strapping Pin can affect the functions of ESP32 after it is reset, as shown in the table below.

Voltage of Internal LDO (VDD_SDIO)					
Pin	Default	3.3 V		1.8 V	
MTDI	Pull-down	0		1	
Bootling Mode					
Pin	Default	SPI Boot		Download Boot	
GPIO0	Pull-up	1		0	
GPIO2	Pull-down	Don't-care		0	
Enabling/Disabling Debugging Log Print over U0TXD During Bootling					
Pin	Default	U0TXD Active		U0TXD Silent	
MTDO	Pull-up	1		0	
Timing of SDIO Slave					
Pin	Default	Falling-edge Sampling Falling-edge Output	Falling-edge Sampling Rising-edge Output	Rising-edge Sampling Falling-edge Output	Rising-edge Sampling Rising-edge Output
MTDO	Pull-up	0	0	1	1
GPIO5	Pull-up	0	1	0	1

If you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com at any time.

or check: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf

Flash Pin

GPIO6-11 has been used to connect the integrated SPI flash on the module, and is used when GPIO 0 is power on and at high level. Flash is related to the operation of the whole chip, so the external pin GPIO6-11 cannot be used as an experimental pin for external circuits, otherwise it may cause errors in the operation of the program.

GPIO16-17 has been used to connect the integrated PSRAM on the module.

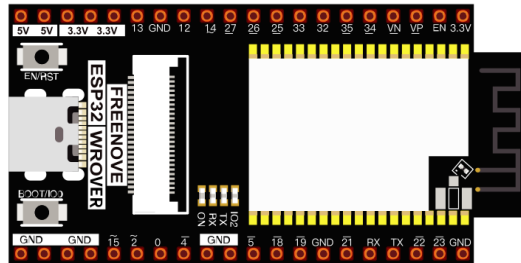
Because of external pull-up, MTDI pin is not suggested to be used as a touch sensor. For details, please refer to Peripheral Interface and Sensor chapter in "[ESP32 Data Sheet](#)".

For more relevant information, please click:

https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf.

Cam Pin

When using the cam camera of our ESP32-WROVER, please check the pins of it. Pins with underlined numbers are used by the cam camera function, if you want to use other functions besides it, please avoid using them.



CAM_Pin	GPIO_pin
I2C_SDA	GPIO26
I2C_SCL	GPIO27
CSI_VYSNC	GPIO25
CSI_HREF	GPIO23
CSI_Y9	GPIO35
XCLK	GPIO21
CSI_Y8	GPIO34
CSI_Y7	GPIO39
CSI_PCLK	GPIO22
CSI_Y6	GPIO36
CSI_Y2	GPIO4
CSI_Y5	GPIO19
CSI_Y3	GPIO5
CSI_Y4	GPIO18

If you have any questions about the information of GPIO, you can click [here](#) to go back to ESP32-WROVER to view specific information about GPIO.

Or check: https://www.espressif.com/sites/default/files/documentation/esp32-wrover_datasheet_en.pdf.

Chapter 1 LED (Important)

This chapter is the Start Point in the journey to build and explore ESP32 electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

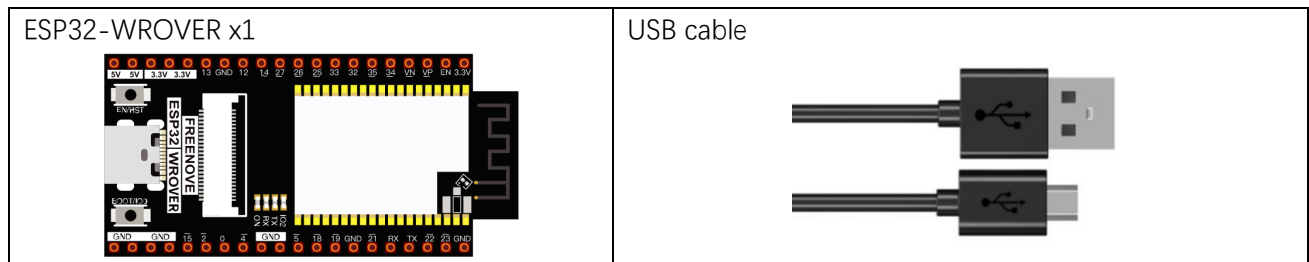
In this project, we will use ESP32 to control blinking a common LED.

If you have not yet installed Thonny, click [here](#).

If you have not yet downloaded Micropython Firmware, click [here](#).

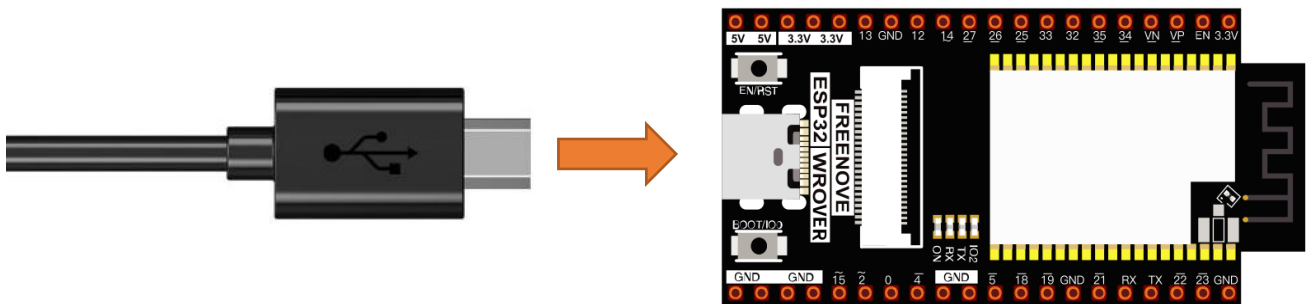
If you have not yet loaded Micropython Firmware, click [here](#).

Component List



Power

ESP32-WROVER needs 5v power supply. In this tutorial, we need connect ESP32-WROVER to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP32-WROVER by default.

In the whole tutorial, we don't use T extension to power ESP32-WROVER. So 5V and 3.3V (including EXT 3.3V) on the extension board are provided by ESP32-WROVER.

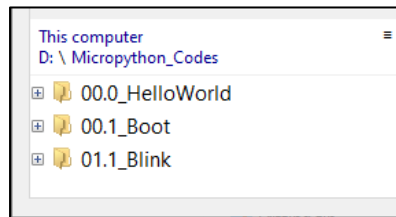
We can also use DC jack of extension board to power ESP32-WROVER. In this way, 5v and EXT 3.3v on extension board are provided by external power resource.

Code

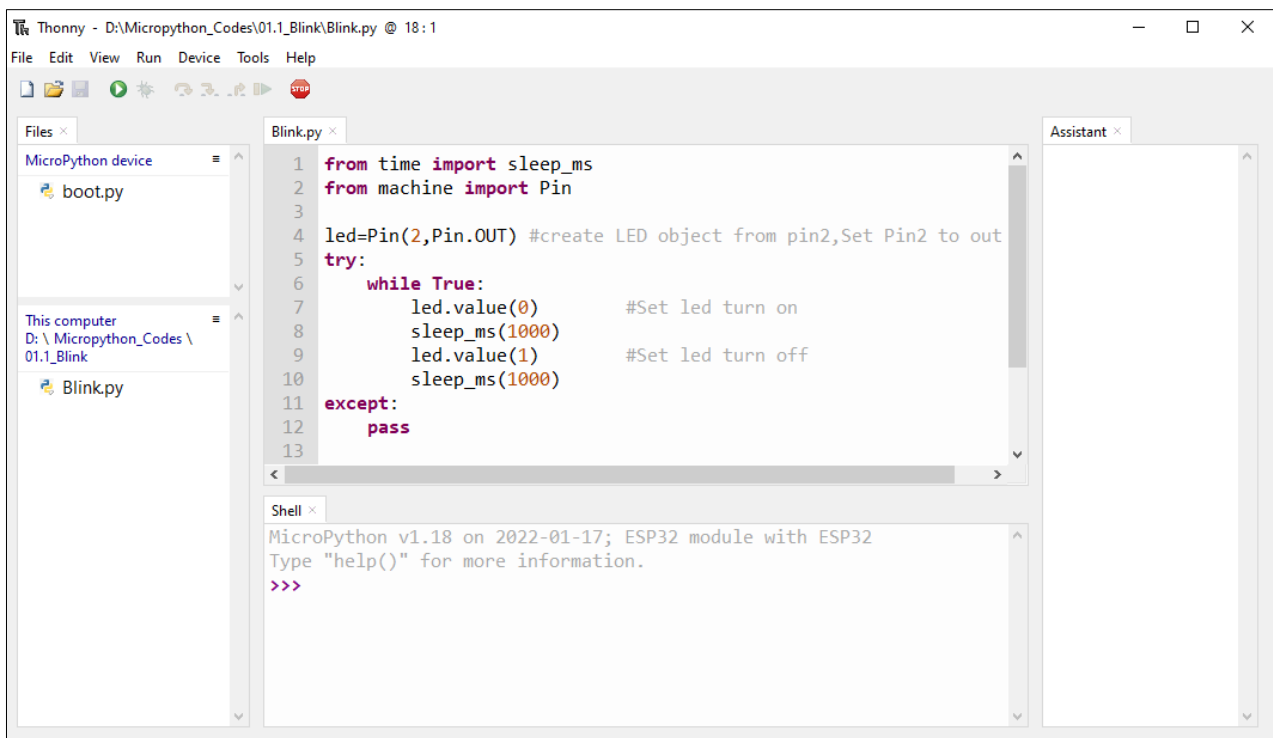
Codes used in this tutorial are saved in “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**”. You can move the codes to any location. For example, we save the codes in Disk(D) with the path of “**D:/Micropython_Codes**”.

01.1_Blink

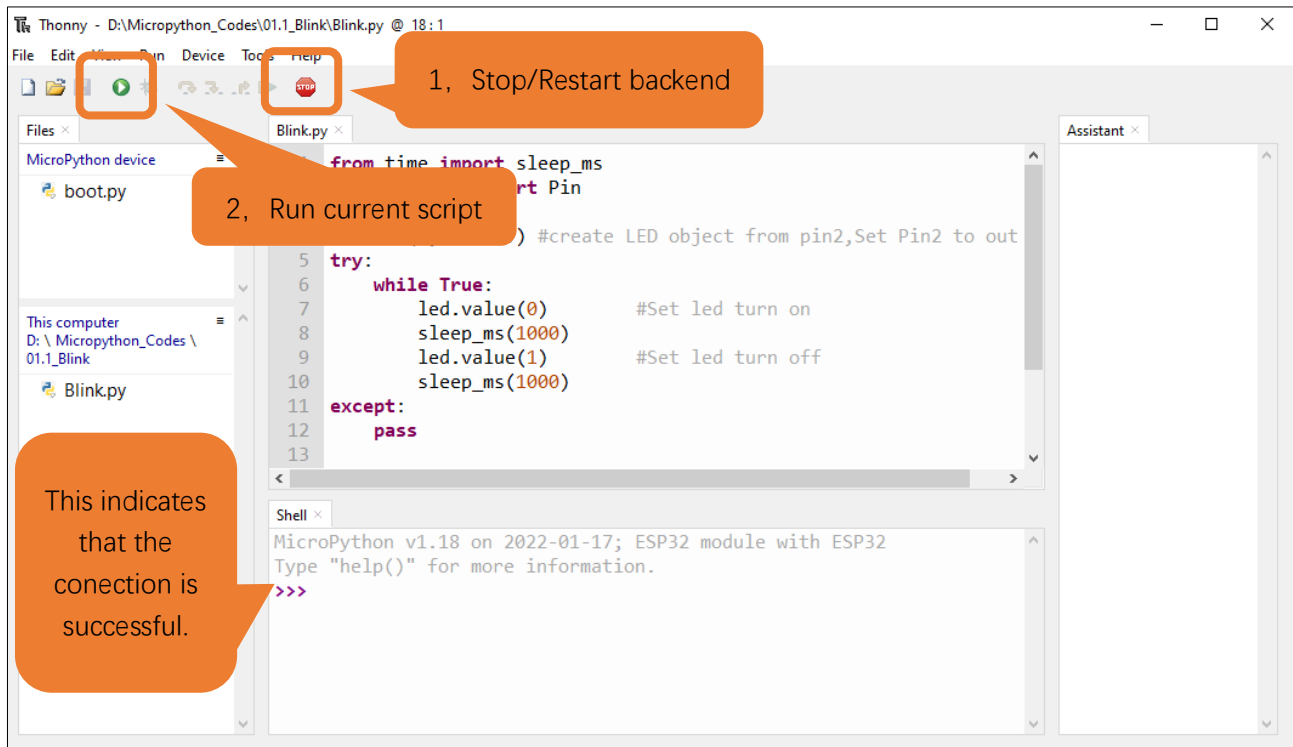
Open “Thonny”, click “This computer”→“D:”→“Micropython_Codes”.



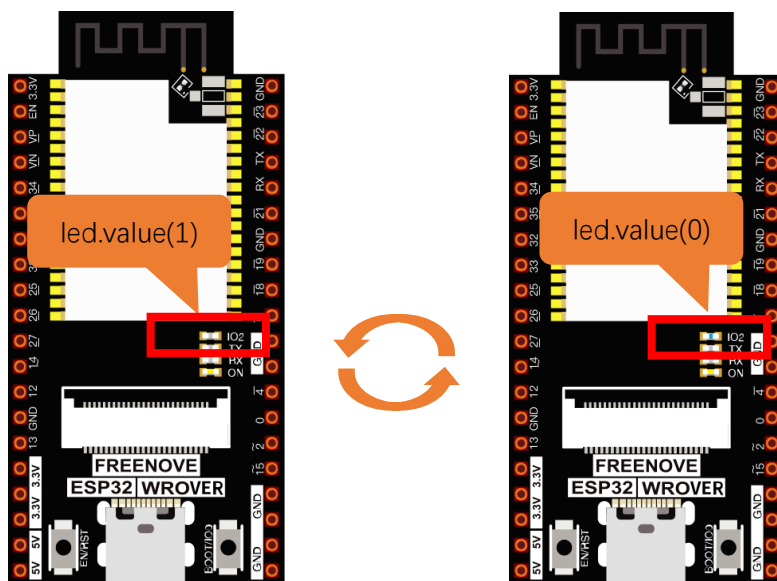
Expand folder “01.1_Blink” and double click “Blink.py” to open it. As shown in the illustration below.



Make sure ESP32 has been connected with the computer with ESP32 correctly. Click “Stop/Restart backend” or press the reset button, and then wait to see what interface will show up.



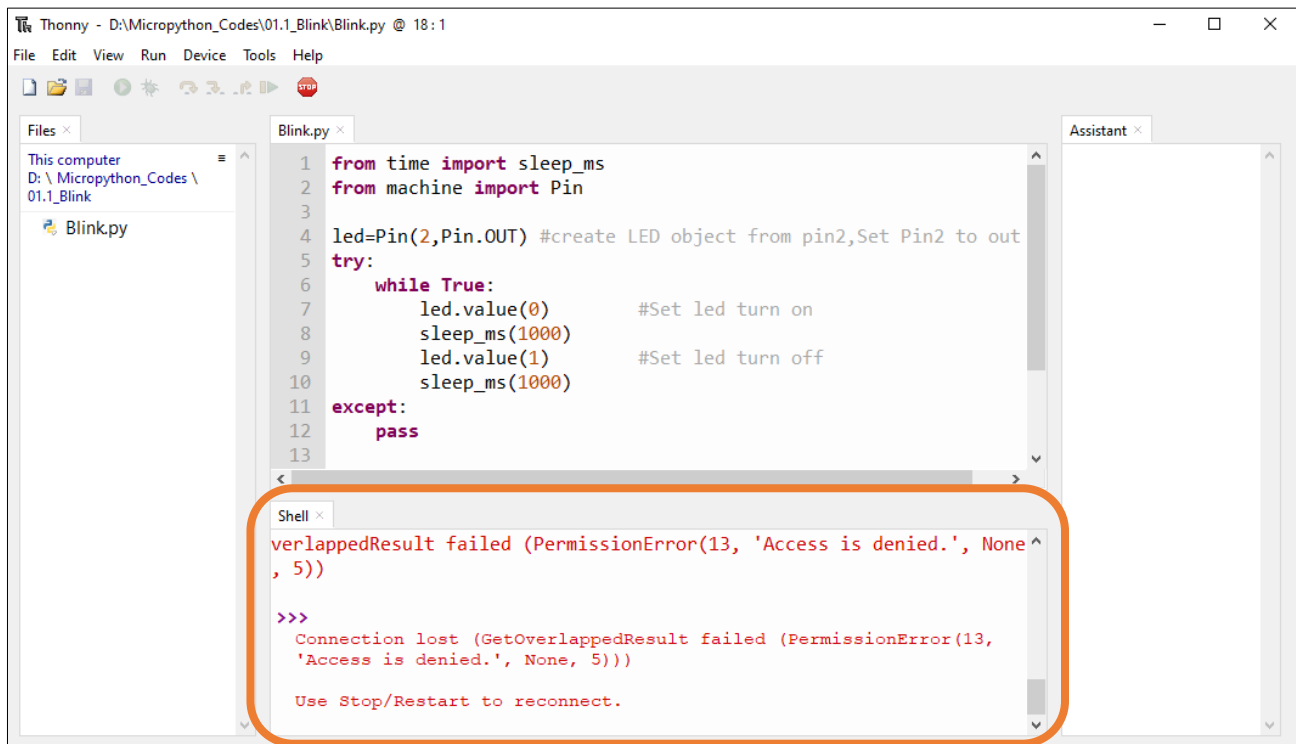
Click "Run current script" shown in the box above, the code starts to be executed and the LED in the circuit starts to blink.



Due to different versions, some board leds will show the opposite effect.

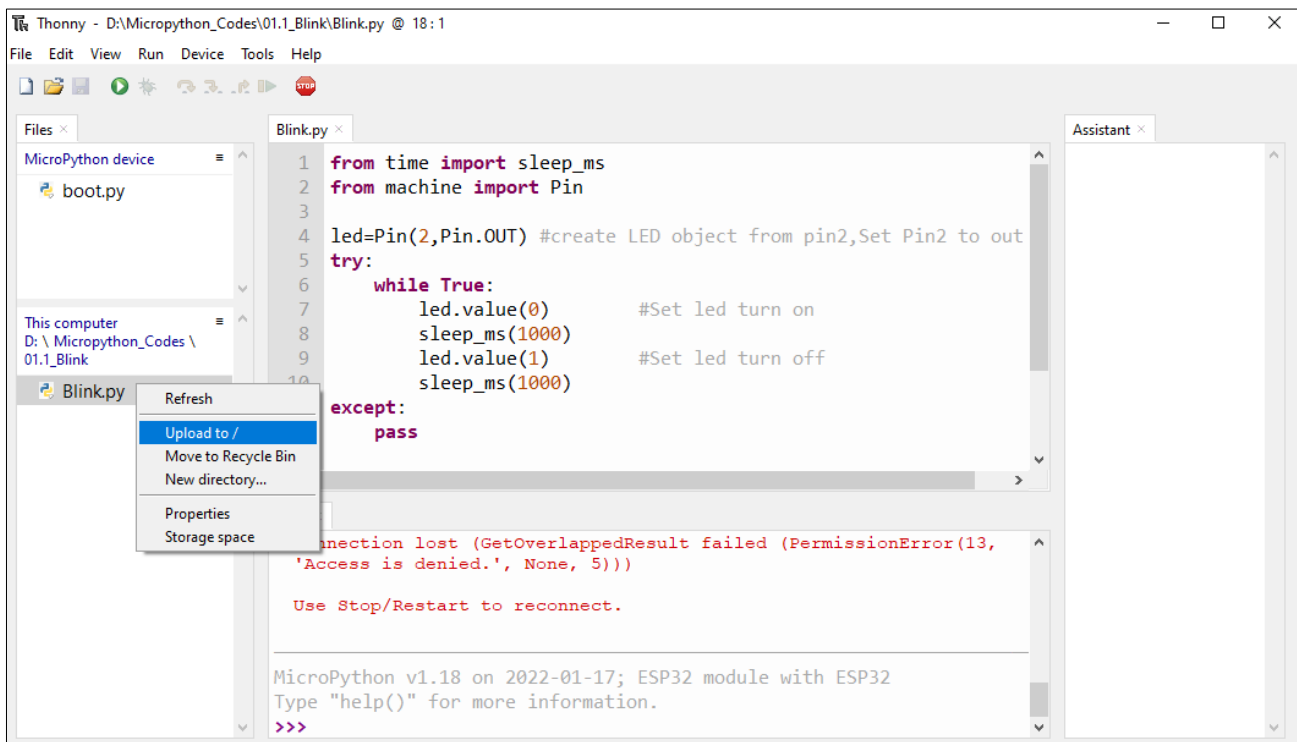
Note:

This is the code [running online](#). If you disconnect USB cable and repower ESP32 or press its reset key, LED stops blinking and the following messages will be displayed in Thonny.

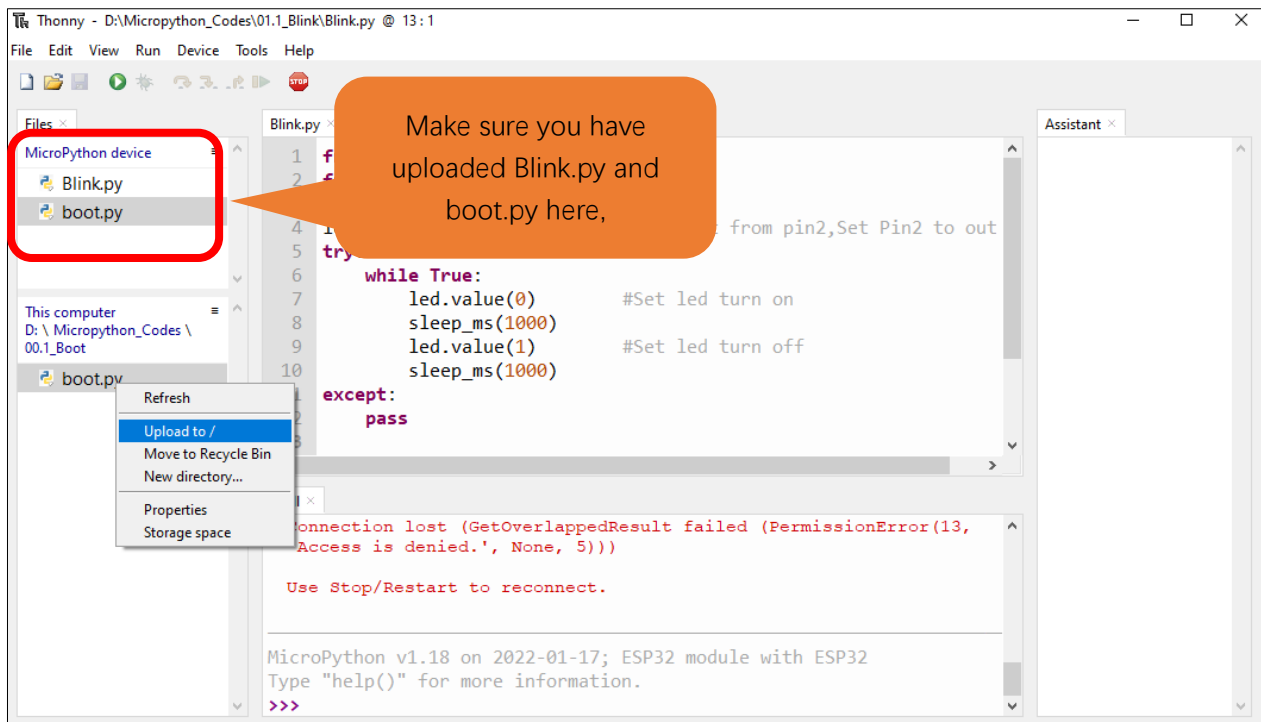


Uploading code to ESP32

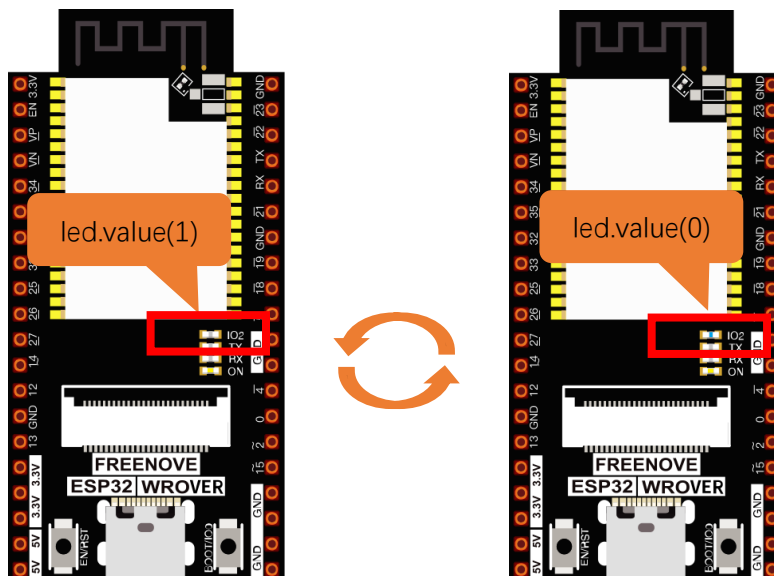
As shown in the following illustration, right-click the file Blink.py and select "Upload to /" to upload code to ESP32.



Upload boot.py in the same way.



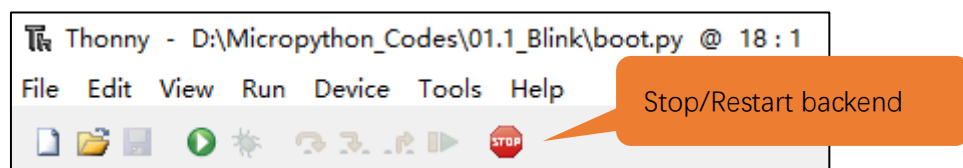
Press the reset key of ESP32 and you can see LED is ON for one second and then OFF for one second, which repeats in an endless loop.



Due to different versions, some board leds will show the opposite effect.

Note:

Codes here is run offline. If you want to stop running offline and enter Shell, just click "Stop" in Thonny.



If you have any concerns, please contact us via: support@freenove.com

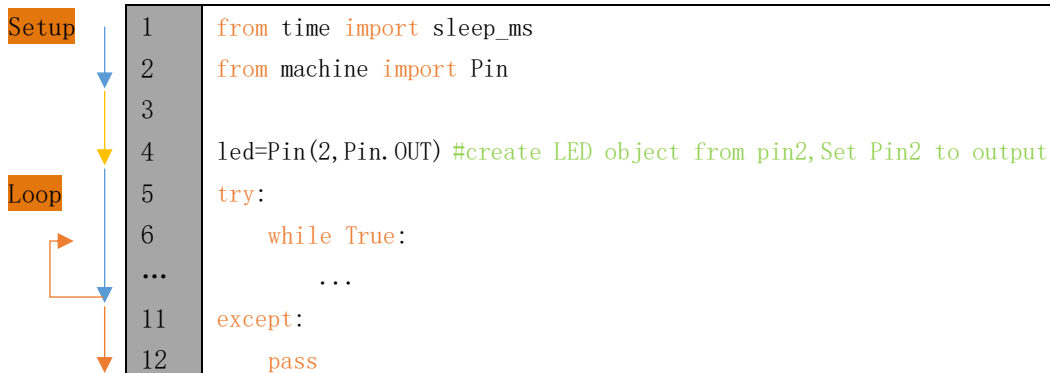
The following is the program code:

```

1  from time import sleep_ms
2  from machine import Pin
3
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
5  try:
6      while True:
7          led.value(0) #Set led turn on
8          sleep_ms(1000)
9          led.value(1) #Set led turn off
10         sleep_ms(1000)
11 except:
12     pass

```

Each time a new file is opened, the program will be executed from top to bottom. When encountering a loop construction, it will execute the loop statement according to the loop condition.



```

1  from time import sleep_ms
2  from machine import Pin
3
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
5  try:
6      while True:
7          ...
8          ...
9          ...
10         ...
11 except:
12     pass

```

Print() function is used to print data to Terminal. It can be executed in Terminal directly or be written in a Python file and executed by running the file.

```
print("Hello world!")
```

Each time when using the functions of ESP32, you need to import modules corresponding to those functions: Import sleep_ms module of time module and Pin module of machine module.

```

1  from time import sleep_ms
2  from machine import Pin

```

Configure GPIO2 of ESP32-WROVER to output mode and assign it to an object named "led".

```
4  led=Pin(2,Pin. OUT) #create LED object from pin2,Set Pin2 to output
```

It means that from now on, LED represents GPIO2 that is in output mode.

Set the value of LED to 1 and GPIO2 will output high level.

```
7  led.value(1) #Set led turn off
```

Set the value of LED to 0 and GPIO2 will output low level.

```
9  led.value(0) #Set led turn on
```

Execute codes in a while loop.

```

6      while True:
...          ...

```

Put statements that may cause an error in “try” block and the executing statements when an error occurs in “except” block. In general, when the program executes statements, it will execute those in “try” block. However, when an error occurs to ESP32 due to some interference or other reasons, it will execute statements in “except” block.

“Pass” is an empty statement. When it is executed, nothing happens. It is useful as a placeholder to make the structure of a program look better.

```
5  try:
...
11 except:
12     pass
```

The single-line comment of Micropython starts with a “#” and continues to the end of the line. Comments help us to understand code. When programs are running, Thonny will ignore comments.

```
9  #Set led turn on
```

MicroPython uses indentations to distinguish different blocks of code instead of braces. The number of indentations is changeable, but it must be consistent throughout one block. If the indentation of the same code block is inconsistent, it will cause errors when the program runs.

```
6  while True:
7      led.value(0) #Set led turn on
8      sleep_ms(1000)
9      led.value(1) #Set led turn off
10     sleep_ms(1000)
```

How to import python files

Whether to import the built-in python module or to import that written by users, the command “import” is needed.

If you import the module directly you should indicate the module to which the function or attribute belongs when using the function or attribute (constant, variable) in the module. The format should be: <module name>.<function or attribute>, otherwise an error will occur.

```
import random

num = random.randint(1, 100)
print(num)
```

If you only want to import a certain function or attribute in the module, use the from...import statement. The format is as follows

```
from random import randint
num = randint(1, 100)
print(num)
```

When using “from...import” statement to import function, to avoid conflicts and for easy understanding, you can use “as” statement to rename the imported function, as follows

```
from random import randint as rand
num = rand(1, 100)
print(num)
```

Reference

Class machine

Before each use of the **machine** module, please add the statement **"import machine"** to the top of python file.

machine.freq(freq_val): When freq_val is not specified, it is to return to the current CPU frequency; Otherwise, it is to set the current CPU frequency.

freq_val: 80000000(80MHz)、160000000(160MHz)、240000000(240MHz)

machine.reset(): A reset function. When it is called, the program will be reset.

machine.unique_id(): Obtains MAC address of the device.

machine.idle(): Turns off any temporarily unused functions on the chip and its clock, which is useful to reduce power consumption at any time during short or long periods.

machine.disable_irq(): Disables interrupt requests and return the previous IRQ state. The `disable_irq ()` function and `enable_irq ()` function need to be used together; Otherwise the machine will crash and restart.

machine.enable_irq(state): To re-enable interrupt requests. The parameter **state** should be the value that was returned from the most recent call to the `disable_irq()` function

machine.time_pulse_us(pin, pulse_level, timeout_us=1000000):

Tests the duration of the external pulse level on the given pin and returns the duration of the external pulse level in microseconds. When pulse level = 1, it tests the high level duration; When pulse level = 0, it tests the low level duration.

If the setting level is not consistent with the current pulse level, it will wait until they are consistent, and then start timing. If the set level is consistent with the current pulse level, it will start timing immediately.

When the pin level is opposite to the set level, it will wait for timeout and return "-2". When the pin level and the set level is the same, it will also wait timeout but return "-1". **timeout_us** is the duration of timeout.

Class Pin(id[, mode, pull, value])

Before each use of the **Pin** module, please add the statement “**from machine import Pin**” to the top of python file.

id: Arbitrary pin number

mode: Mode of pins

Pin.IN: Input Mode

Pin.OUT: Output Mode

Pin.OPEN_DRAIN: Open-drain Mode

Pull: Whether to enable the internal pull up and down mode

None: No pull up or pull down resistors

Pin.PULL_UP: Pull-up Mode, outputting high level by default

Pin.PULL_DOWN: Pull-down Mode, outputting low level by default

Value: State of the pin level, 0/1

Pin.init(mode, pull): Initialize pins

Pin.value([value]): Obtain or set state of the pin level, return 0 or 1 according to the logic level of pins. Without parameter, it reads input level. With parameter given, it is to set output level.

value: It can be either True/False or 1/0.

Pin.irq(trigger, handler): Configures an interrupt handler to be called when the pin level meets a condition.

trigger:

Pin.IRQ_FALLING: interrupt on falling edge

Pin.IRQ_RISING: interrupt on rising edge

3: interrupt on both edges

Handler: callback function

Class time

Before each use of the **time** module, please add the statement “**import time**” to the top of python file

time.sleep(sec): Sleeps for the given number of seconds

sec: This argument should be either an int or a float.

time.sleep_ms(ms): Sleeps for the given number of milliseconds, ms should be an int.

time.sleep_us(us): Sleeps for the given number of microseconds, us should be an int.

time.time(): Obtains the timestamp of CPU, with second as its unit.

time.ticks_ms(): Returns the incrementing millisecond counter value, which recounts after some values.

time.ticks_us(): Returns microsecond

time.ticks_cpu(): Similar to ticks_ms() and ticks_us(), but it is more accurate(return clock of CPU).

time.ticks_add(ticks, delta): Gets the timestamp after the offset.

ticks: ticks_ms()、ticks_us()、ticks_cpu()

delta: Delta can be an arbitrary integer number or numeric expression

time.ticks_diff(old_t, new_t): Calculates the interval between two timestamps, such as ticks_ms(), ticks_us() or ticks_cpu().

old_t: Starting time

new_t: Ending time

Chapter 2 Bluetooth

This chapter mainly introduces how to make simple data transmission through Bluetooth of ESP32-WROVER and mobile phones.

Project 2.1 Bluetooth Low Energy Data Passthrough

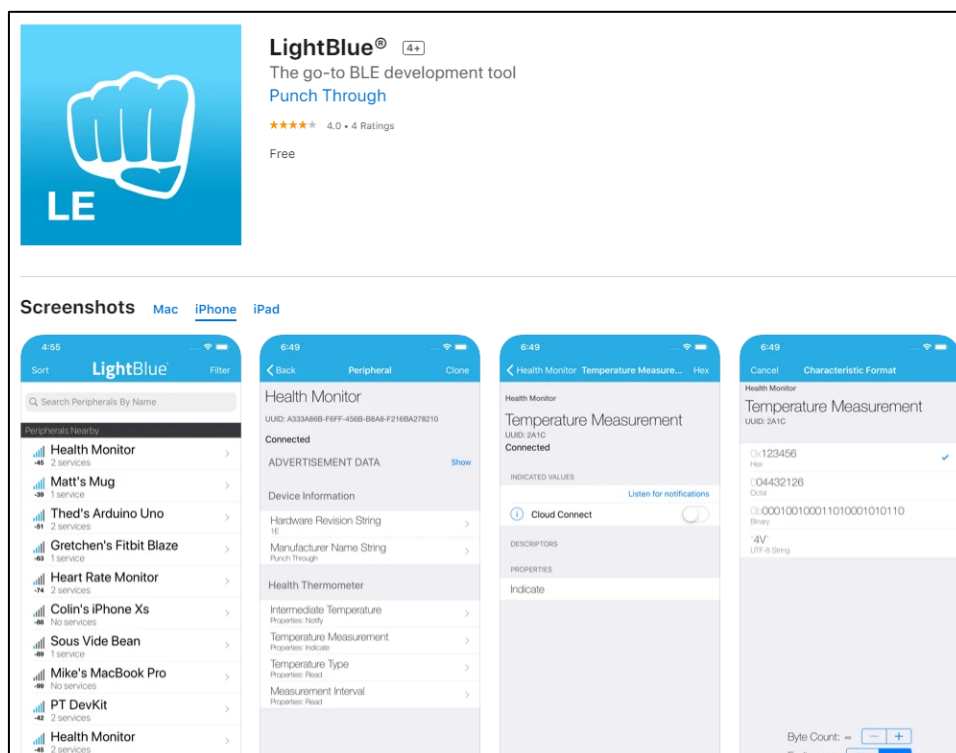
Component List

ESP32-WROVER x1	Micro USB Wire x1
-----------------	-------------------

Lightblue

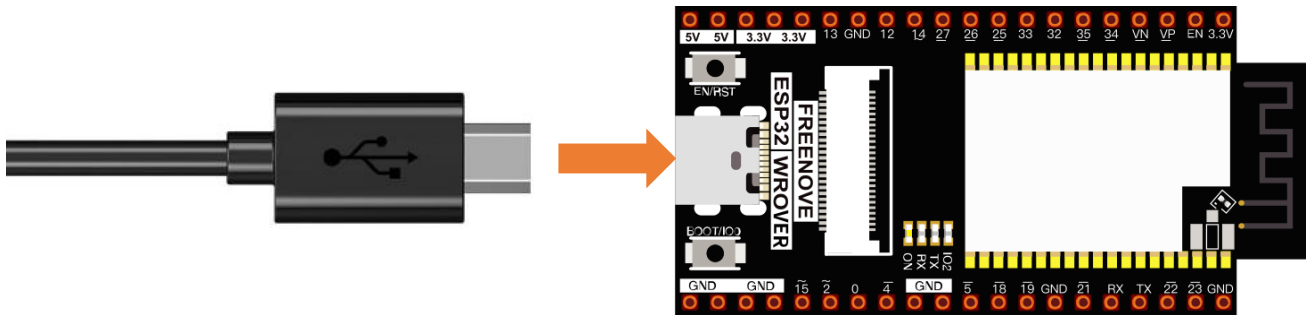
If you can't install Serial Bluetooth on your phone, try LightBlue. If you do not have this software installed on your phone, you can refer to this link:

<https://apps.apple.com/us/app/lightblue/id557428110?platform=iphone>



Circuit

Connect Freenove ESP32 to the computer using the USB cable.

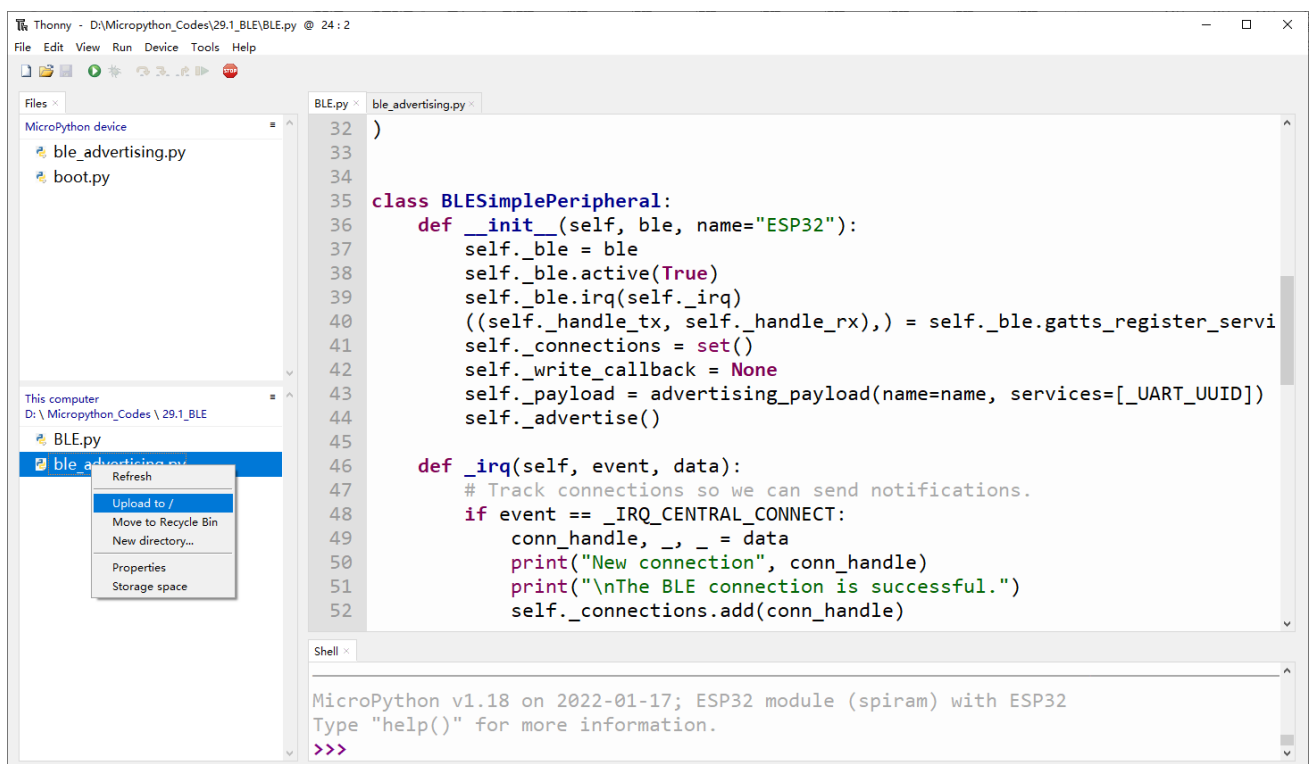


Code

Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “02.1_BLE”. Select “ble_advertising.py”, right click your mouse to select “Upload to /”, wait for “ble_advertising.py” to be uploaded to ESP32-WROVER and then double click “BLE.py”.

02.1_BLE



Click run for BLE.py.

```

Thonny - D:\MicroPython_Codes\29.1_BLE\BLE.py @ 43 : 10
File Edit View Run Device Tools Help

Files
MicroPython device
  ble_advertising.py
  boot.py

This computer
D:\MicroPython_Codes\29.1_BLE
  BLE.py
  ble_advertising.py

BLE.py
34
35 class BLESimplePeripheral:
36     def __init__(self, ble, name="ESP32"):
37         self._ble = ble
38         self._ble.active(True)
39         self._ble.irq(self._irq)

Shell
Please use LightBlue to connect to ESP32.
New connection 0

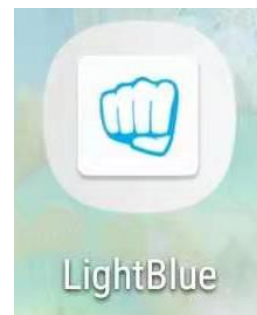
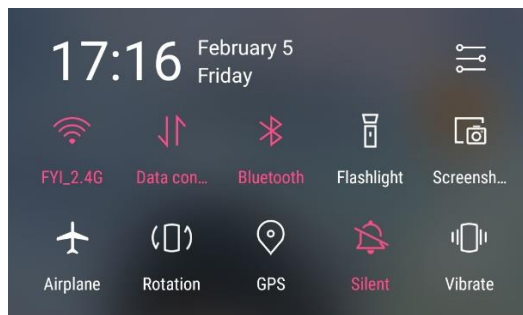
The BLE connection is successful.
Enter anything: 11112
Send: 11112
Enter anything: 2121
Send: 2121
Enter anything:

MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

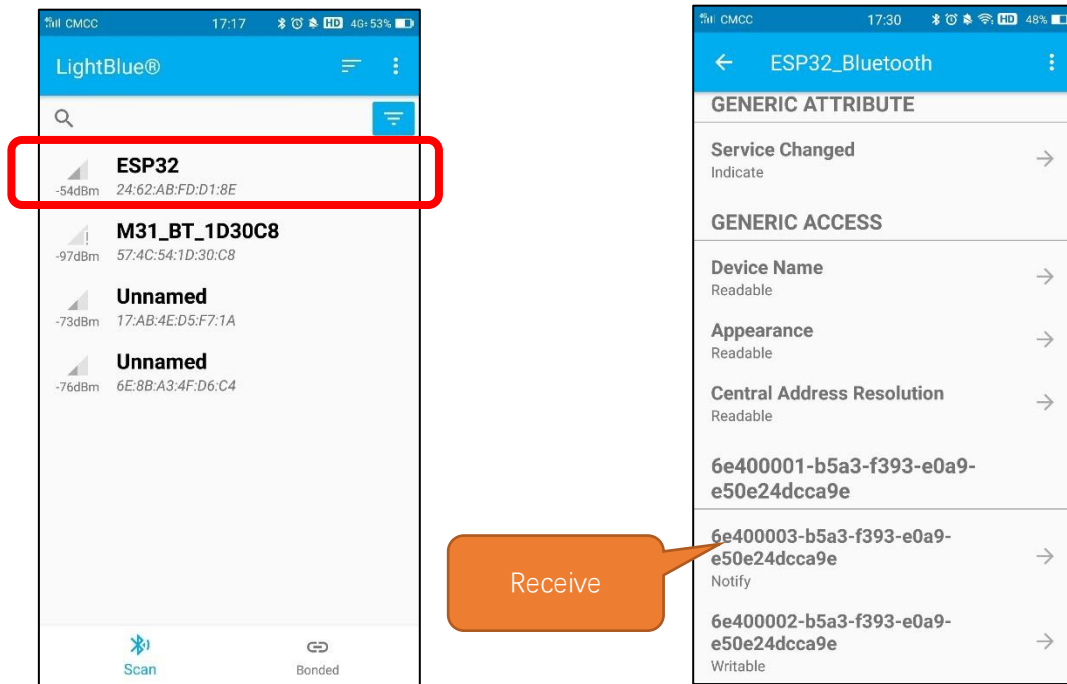
Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.

```

Turn ON Bluetooth on your phone, and open the Lightblue APP.



In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click ESP32.



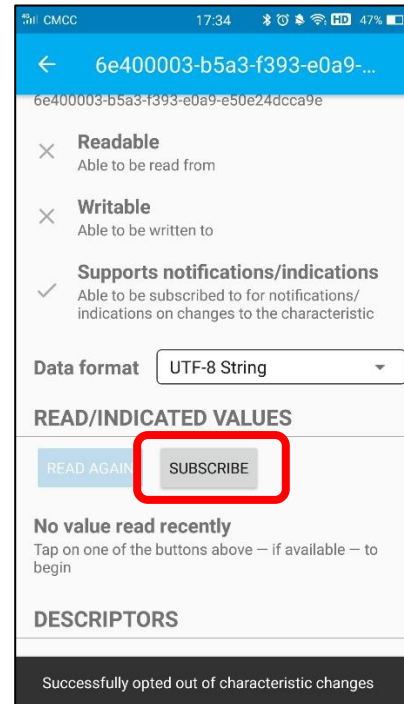
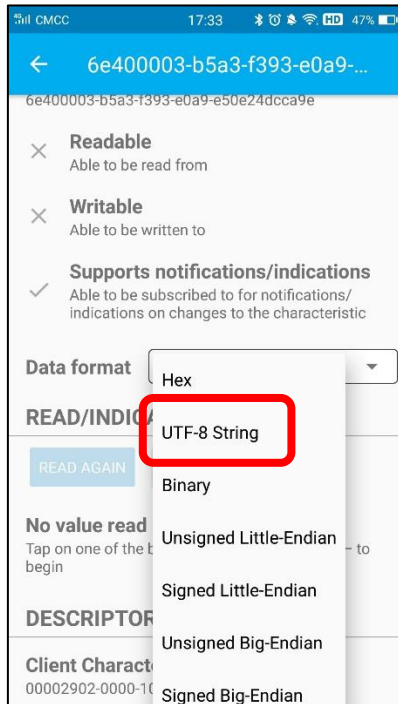
After Bluetooth is connect successfully, Shell will printer the information.

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything:
```

Click "Receive". Select the appropriate Data format in the box to the right of Data Format. For example, HEX for hexadecimal, utf-string for character, Binary for Binary, etc. Then click SUBSCRIBE.



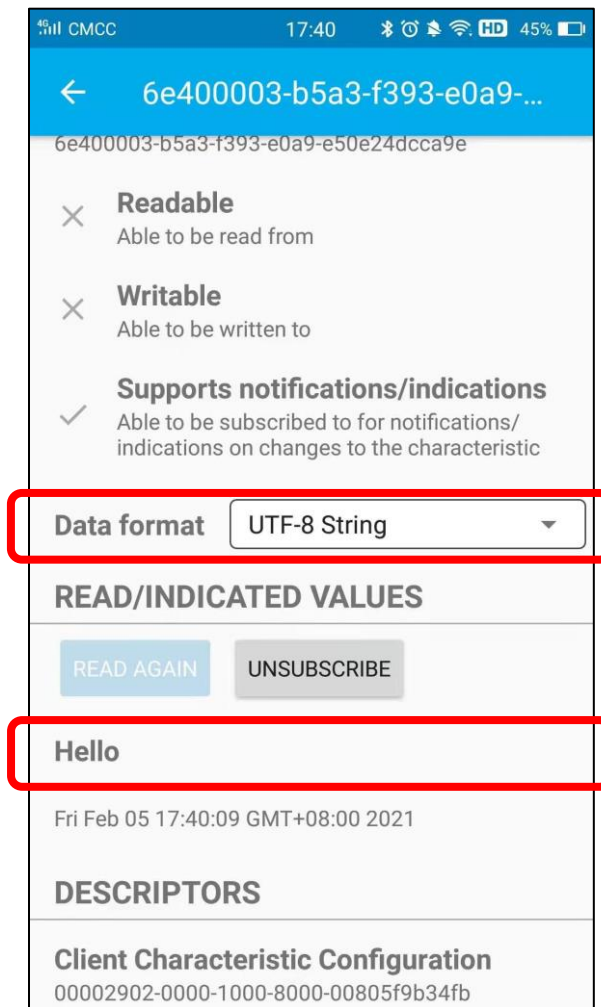
You can type "Hello" in Shell and press "Enter" to send.

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

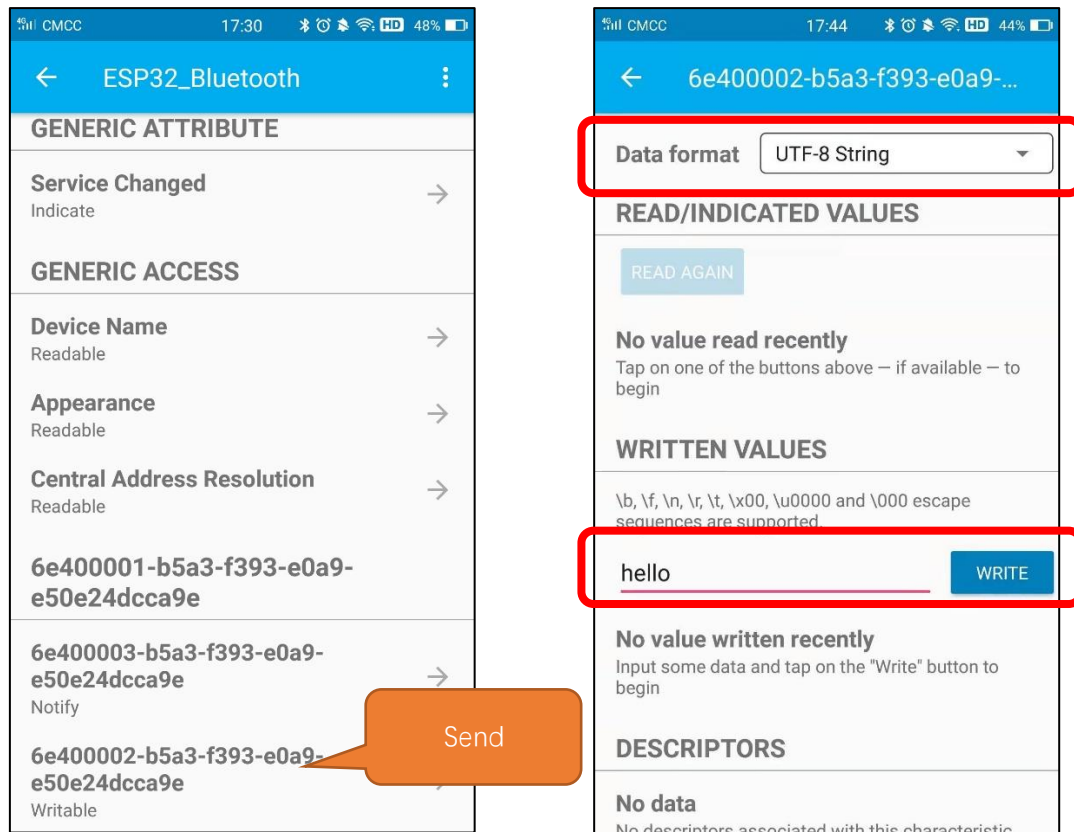
Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything: Hello
Send: Hello
Enter anything:
```

And then you can see the mobile Bluetooth has received the message.



Similarly, you can select "Send" on your phone. Set Data format, and then enter anything in the sending box and click Write to send.



You can check the message from Bluetooth in "Shell".

```
MicroPython v1.18 on 2022-01-17; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Disconnected 0
Starting advertising
Traceback (most recent call last):
  File "<stdin>", line 58, in _irq
  File "<stdin>", line 74, in _advertise
OSError: [Errno 19] ENODEV
Starting advertising
Please use LightBlue to connect to ESP32.
New connection 0

The BLE connection is successful.
Enter anything: Hello
Send: Hello
Enter anything: RX b'hello'
```

And now data can be transferred between your mobile phone and computer via ESP32-WROVER.

The following is the program code:

```
1  import bluetooth
2  import random
3  import struct
4  import time
5  from ble_advertising import advertising_payload
6  from micropython import const
7
8  _IRQ_CENTRAL_CONNECT = const(1)
9  _IRQ_CENTRAL_DISCONNECT = const(2)
10 _IRQ_GATTS_WRITE = const(3)
11 _FLAG_READ = const(0x0002)
12 _FLAG_WRITE_NO_RESPONSE = const(0x0004)
13 _FLAG_WRITE = const(0x0008)
14 _FLAG_NOTIFY = const(0x0010)
15
16 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
17 _UART_TX = (
18     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),
19     _FLAG_READ | _FLAG_NOTIFY,
20 )
21 _UART_RX = (
22     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
23     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
24 )
25 _UART_SERVICE = (
26     _UART_UUID,
27     (_UART_TX, _UART_RX),
28 )
29 class BLESimplePeripheral:
30     def __init__(self, ble, name="ESP32"):
31         self._ble = ble
32         self._ble.active(True)
33         self._ble.irq(self._irq)
34         ((self._handle_tx, self._handle_rx),) =
35 self._ble.gatts_register_services((_UART_SERVICE,))
36         self._connections = set()
37         self._write_callback = None
38         self._payload = advertising_payload(name=name, services=[_UART_UUID])
39         self._advertise()
40     def _irq(self, event, data):
41         # Track connections so we can send notifications.
42         if event == _IRQ_CENTRAL_CONNECT:
```

```

43         conn_handle, _, _ = data
44         print("New connection", conn_handle)
45         print("\nThe BLE connection is successful.")
46         self._connections.add(conn_handle)
47     elif event == _IRQ_CENTRAL_DISCONNECT:
48         conn_handle, _, _ = data
49         print("Disconnected", conn_handle)
50         self._connections.remove(conn_handle)
51         # Start advertising again to allow a new connection.
52         self._advertise()
53     elif event == _IRQ_GATTS_WRITE:
54         conn_handle, value_handle = data
55         value = self._ble.gatts_read(value_handle)
56         if value_handle == self._handle_rx and self._write_callback:
57             self._write_callback(value)
58     def send(self, data):
59         for conn_handle in self._connections:
60             self._ble.gatts_notify(conn_handle, self._handle_tx, data)
61     def is_connected(self):
62         return len(self._connections) > 0
63     def _advertise(self, interval_us=500000):
64         print("Starting advertising")
65         self._ble.gap_advertise(interval_us, adv_data=self._payload)
66     def on_write(self, callback):
67         self._write_callback = callback
68     def demo():
69         ble = bluetooth.BLE()
70         p = BLESimplePeripheral(ble)
71         def on_rx(rx_data):
72             print("RX", rx_data)
73         p.on_write(on_rx)
74         print("Please use LightBlue to connect to ESP32.")
75         while True:
76             if p.is_connected():
77                 # Short burst of queued notifications.
78                 tx_data = input("Enter anything: ")
79                 print("Send: ", tx_data)
80                 p.send(tx_data)
81 if __name__ == "__main__":
82     demo()

```

Define the specified UUID number for BLE vendor.

```

18 _UART_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
19 _UART_TX = (
20     bluetooth.UUID("6E400003-B5A3-F393-E0A9-E50E24DCCA9E"),

```

```

21     _FLAG_READ | _FLAG_NOTIFY,
22 )
23 _UART_RX = (
24     bluetooth.UUID("6E400002-B5A3-F393-E0A9-E50E24DCCA9E"),
25     _FLAG_WRITE | _FLAG_WRITE_NO_RESPONSE,
26 )

```

Write an `_irq` function to manage BLE interrupt events.

```

42     def _irq(self, event, data):
43         # Track connections so we can send notifications.
44         if event == _IRQ_CENTRAL_CONNECT:
45             conn_handle, _, _ = data
46             print("New connection", conn_handle)
47             print("\nThe BLE connection is successful.")
48             self._connections.add(conn_handle)
49         elif event == _IRQ_CENTRAL_DISCONNECT:
50             conn_handle, _, _ = data
51             print("Disconnected", conn_handle)
52             self._connections.remove(conn_handle)
53             # Start advertising again to allow a new connection.
54             self._advertise()
55         elif event == _IRQ_GATTS_WRITE:
56             conn_handle, value_handle = data
57             value = self._ble.gatts_read(value_handle)
58             if value_handle == self._handle_rx and self._write_callback:
59                 self._write_callback(value)

```

Initialize the BLE function and name it.

```

33     def __init__(self, ble, name="ESP32"):

```

When the mobile phone send data to ESP32 via BLE Bluetooth, it will print them out with serial port; When the serial port of ESP32 receive data, it will send them to mobile via BLE Bluetooth.

```

70     def demo():
71         ble = bluetooth.BLE()
72         p = BLESimplePeripheral(ble)
73         def on_rx(rx_data):
74             print("RX", rx_data)
75         p.on_write(on_rx)
76         print("Please use LightBlue to connect to ESP32.")
77         while True:
78             if p.is_connected():
79                 # Short burst of queued notifications.
80                 tx_data = input("Enter anything: ")
81                 print("Send: ", tx_data)
82                 p.send(tx_data)
83         lastMsg = now;
84     }

```

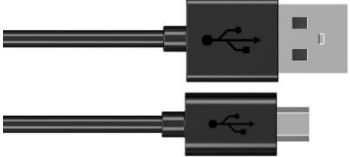
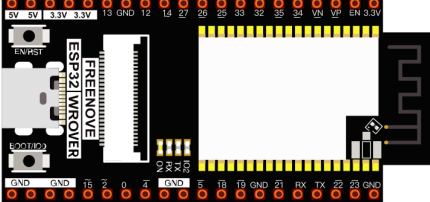

Chapter 3 WiFi Working Modes

In this chapter, we'll focus on the WiFi infrastructure for ESP32-WROVER.

ESP32-WROVER has 3 different WiFi operating modes: Station mode, AP mode and AP+Station mode. All WiFi programming projects must be configured with WiFi operating mode before using WiFi, otherwise WiFi cannot be used.

Project 3.1 Station mode

Component List

Micro USB Wire x1	ESP32-WROVER x1
	

Component knowledge

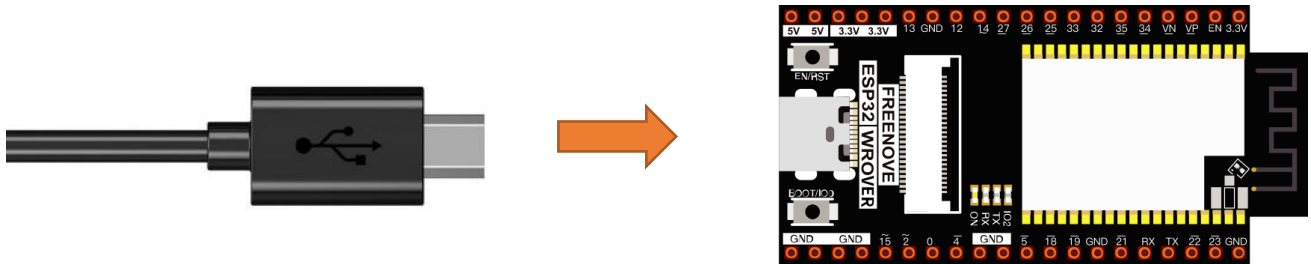
Station mode

When ESP32 selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if ESP32 wants to communicate with the PC, it needs to be connected to the router.



Circuit

Connect Freenove ESP32 to the computer using the USB cable.

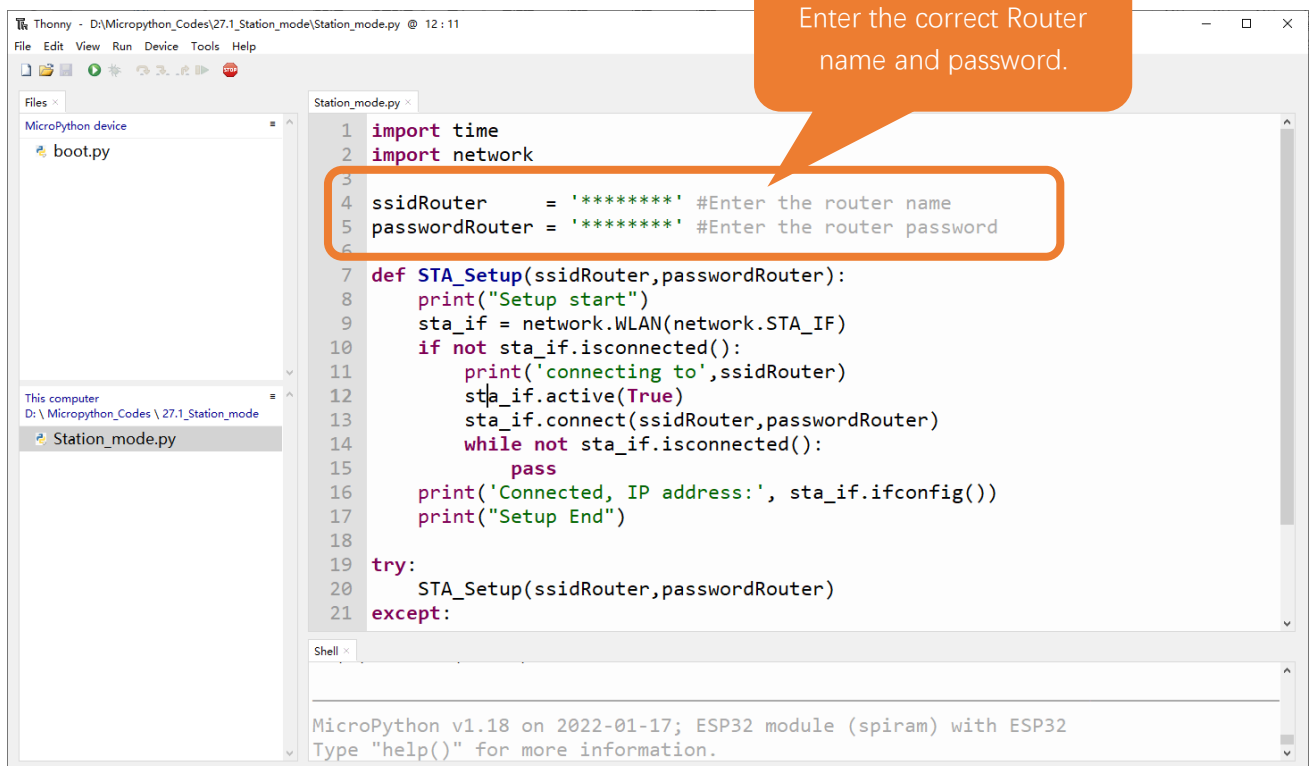


Code

Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “03.1_Station_mode” and double click “Station_mode.py”.

03.1_Station_mode



Because the names and passwords of routers in various places are different, before the Code runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to ESP32-WROVER, wait for ESP32 to connect to your router and print the IP address assigned by the router to ESP32 in "Shell".

```
MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Setup start
connecting to FYI_2.4G
I (4155528) phy: phy_version: 4102, 2fa7a43, Jul 15 2019, 13:06:06,
0, 2
Connected, IP address: ('192.168.1.102', '255.255.255.0', '192.168.
1.1', '192.168.1.1')
Setup End

>>>
```

The following is the program code:

```
1  import time
2  import network
3
4  ssidRouter    = '*****' #Enter the router name
5  passwordRouter = '*****' #Enter the router password
6
7  def STA_Setup(ssidRouter, passwordRouter):
8      print("Setup start")
9      sta_if = network.WLAN(network.STA_IF)
10     if not sta_if.isconnected():
11         print('connecting to', ssidRouter)
12         sta_if.active(True)
13         sta_if.connect(ssidRouter, passwordRouter)
14         while not sta_if.isconnected():
15             pass
16         print('Connected, IP address:', sta_if.ifconfig())
17         print("Setup End")
18
19     try:
20         STA_Setup(ssidRouter, passwordRouter)
21     except:
22         sta_if.disconnect()
```

Import network module.

```
2  import network
```

Enter correct router name and password.

```
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
```

Set ESP32 in Station mode.

```
9      sta_if = network.WLAN(network.STA_IF)
```

Activate ESP32's Station mode, initiate a connection request to the router and enter the password to connect.

```
12     sta_if.active(True)
13     sta_if.connect(ssidRouter, passwordRouter)
```

Wait for ESP32 to connect to router until they connect to each other successfully.

```
14     while not sta_if.isconnected():
15         pass
```

Print the IP address assigned to ESP32-WROVER in "Shell".

```
16     print('Connected, IP address:', sta_if.ifconfig())
```

Reference

Class network

Before each use of **network**, please add the statement "**import network**" to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points.

network.AP_IF: Access points, allowing other WiFi clients to connect.

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface.

scan(ssid, bssid, channel, RSSI, authmode, hidden): Scan for wireless networks available nearby (only scan on STA interface), return a tuple list of information about the WiFi access point.

bssid: The hardware address of the access point, returned in binary form as a byte object. You can use `ubinascii.hexlify()` to convert it to ASCII format.

authmode: Access type

AUTH_OPEN = 0

AUTH_WEP = 1

AUTH_WPA_PSK = 2

AUTH_WPA2_PSK = 3

AUTH_WPA_WPA2_PSK = 4

AUTH_MAX = 6

Hidden: Whether to scan for hidden access points

False: Only scanning for visible access points

True: Scanning for all access points including the hidden ones.

isconnected(): Check whether ESP32 is connected to AP in Station mode. In STA mode, it returns True if it is connected to a WiFi access point and has a valid IP address; Otherwise it returns False.

connect(ssid, password): Connecting to wireless network.

ssid: WiFi name

password: WiFi password

disconnect(): Disconnect from the currently connected wireless network.

Project 3.2 AP mode

Component List & Circuit

Component List & Circuit are the same as in Section 03.1.

Component knowledge

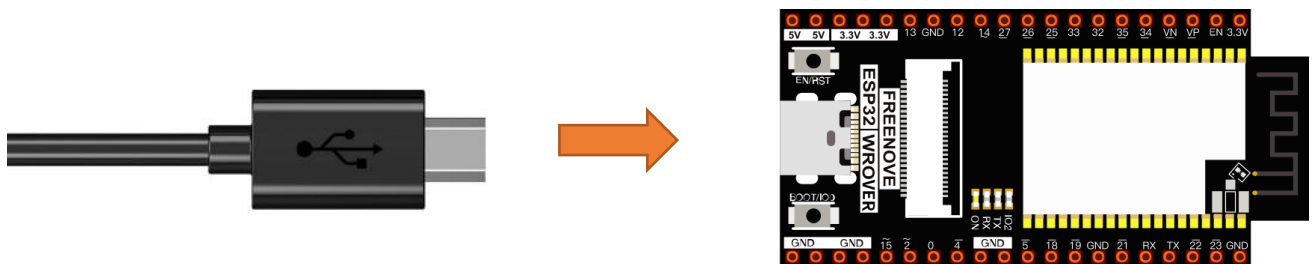
AP mode

When ESP32 selects AP mode, it creates a hotspot network that is separated from the Internet and waits for other WiFi devices to connect. As shown in the figure below, ESP32 is used as a hotspot. If a mobile phone or PC wants to communicate with ESP32, it must be connected to the hotspot of ESP32. Only after a connection is established with ESP32 can they communicate.



Circuit

Connect Freenove ESP32 to the computer using the USB cable.

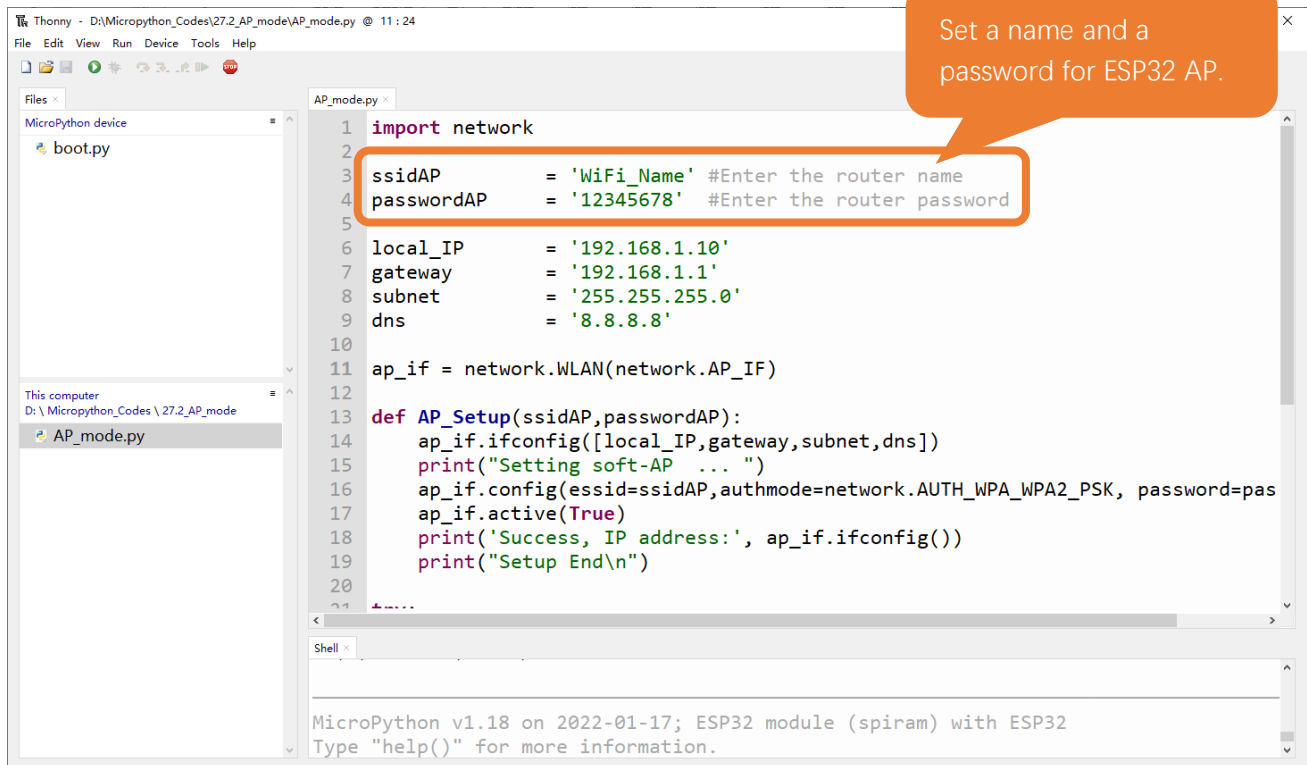


Code

Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

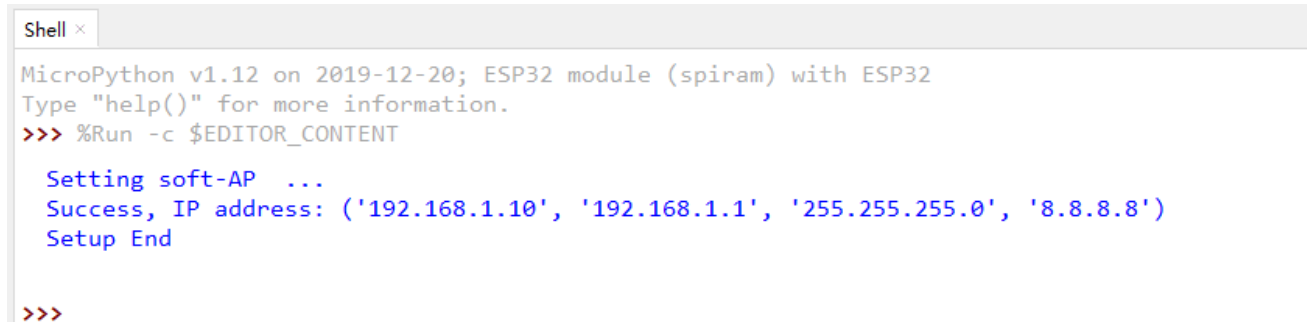
Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “03.2_AP_mode”. and double click “AP_mode.py”.

03.2_AP_mode

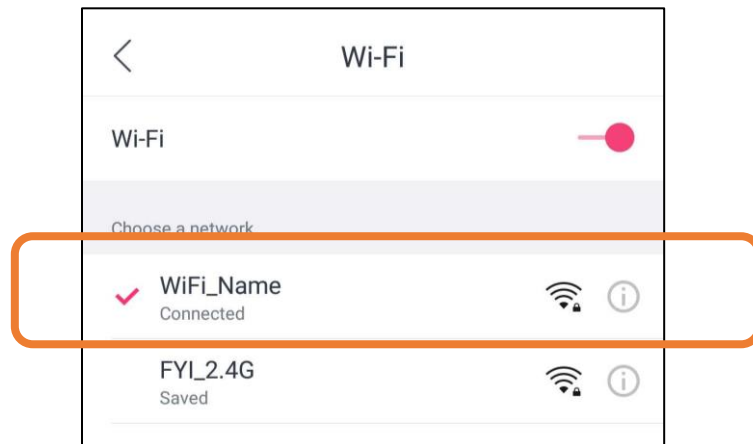


Before the Code runs, you can make any changes to the AP name and password for ESP32 in the box as shown in the illustration above. Of course, you can leave it alone by default.

Click “Run current script”, open the AP function of ESP32 and print the access point information.



Turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP32, which is called "WiFi_Name" in this Code. You can enter the password "12345678" to connect it or change its AP name and password by modifying Code.



The following is the program code:

```

1  import network
2
3  ssidAP      = 'WiFi_Name' #Enter the router name
4  passwordAP  = '12345678' #Enter the router password
5
6  local_IP    = '192.168.1.10'
7  gateway     = '192.168.1.1'
8  subnet      = '255.255.255.0'
9  dns         = '8.8.8.8'
10
11  ap_if = network.WLAN(network.AP_IF)
12
13  def AP_Setup(ssidAP, passwordAP):
14      ap_if.ifconfig([local_IP, gateway, subnet, dns])
15      print("Setting soft-AP ... ")
16      ap_if.config(essid=ssidAP, authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
17      ap_if.active(True)
18      print('Success, IP address:', ap_if.ifconfig())
19      print("Setup End\n")
20
21  try:
22      AP_Setup(ssidAP, passwordAP)
23  except:
24      ap_if.disconnect()

```

Import network module.

```

1  import network

```

Enter correct AP name and password.

```
3 ssidAP = 'WiFi_Name' #Enter the router name
4 passwordAP = '12345678' #Enter the router password
```

Set ESP32 in AP mode.

```
11 ap_if = network.WLAN(network.AP_IF)
```

Configure IP address, gateway and subnet mask for ESP32.

```
14 ap_if.ifconfig([local_IP, gateway, subnet, dns])
```

Turn on an AP in ESP32, whose name is set by ssid_AP and password is set by password_AP.

```
16 ap_if.config(essid=ssidAP, authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
17 ap_if.active(True)
```

If the program is running abnormally, the AP disconnection function will be called.

```
14 ap_if.disconnect()
```

Reference

Class network

Before each use of **network**, please add the statement “**import network**” to the top of the python file.

WLAN(interface_id): Set to WiFi mode.

network.STA_IF: Client, connecting to other WiFi access points

network.AP_IF: Access points, allowing other WiFi clients to connect

active(is_active): With parameters, it is to check whether to activate the network interface; Without parameters, it is to query the current state of the network interface

isconnected(): In AP mode, it returns True if it is connected to the station; otherwise it returns False.

connect(ssid, password): Connecting to wireless network

ssid: WiFiname

password: WiFipassword

config(essid, channel): To obtain the MAC address of the access point or to set the WiFi channel and the name of the WiFi access point.

ssid: WiFi account name

channel: WiFichannel

ifconfig([(ip, subnet, gateway, dns)]): Without parameters, it returns a 4-tuple (ip, subnet_mask, gateway, DNS_server); With parameters, it configures static IP.

ip: IPaddress

subnet_mask: subnet mask

gateway: gateway

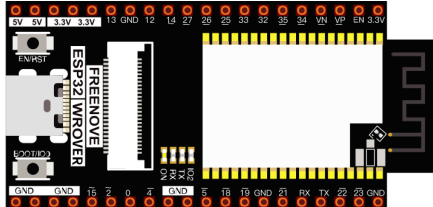
DNS_server: DNSserver

disconnect(): Disconnect from the currently connected wireless network

status(): Return the current status of the wireless connection

Project 3.3 AP+Station mode

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
--	---

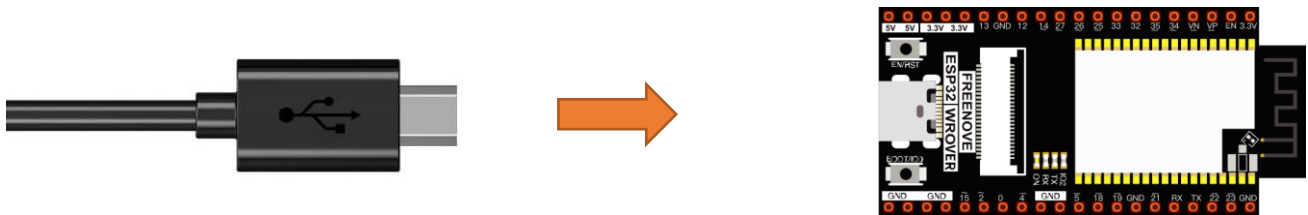
Component knowledge

AP+Station mode

In addition to AP mode and Station mode, ESP32 can also use AP mode and Station mode at the same time. This mode contains the functions of the previous two modes. Turn on ESP32's Station mode, connect it to the router network, and it can communicate with the Internet via the router. At the same time, turn on its AP mode to create a hotspot network. Other WiFi devices can choose to connect to the router network or the hotspot network to communicate with ESP32.

Circuit

Connect Freenove ESP32 to the computer using the USB cable.

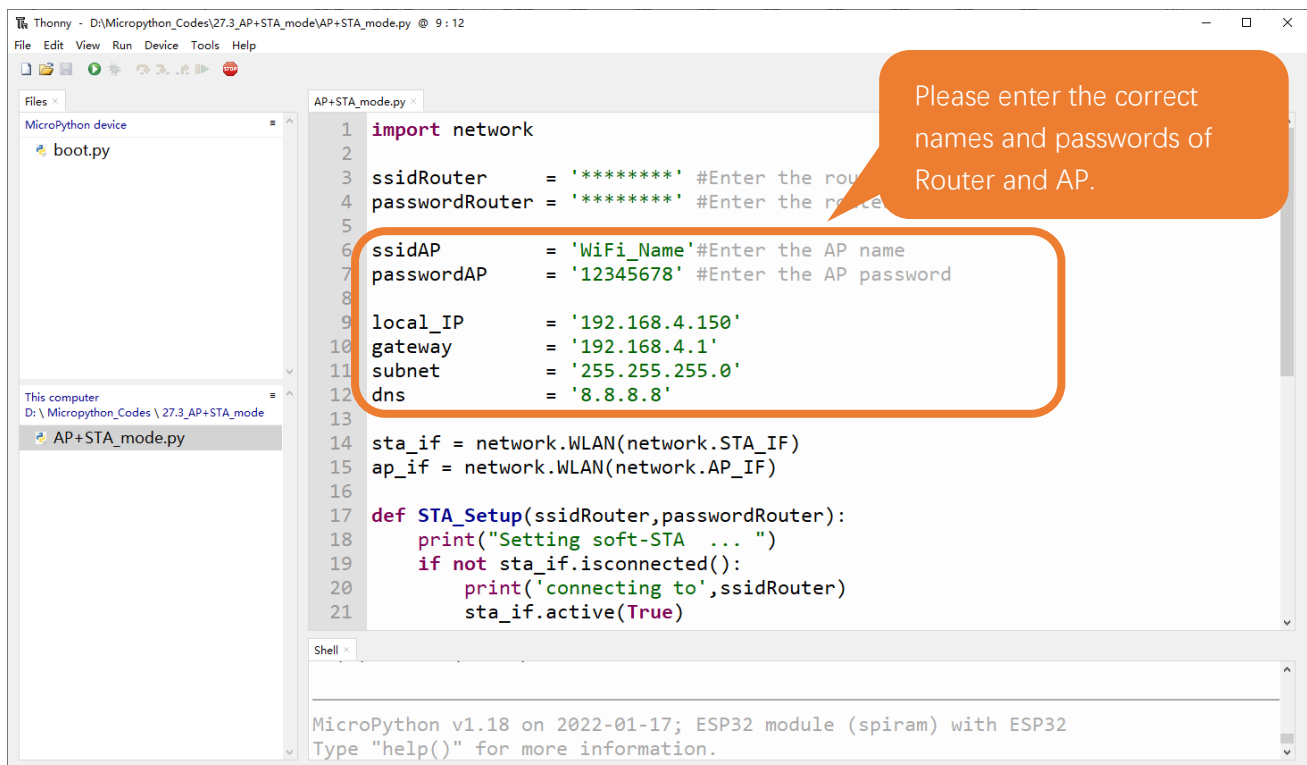


Code

Move the program folder "**Freenove_ESP32_WROVER_Board/Python/Python_Codes**" to disk(D) in advance with the path of "**D:/Micropython_Codes**".

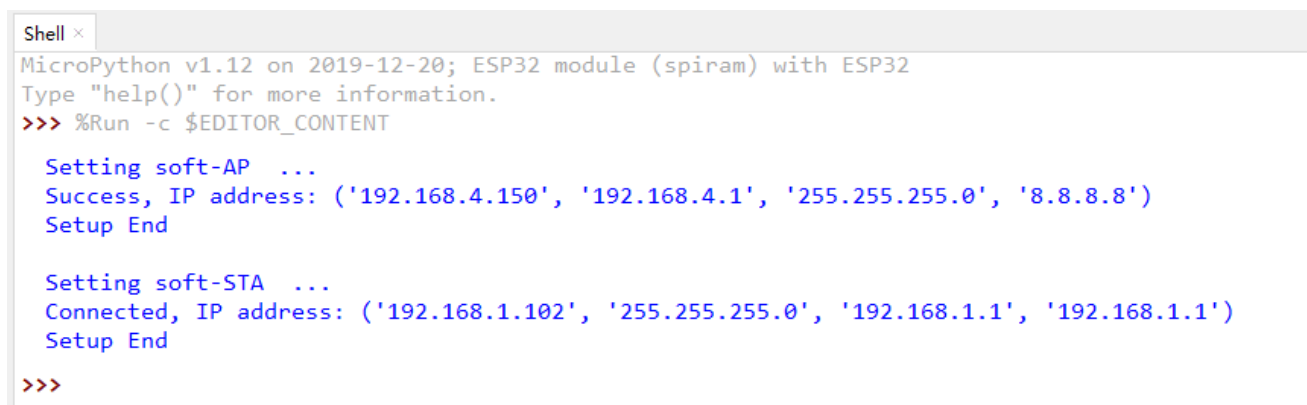
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "03.3_AP+STA_mode" and double click "AP+STA_mode.py".

03.3_AP+STA_mode

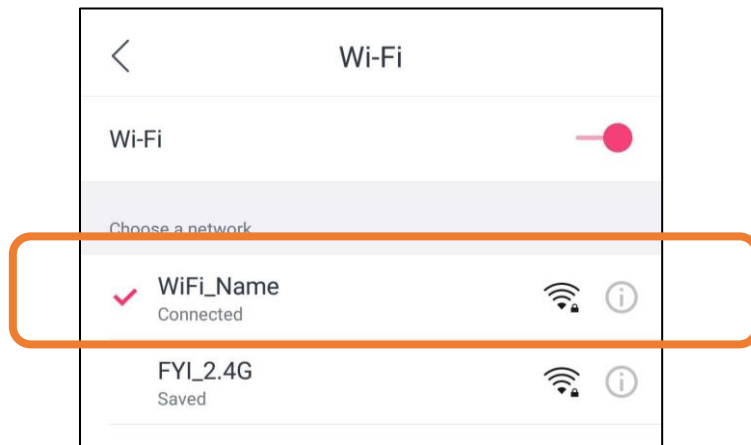


It is analogous to Project 3.1 and Project 3.2. Before running the Code, you need to modify ssidRouter, passwordRouter, ssidAP and passwordAP shown in the box of the illustration above.

After making sure that the code is modified correctly, click "Run current script" and the "Shell" will display as follows:



Turn on the WiFi scanning function of your phone, and you can see the ssidAP on ESP32.



The following is the program code:

```

1  import network
2
3  ssidRouter    = '*****' #Enter the router name
4  passwordRouter = '*****' #Enter the router password
5
6  ssidAP        = 'WiFi_Name' #Enter the AP name
7  passwordAP    = '12345678' #Enter the AP password
8
9  local_IP      = '192.168.4.150'
10 gateway       = '192.168.4.1'
11 subnet        = '255.255.255.0'
12 dns           = '8.8.8.8'
13
14 sta_if = network.WLAN(network.STA_IF)
15 ap_if = network.WLAN(network.AP_IF)
16
17 def STA_Setup(ssidRouter, passwordRouter):
18     print("Setting soft-STA ... ")
19     if not sta_if.isconnected():
20         print('connecting to', ssidRouter)
21         sta_if.active(True)
22         sta_if.connect(ssidRouter, passwordRouter)
23         while not sta_if.isconnected():
24             pass
25     print('Connected, IP address:', sta_if.ifconfig())
26     print("Setup End")
27
28 def AP_Setup(ssidAP, passwordAP):
29     ap_if.ifconfig([local_IP, gateway, subnet, dns])
30     print("Setting soft-AP ... ")

```

```
31     ap_if.config(essid=ssidAP,authmode=network.AUTH_WPA_WPA2_PSK, password=passwordAP)
32     ap_if.active(True)
33     print('Success, IP address:', ap_if.ifconfig())
34     print("Setup End\n")
35
36 try:
37     AP_Setup(ssidAP, passwordAP)
38     STA_Setup(ssidRouter, passwordRouter)
39 except:
40     sta_if.disconnect()
41     ap_if.idconnect()
```

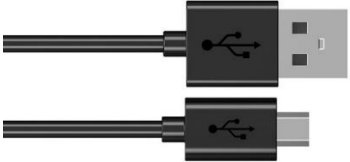
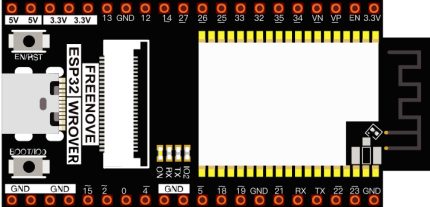
Chapter 4 TCP/IP

In this chapter, we will introduce how ESP32 implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

Project 4.1 As Client

In this section, ESP32 is used as Client to connect Server on the same LAN and communicate with it.

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
---	--

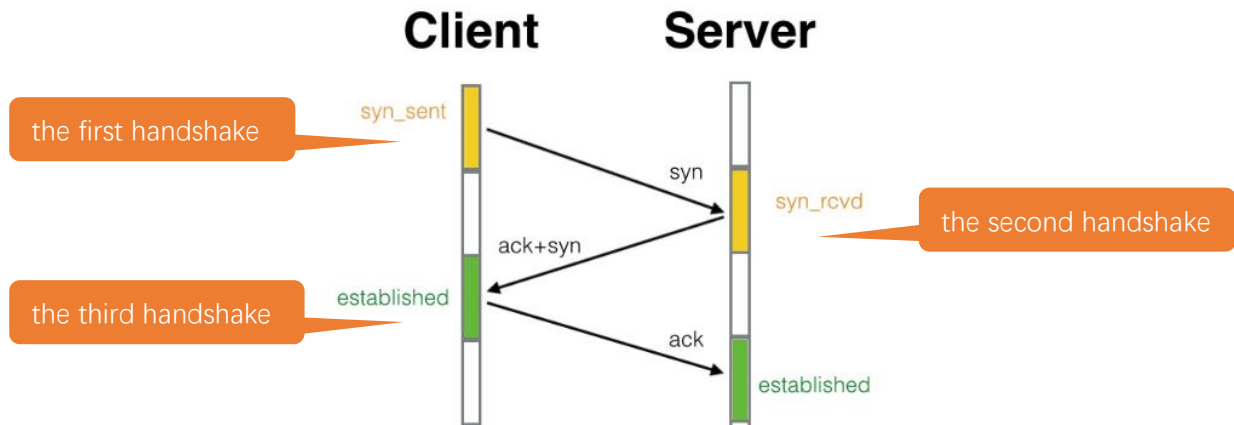
Component knowledge

TCP connection

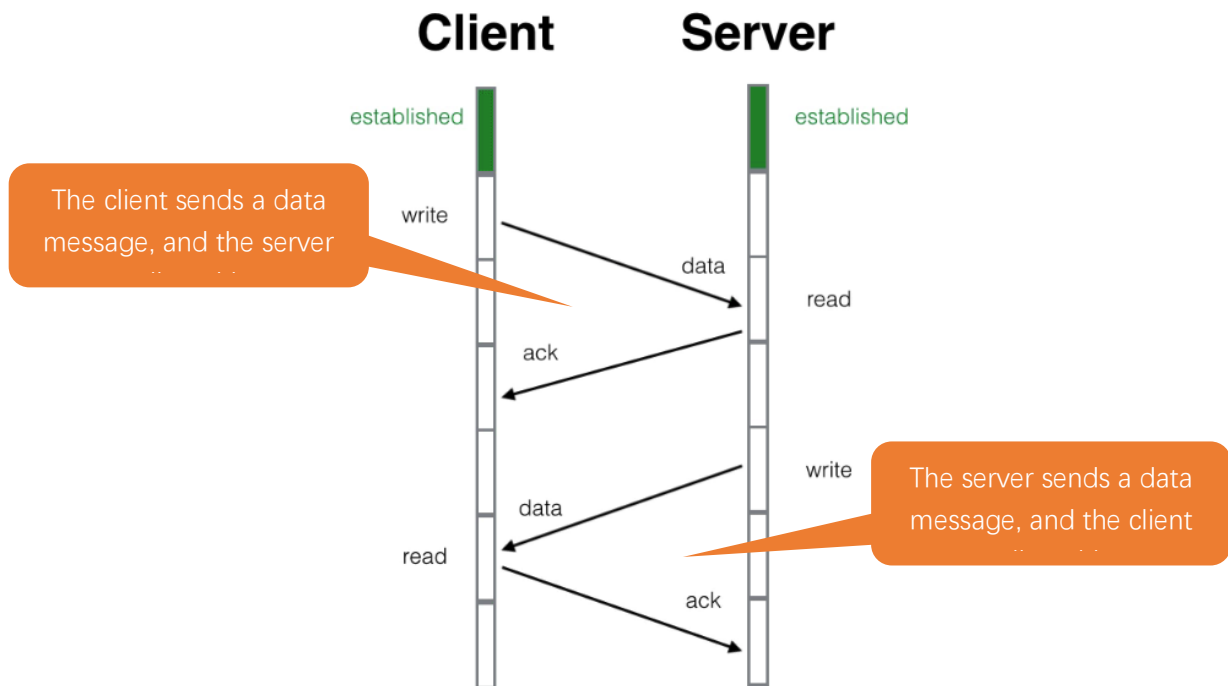
Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-times handshake" is required.

Three-times handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-times handshake". The first handshake, the client sends a connection request to the server and waits for the server to confirm. The second handshake, the server sends a response back to the client informing that it has received the connection request.

The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you've not installed Processing, you can download it by clicking <https://processing.org/download/>. You can choose an appropriate version to download according to your PC system.



Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.

core	2020/1/17 12:16
java	2020/1/17 12:17
lib	2020/1/17 12:16
modes	2020/1/17 12:16
tools	2020/1/17 12:16
processing.exe	2020/1/17 12:16
processing-java.exe	2020/1/17 12:16
revisions.txt	2020/1/17 12:16

Use Server mode for communication

Open the "Freenove_ESP32_WROVER_Board/Codes/Micropython_Codes/04.1_TCP_as_Client/sketchWiFi/sketchWiFi.pde". Click "Run".

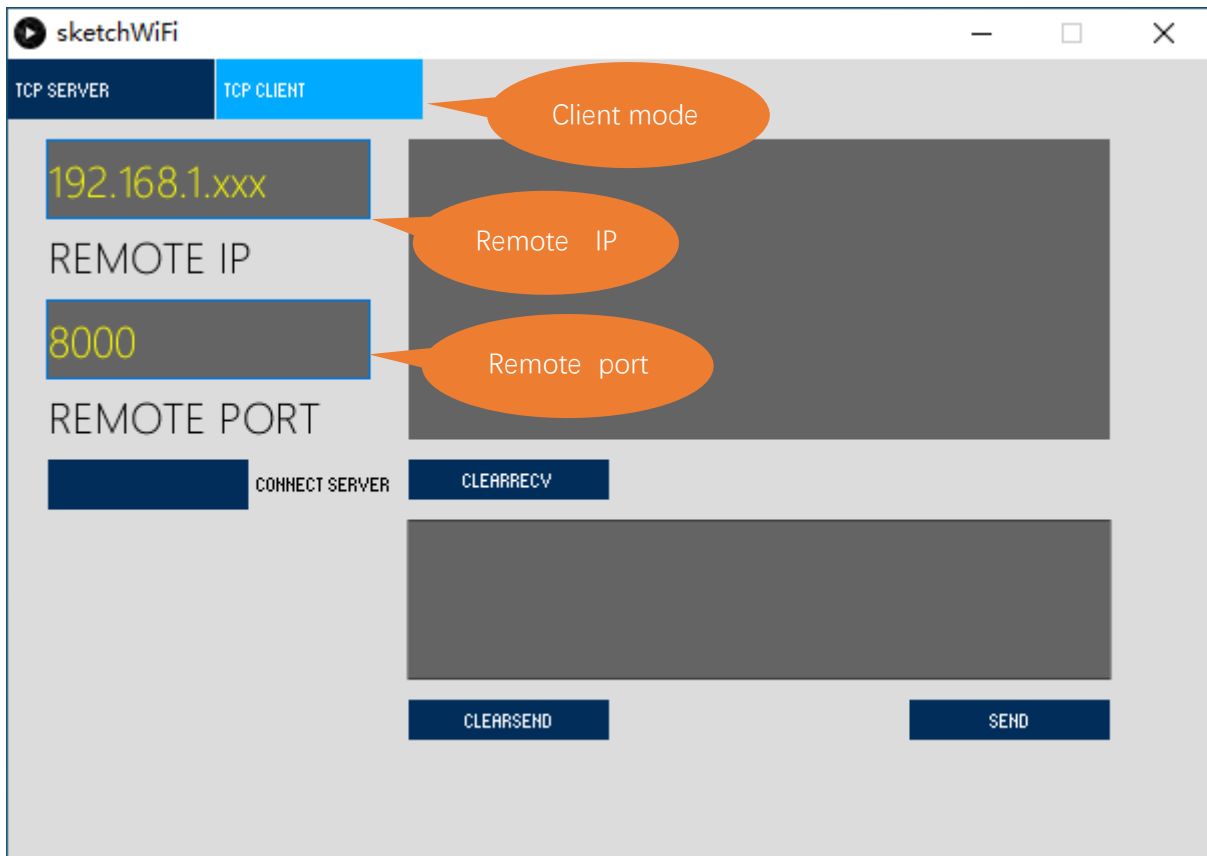


The new pop-up interface is as follows. If ESP32 is used as Client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, ESP32 Code needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If ESP32 serves as Server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

Mode selection: select **Server mode/Client mode**.

IP address: In Server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In Client mode, fill in the remote IP address to be connected.

Port number: In Server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

Start button: In server mode, push the button, and then the computer will serve as Server and open a port number for Client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a Client.

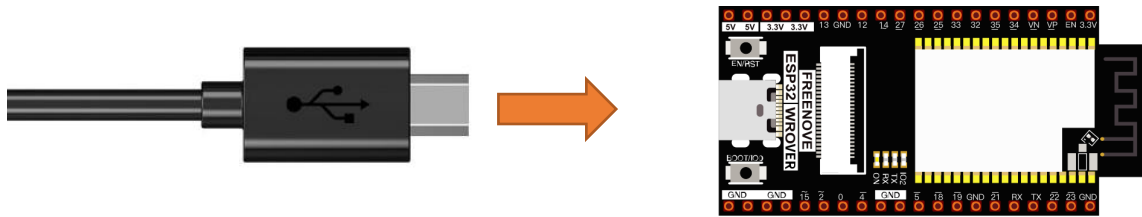
clear receive: clear out the content in the receiving text box

clear send: clear out the content in the sending text box

Sending button: push the sending button, the computer will send the content in the text box to others.

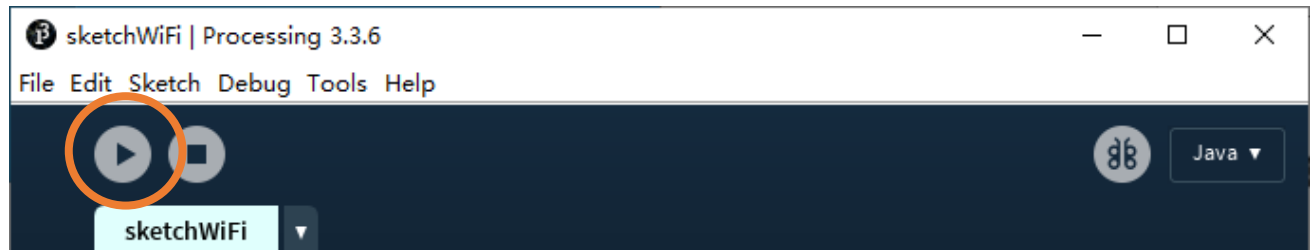
Circuit

Connect Freenove ESP32 to the computer using USB cable.

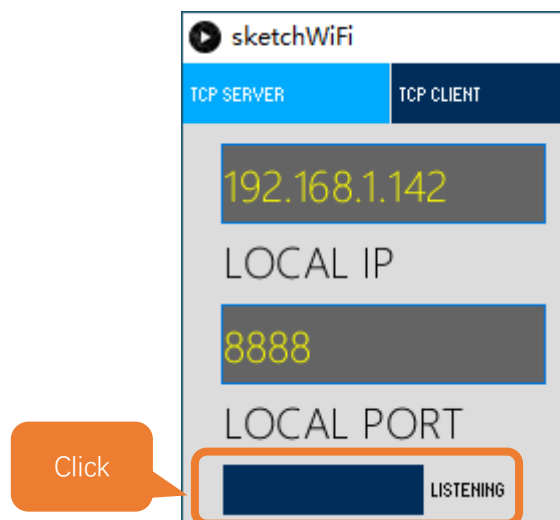


Code

Before running the Code, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port. Click "Listening".

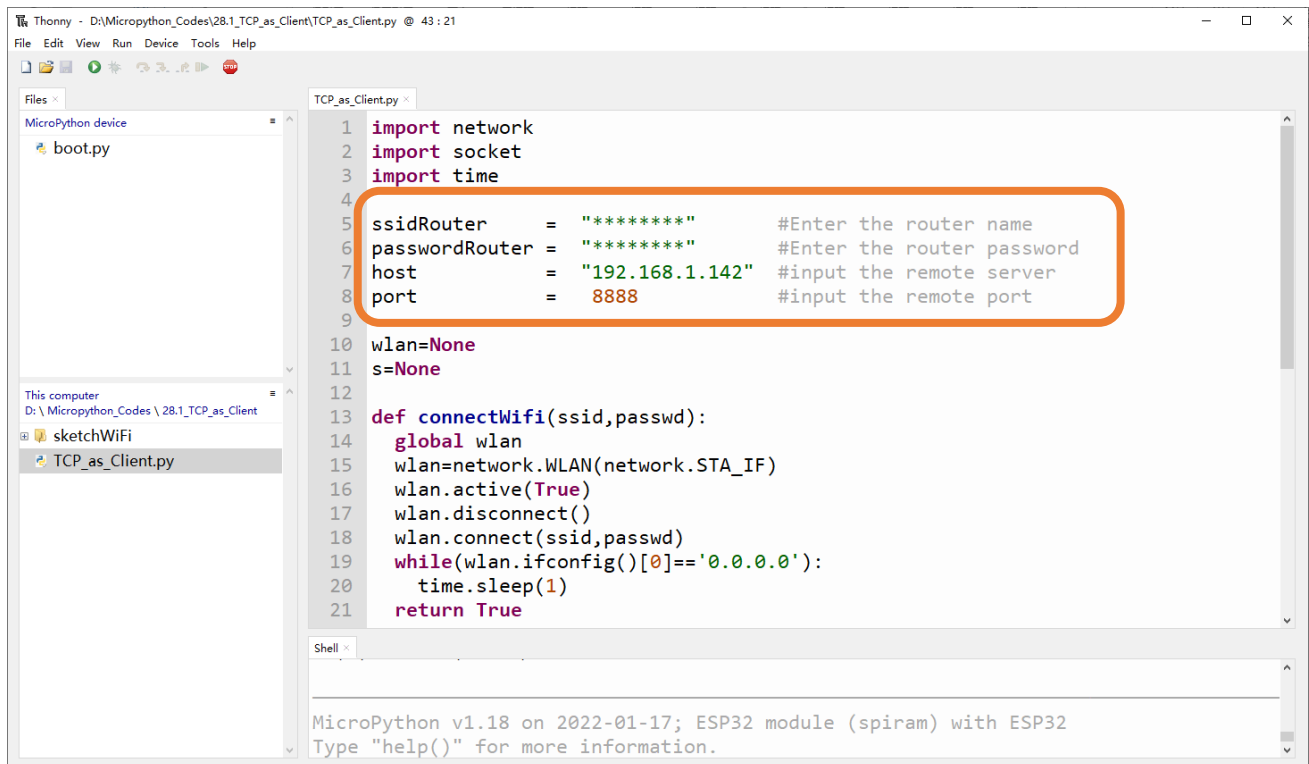


Move the program folder “Freenove_ESP32_WROVER_Board/Python/Python_Codes” to disk(D) in advance with the path of “D:/Micropython_Codes”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “04.1_TCP_as_Client” and double click “TCP_as_Client.py”.

Before clicking “Run current script”, please modify the name and password of your router and fill in the “host” and “port” according to the IP information shown in the box below:

04.1_TCP_as_Client



Click “Run current script” and in “Shell”, you can see ESP32-WROVER automatically connects to sketchWiFi.



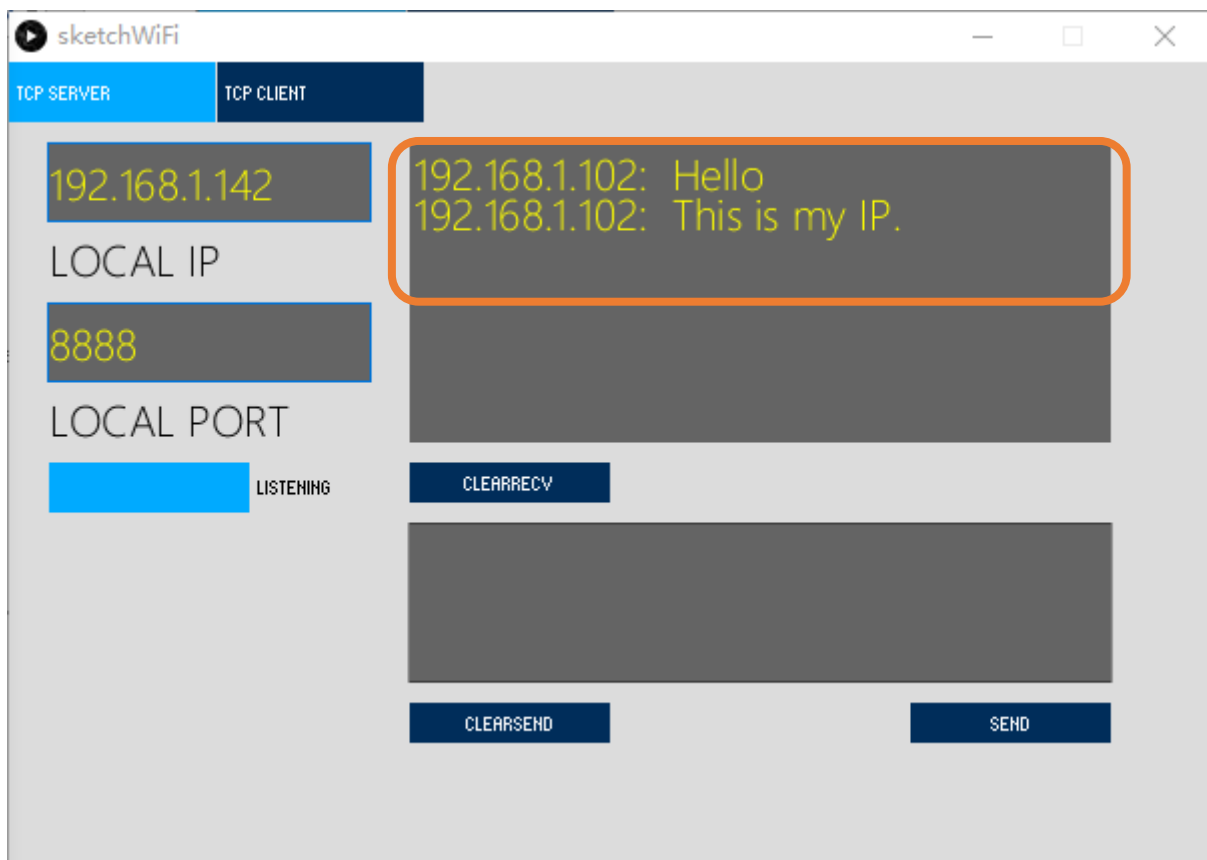
If you don't click "Listening" for sketchWiFi, ESP32-WROVER will fail to connect and will print information as follows:

```

Shell x
TCP connected to: 192.168.1.142 : 8888
Close socket
>>> %Run -c $EDITOR_CONTENT
TCP close, please reset!
>>>

```

ESP32 connects with TCP SERVER, and TCP SERVER receives messages from ESP32, as shown in the figure below.



The following is the program code:

```

1  import network
2  import socket
3  import time
4
5  ssidRouter    = "*****"    #Enter the router name
6  passwordRouter = "*****"    #Enter the router password
7  host          = "*****"    #input the remote server
8  port          = 8888        #input the remote port
9
10 wlan=None
11 s=None

```

```

12
13 def connectWifi(ssid,passwd):
14     global wlan
15     wlan= network.WLAN(network.STA_IF)
16     wlan.active(True)
17     wlan.disconnect()
18     wlan.connect(ssid,passwd)
19     while(wlan.ifconfig()[0]=='0.0.0.0'):
20         time.sleep(1)
21     return True
22 try:
23     connectWifi(ssidRouter,passwordRouter)
24     s = socket.socket()
25     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26     s.connect((host,port))
27     print("TCP Connected to:", host, ":", port)
28     s.send('Hello')
29     s.send('This is my IP.')
30     while True:
31         data = s.recv(1024)
32         if(len(data) == 0):
33             print("Close socket")
34             s.close()
35             break
36         print(data)
37         ret=s.send(data)
38 except:
39     print("TCP close, please reset!")
40     if (s):
41         s.close()
42     wlan.disconnect()
43     wlan.active(False)

```

Import network、socket、time modules.

```

1 import network
2 import socket
3 import time

```

Enter the actual router name, password, remote server IP address, and port number.

```

5 ssidRouter    = "*****"    #Enter the router name
6 passwordRouter = "*****"    #Enter the router password
7 host          = "*****"    #input the remote server
8 port          = 8888         #input the remote port

```

Connect specified Router until it is successful.

```
13 def connectWifi(ssid, passwd):
14     global wlan
15     wlan= network.WLAN(network.STA_IF)
16     wlan.active(True)
17     wlan.disconnect()
18     wlan.connect(ssid, passwd)
19     while(wlan.ifconfig()[0]!='0.0.0.0'):
20         time.sleep(1)
21     return True
```

Connect router and then connect it to remote server.

```
23 connectWifi(ssidRouter, passwordRouter)
24 s = socket.socket()
25 s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26 s.connect((host, port))
27 print("TCP Connected to:", host, ":", port)
```

Send messages to the remote server, receive the messages from it and print them out, and then send the messages back to the server.

```
28 s.send('Hello')
29 s.send('This is my IP.')
30 while True:
31     data = s.recv(1024)
32     if(len(data) == 0):
33         print("Close socket")
34         s.close()
35         break
36     print(data)
37     ret=s.send(data)
```

If an exception occurs in the program, for example, the remote server is shut down, execute the following program, turn off the socket function, and disconnect the WiFi.

```
39 print("TCP close, please reset!")
40 if (s):
41     s.close()
42 wlan.disconnect()
43 wlan.active(False)
```

Reference

Class socket

Before each use of **socket**, please add the statement “**import socket**” to the top of the python file.

socket([af, type, proto]): Create a socket.

af: address

socket.AF_INET: IPv4

socket.AF_INET6: IPv6

type: type

socket.SOCK_STREAM : TCP stream

socket.SOCK_DGRAM : UDP datagram

socket.SOCK_RAW : Original socket

socket.SO_REUSEADDR : socket reusable

proto: protocol number

socket.IPPROTO_TCP: TCPmode

socket.IPPROTO_UDP: UDPmode

socket.setsockopt(level, optname, value): Set the socket according to the options.

Level: Level of socket option

socket.SOL_SOCKET: Level of socket option. By default, it is 4095.

optname: Options of socket

socket.SO_REUSEADDR: Allowing a socket interface to be tied to an address that is already in use.

value: The value can be an integer or a bytes-like object representing a buffer.

socket.connect(address): To connect to server.

Address: Tuple or list of the server's address and port number

send(bytes): Send data and return the bytes sent.

recv(bufsize): Receive data and return a bytes object representing the data received.

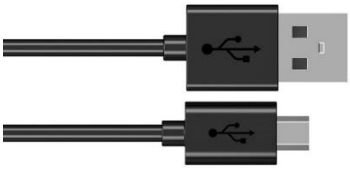
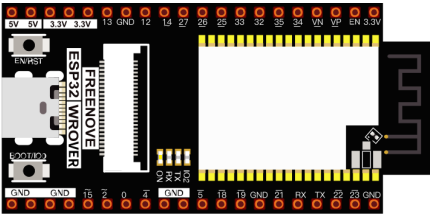
close(): Close socket.

To learn more please visit: <http://docs.micropython.org/en/latest/>

Project 4.2 As Server

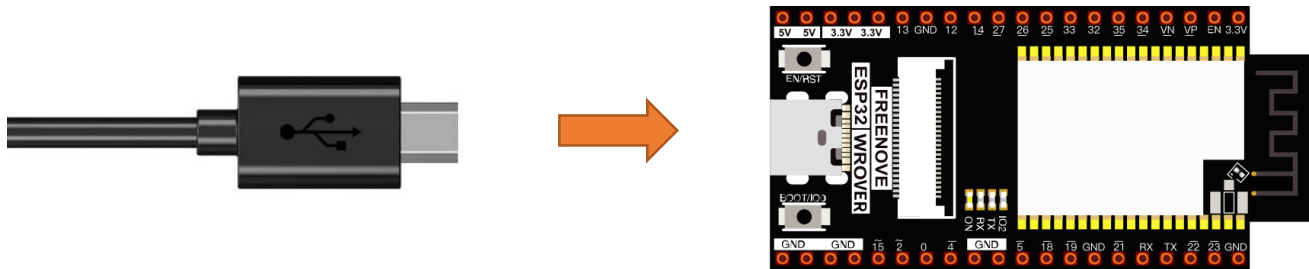
In this section, ESP32 is used as a Server to wait for the connection and communication with Client on the same LAN.

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
--	---

Circuit

Connect Freenove ESP32 to the computer using the USB cable.



Code

Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Open “Thonny”, click “This computer” → “D:” → “Micropython_Codes” → “04.2_TCP_as_Server” and double click “TCP_as_Server.py”.

Before clicking “Run current script”, please modify the name and password of your router shown in the box below.

04.2_TCP_as_Server

```

1 import network
2 import socket
3 import time
4
5 ssidRouter = "*****" #Enter the router name
6 passwordRouter = "*****" #Enter the router password
7 port = 8000 #input the remote port
8 wlan=None
9 listenSocket=None
10
11 def connectWifi(ssid,passwd):
12     global wlan
13     wlan=network.WLAN(network.STA_IF)
14     wlan.active(True)
15     wlan.disconnect()
16     wlan.connect(ssid,passwd)
17     while(wlan.ifconfig()[0]!='0.0.0.0'):
18         time.sleep(1)
19     return True
20
21 try:

```

After making sure that the router’s name and password are correct, click “Run current script” and in “Shell”, you can see a server opened by the ESP32- WROVER waiting to connecting to other network devices.

```

MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

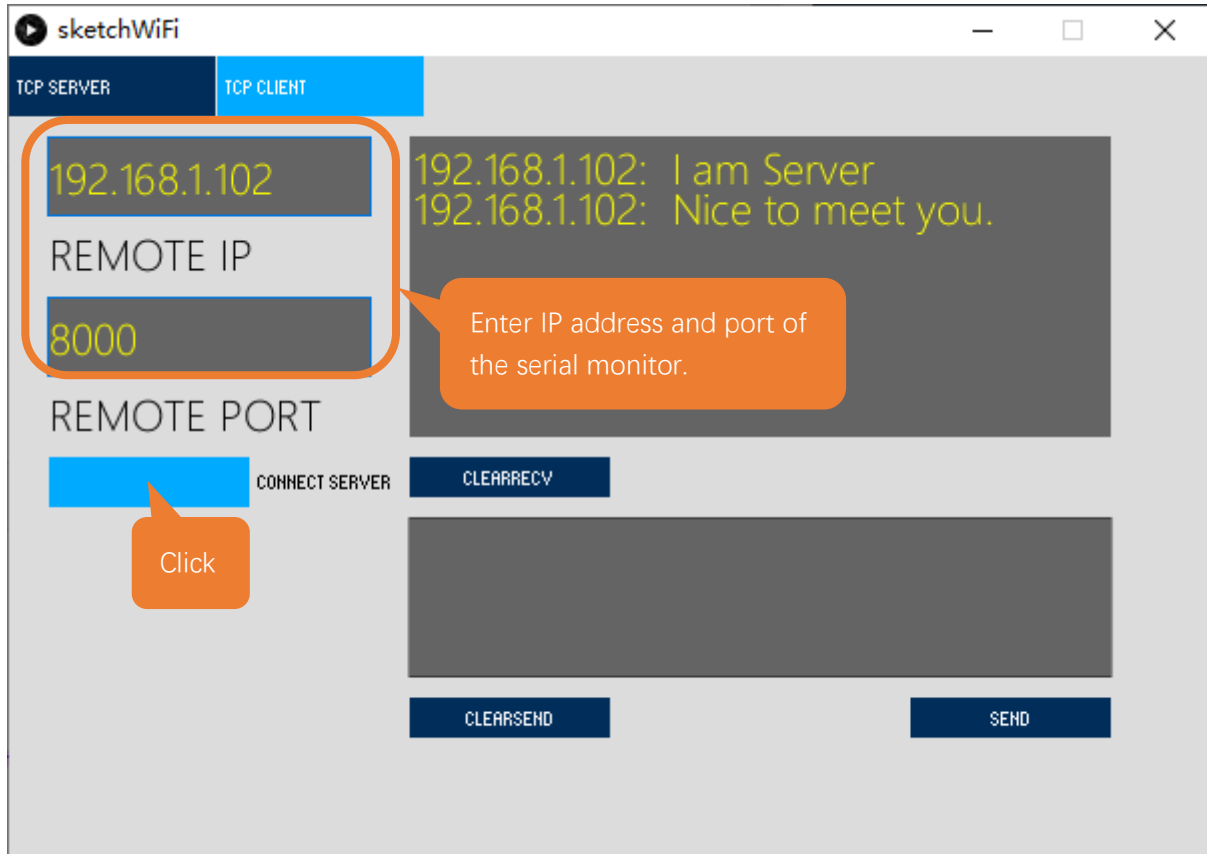
tcp waiting...
accepting....
Server IP: 192.168.1.102 Port: 8000

```

Processing:

Open the "Freenove_ESP32_WROVER_Board/Codes/MicroPython_Codes/04.2_TCP_as_Server/sketchWiFi/sketchWiFi.pde".

Based on the message printed in "Shell", enter the correct IP address and port when processing, and click to establish a connection with ESP32 to communicate.



You can enter any information in the "Send Box" of sketchWiFi. Click "Send" and ESP32 will print the received messages to "Shell" and send them back to sketchWiFi.

```
Shell x
MicroPython v1.12 on 2019-12-20; ESP32 module (spiram) with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT|

tcp waiting...
accepting.....
Server IP: 192.168.1.102      Port: 8000
('192.168.1.142', 51326) connected
b'Nice to meet you.'
```

The following is the program code:

```
1  import network
2  import socket
3  import time
4
5  ssidRouter    = "*****"      #Enter the router name
6  passwordRouter = "*****"      #Enter the router password
7  port          = 8000           #input the remote port
8  wlan          = None
9  listenSocket  = None
10
11 def connectWifi(ssid,passwd):
12     global wlan
13     wlan=network.WLAN(network.STA_IF)
14     wlan.active(True)
15     wlan.disconnect()
16     wlan.connect(ssid,passwd)
17     while(wlan.ifconfig()[0]!='0.0.0.0'):
18         time.sleep(1)
19     return True
20
21 try:
22     connectWifi(ssidRouter,passwordRouter)
23     ip=wlan.ifconfig()[0]
24     listenSocket = socket.socket()
25     listenSocket.bind((ip,port))
26     listenSocket.listen(1)
27     listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
28     print('tcp waiting...')
29     while True:
30         print("Server IP:", ip, "\tPort:", port)
31         print("accepting....")
32         conn,addr = listenSocket.accept()
33         print(addr,"connected")
34         break
35     conn.send(' I am Server')
36     while True:
37         data = conn.recv(1024)
38         if(len(data) == 0):
39             print("close socket")
40             listenSocket.close()
41             wlan.disconnect()
42             wlan.active(False)
43             break
```

```

44     else:
45         print(data)
46         ret = conn.send(data)
47 except:
48     print("Close TCP-Server, please reset.")
49     if(listenSocket):
50         listenSocket.close()
51     wlan.disconnect()
52     wlan.active(False)

```

Call function connectWifi() to connect to router and obtain the dynamic IP that it assigns to ESP32.

```

22     connectWifi(ssidRouter, passwordRouter)
23     ip=wlan.ifconfig()[0]

```

Open the socket server, bind the server to the dynamic IP, and open a data monitoring port.

```

24     listenSocket = socket.socket()
25     listenSocket.bind((ip, port))
26     listenSocket.listen(1)
27     listenSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

```

Print the server's IP address and port, monitor the port and wait for the connection of other network devices.

```

29     while True:
30         print("Server IP:", ip, "\tPort:", port)
31         print("accepting....")
32         conn, addr = listenSocket.accept()
33         print(addr, "connected")
34         break

```

Each time receiving data, print them in "Shell" and send them back to the client.

```

36     while True:
37         data = conn.recv(1024)
38         if(len(data) == 0):
39             print("close socket")
40             listenSocket.close()
41             wlan.disconnect()
42             wlan.active(False)
43             break
44         else:
45             print(data)
46             ret = conn.send(data)

```

If the client is disconnected, close the server and disconnect WiFi.

```

47 except:
48     print("Close TCP-Server, please reset.")
49     if(listenSocket):
50         listenSocket.close()
51     wlan.disconnect()
52     wlan.active(False)

```

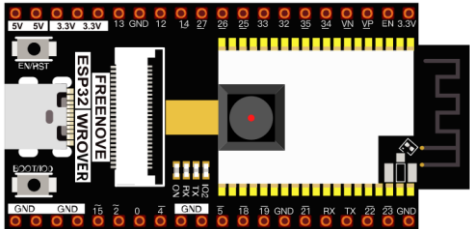
Chapter 5 Camera Web Server

In this section, we'll use ESP32's video function as an example to study.

Project 5.1 Camera Web Server

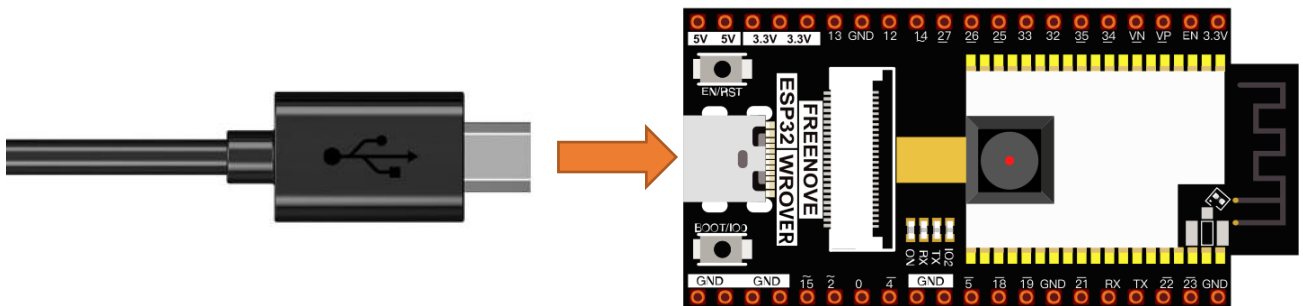
Connect ESP32 using USB and check its IP address through serial monitor. Use web page to access IP address to obtain video and image data.

Component List

Micro USB Wire x1 	ESP32-WROVER x1 
--	--

Circuit

Connect Freenove ESP32 to the computer using USB cable.



Code

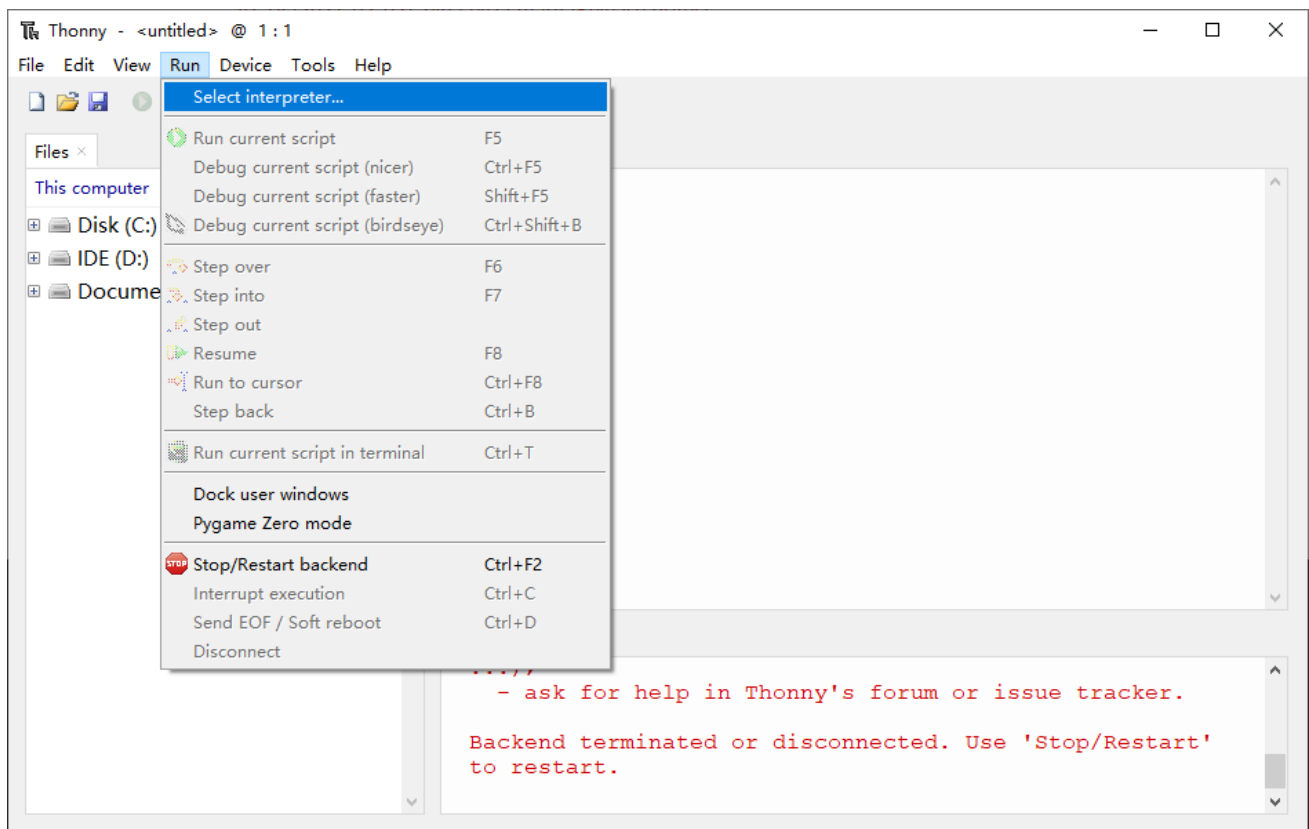
Move the program folder “**Freenove_ESP32_WROVER_Board/Python/Python_Codes**” to disk(D) in advance with the path of “**D:/Micropython_Codes**”.

Since Micropython does not provide firmware including camera module, in this chapter, we will use the camera based on the firmware in lemariva's Github project, micropython-camera-driver.

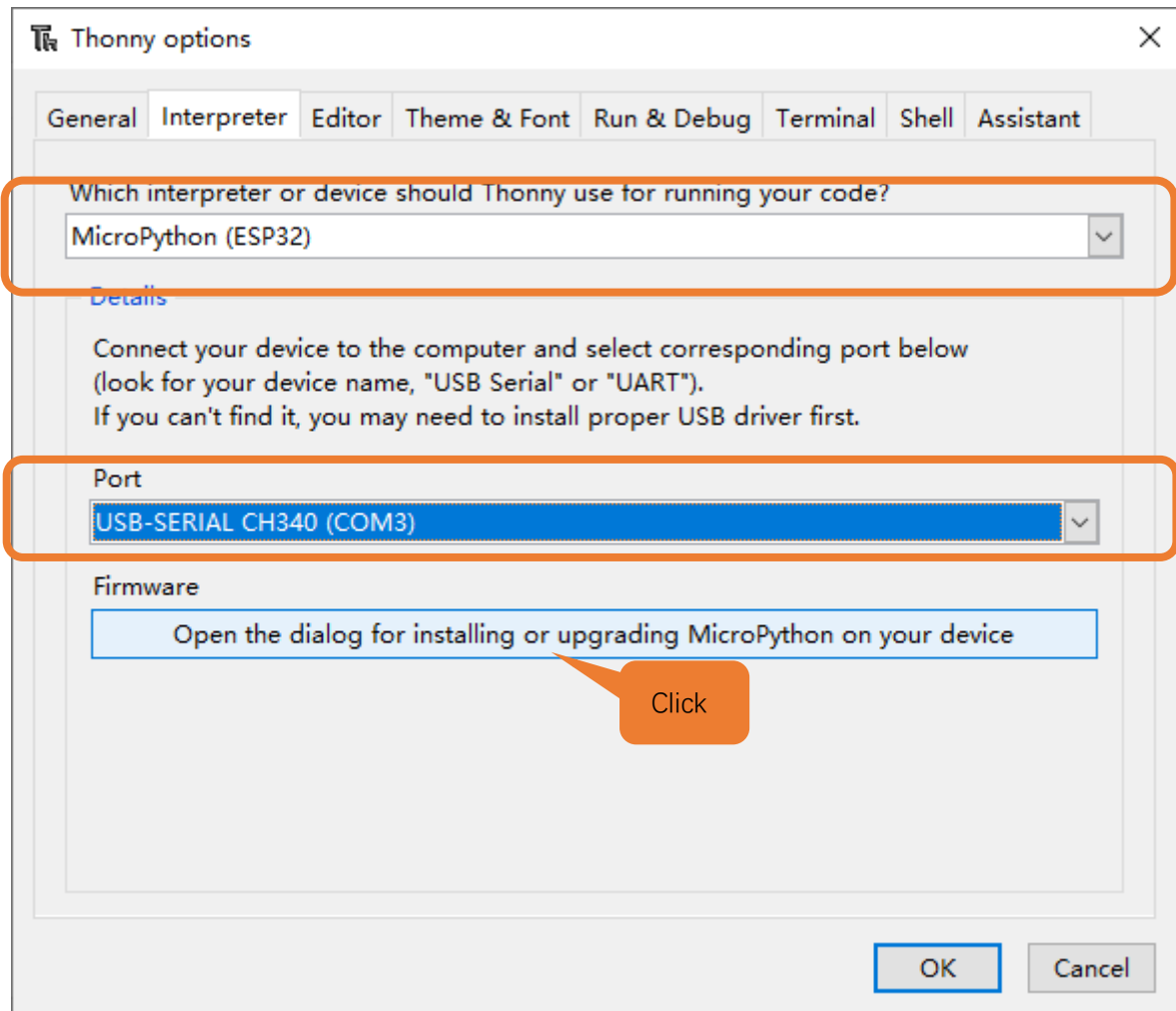
Project link: <https://github.com/lemariva/micropython-camera-driver>

Before starting the project, we need to re-upload the firmware with the camera module via steps below.

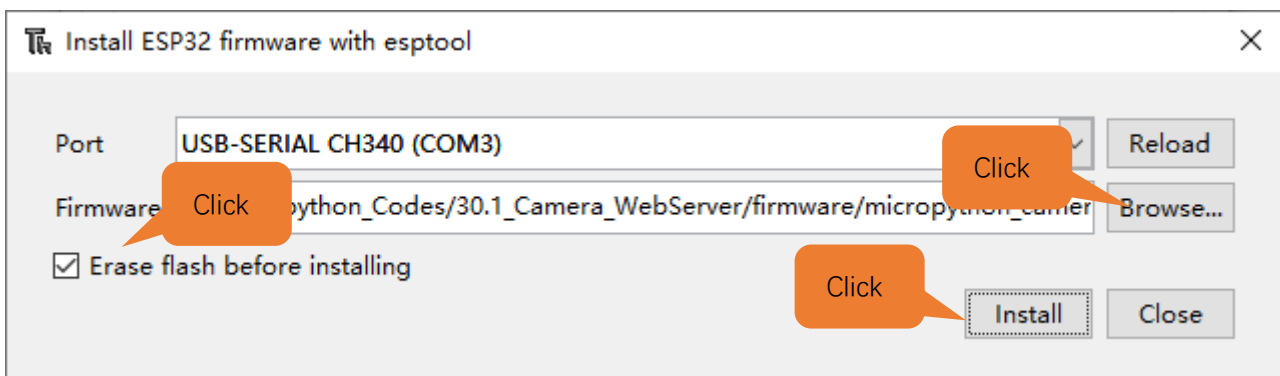
Open Thonny, click “run” and select “Select interpreter...”



Select “Micropython (ESP32)”, select “USB-SERIAL CH340 (COM4)”, and then click the long button under “Firmware”.



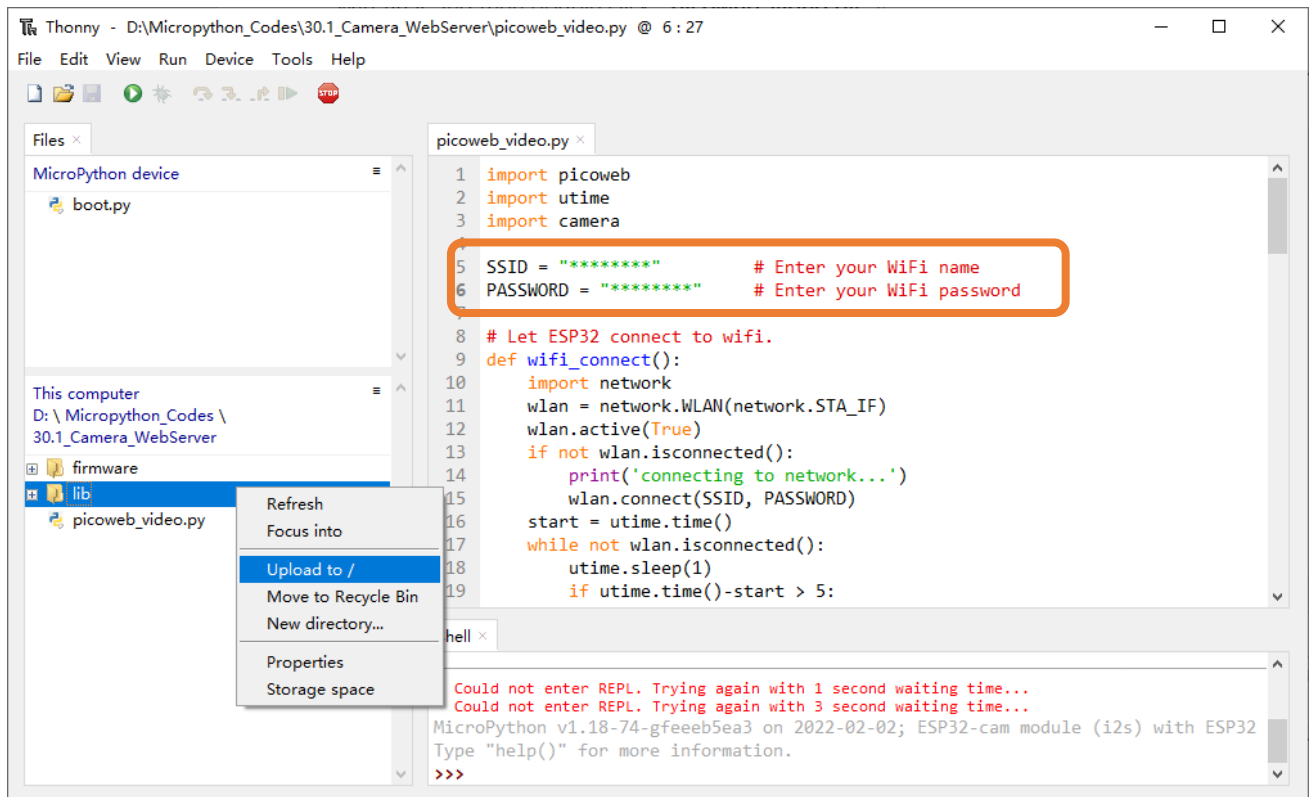
In the new popup window, select corresponding “USB-SERIAL CH340 (COM3)” for port. Click “Browse”, select “05.1_Camera_WebServer\firmware\micropython_camera_feeeb5ea3_esp32_idf4_4.bin”. Select “Erase...” and click “Install”.



Wait for completion.

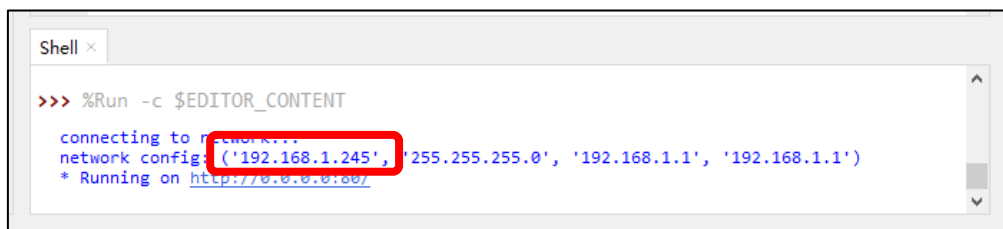
Open "Thonny", click "This computer" → "D:" → "Micropython_Codes" → "05.1_Camera_WebServer". Select folder "lib", right click your mouse to select "Upload to /", wait for "lib" to be uploaded to ESP32-WROVER and then double click "picoweb_video.py".

05.1_Camera_WebServer



Before running the program, please modify your router's name and password in the box shown in the illustration above to make sure that your code can compile and work successfully.

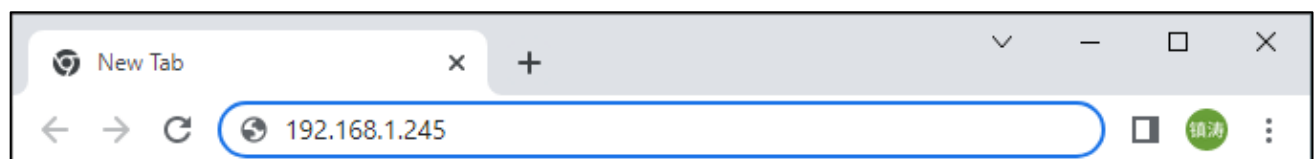
Click "run" to run the code "picoweb_video.py", then you can see the following content in the shell area.



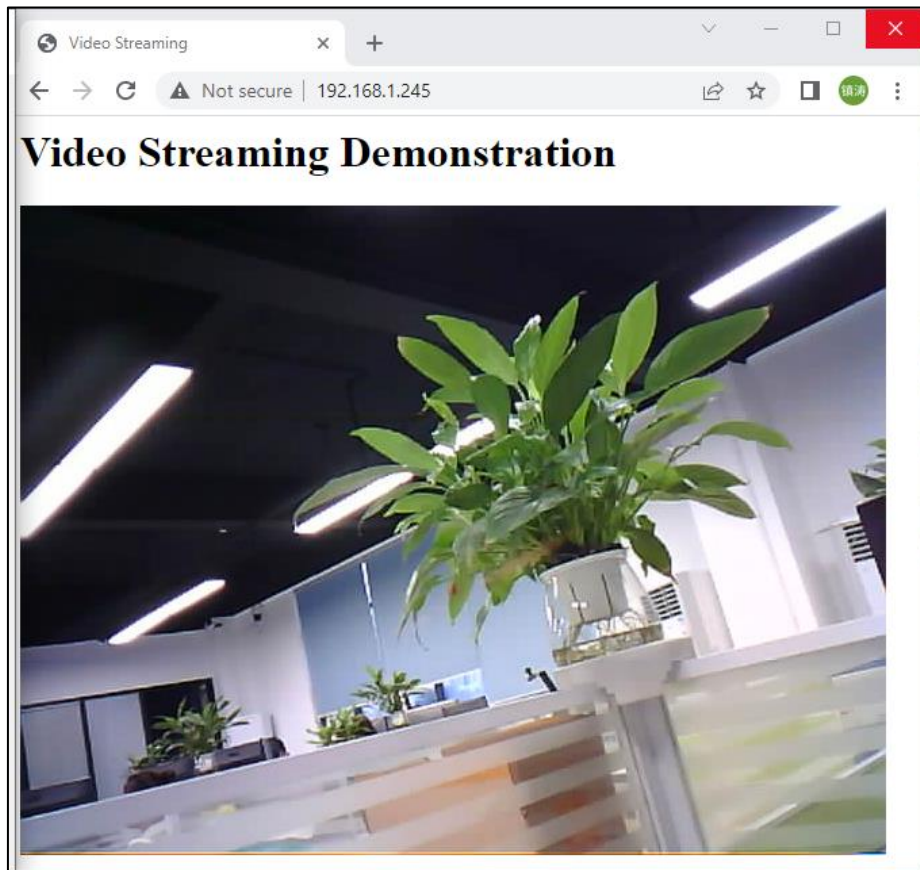
If your ESP32 has been in the process of connecting to router, but the information above has not been printed out, please re-check whether the router name and password have been entered correctly and press the reset key on ESP32-WROVER to wait for a successful connection prompt.

Open a web browser, enter the IP address printed by the serial monitor in the address bar, and access it.

Taking the Google browser as an example, here's what the browser prints out after successful access to ESP32's IP.



The effect is shown in the image below.



Please note:

If the shell area prompts an error when you click to run the code, please press the rst button on the esp32, wait for the system reset to complete, and then re-run the code.

The following is the program code.

```
1  import picoweb
2  import utime
3  import camera
4  import gc
5
6  SSID = "*****"      # Enter your WiFi name
7  PASSWORD = "*****"  # Enter your WiFi password
8
9  # Let ESP32 connect to wifi.
10 def wifi_connect():
11     import network
12     wlan = network.WLAN(network.STA_IF)
13     wlan.active(True)
14     if not wlan.isconnected():
15         print('connecting to network...')
16         wlan.connect(SSID, PASSWORD)
17     start = utime.time()
18     while not wlan.isconnected():
```

```

19         utime.sleep(1)
20         if utime.time()-start > 5:
21             print("connect timeout!")
22             break
23     if wlan.isconnected():
24         print('network config:', wlan.ifconfig())
25
26 # Initializing the Camera
27 def camera_init():
28     # Disable camera initialization
29     camera.deinit()
30     # Enable camera initialization
31     camera.init(0, d0=4, d1=5, d2=18, d3=19, d4=36, d5=39, d6=34, d7=35,
32                 format=camera.JPEG, framesize=camera.FRAME_VGA,
33                 xclk_freq=camera.XCLK_20MHz,
34                 href=23, vsync=25, reset=-1, pwn=-1,
35                 sioc=27, siod=26, xclk=21, pclk=22, fb_location=camera.PSRAM)
36
37     camera.framesize(camera.FRAME_VGA) # Set the camera resolution
38     # The options are the following:
39     # FRAME_96X96 FRAME_QQVGA FRAME_QCIF FRAME_HQVGA FRAME_240X240
40     # FRAME_QVGA FRAME_CIF FRAME_HVGA FRAME_VGA FRAME_SVGA
41     # FRAME_XGA FRAME_HD FRAME_SXGA FRAME_UXGA
42     # Note: The higher the resolution, the more memory is used.
43     # Note: And too much memory may cause the program to fail.
44
45     camera.flip(1) # Flip up and down window: 0-1
46     camera.mirror(1) # Flip window left and right: 0-1
47     camera.saturation(0) # saturation: -2,2 (default 0). -2 grayscale
48     camera.brightness(0) # brightness: -2,2 (default 0). 2 brightness
49     camera.contrast(0) # contrast: -2,2 (default 0). 2 highcontrast
50     camera.quality(10) # quality: # 10-63 lower number means higher quality
51     # Note: The smaller the number, the sharper the image. The larger the number, the more
52     blurry the image
53
54     camera.speffect(camera.EFFECT_NONE) # special effects:
55     # EFFECT_NONE (default) EFFECT_NEG EFFECT_BW EFFECT_RED EFFECT_GREEN EFFECT_BLUE
56     EFFECT_RETRO
57     camera.whitebalance(camera.WB_NONE) # white balance
58     # WB_NONE (default) WB_SUNNY WB_CLOUDY WB_OFFICE WB_HOME
59
60 # HTTP Response Content
61 index_web="""
62 HTTP/1.0 200 OK\r\n

```

```
61 <html>
62   <head>
63     <title>Video Streaming</title>
64   </head>
65   <body>
66     <h1>Video Streaming Demonstration</h1>
67     
68   </body>
69 </html>
70 """
71
72 # HTTP Response
73 def index(req, resp):
74     # You can construct an HTTP response completely yourself, having
75     yield from resp.awrite(index_web)
76
77 # Send camera pictures
78 def send_frame():
79     buf = camera.capture()
80     yield (b'--frame\r\n'
81           b'Content-Type: image/jpeg\r\n\r\n'
82           + buf + b'\r\n')
83     del buf
84     gc.collect()
85
86 # Video transmission
87 def video(req, resp):
88     yield from picoweb.start_response(resp, content_type="multipart/x-mixed-replace;
89 boundary=frame")
90     while True:
91         yield from resp.awrite(next(send_frame()))
92         gc.collect()
93
94 ROUTES = [
95     # You can specify exact URI string matches...
96     ("/", index),
97     ("/video", video),
98 ]
99
100 if __name__ == '__main__':
101
102     import ulogging as logging
103     logging.basicConfig(level=logging.INFO)
104     camera_init()
```

```

105     wifi_connect()
106
107     #Create an app object that contains two decorators
108     app = picoweb.WebApp(__name__, ROUTES)
109
110     app.run(debug=1, port=80, host="0.0.0.0")
111     # debug values:
112     # -1 disable all logging
113     # 0 (False) normal logging: requests and errors
114     # 1 (True) debug logging
115     # 2 extra debug logging

```

Import picoweb、utime、camera、gc modules.

```

1     import picoweb
2     import utime
3     import camera
4     import gc

```

Before running the code, please modify the WiFi name and password in the code to ensure that the ESP32 can connect to the network.

```

6     SSID = "*****"           # Enter your WiFi name
7     PASSWORD = "*****"       # Enter your WiFi password

```

Define the WiFi connection function, set the ESP32 to STA mode, and let the ESP32 connect to the nearby WiFi. If the connection is successful, the WiFi configuration information of the ESP32 will be printed; if the connection fails, the connection timeout will be printed.

```

10     def wifi_connect():
11         import network
12         wlan = network.WLAN(network.STA_IF)
13         wlan.active(True)
14         if not wlan.isconnected():
15             print('connecting to network...')
16             wlan.connect(SSID, PASSWORD)
17         start = utime.time()
18         while not wlan.isconnected():
19             utime.sleep(1)
20             if utime.time()-start > 5:
21                 print("connect timeout!")
22                 break
23         if wlan.isconnected():
24             print('network config:', wlan.ifconfig())

```

The deinit() is used to disable the configuration of the camera to prevent the previous configuration from interfering with the following configuration.

The init() is used to configure the camera's pin driver, image data format, resolution and other information. By default, please do not modify this function, otherwise the camera initialization fails and the image cannot be obtained.

```

29 camera.deinit()
30 # Enable camera initialization
31 camera.init(0, d0=4, d1=5, d2=18, d3=19, d4=36, d5=39, d6=34, d7=35,
32             format=camera.JPEG, framesize=camera.FRAME_VGA,
33             xclk_freq=camera.XCLK_20MHz,
34             href=23, vsync=25, reset=-1, pwn=-1,
35             sioc=27, siod=26, xclk=21, pclk=22, fb_location=camera.PSRAM)

```

This function can set the resolution of the camera individually, you can refer to the notes below to select the appropriate resolution size.

```

37 camera.framesize(camera.FRAME_VGA) # Set the camera resolution
38 # The options are the following:
39 # FRAME_96X96 FRAME_QQVGA FRAME_QCIF FRAME_HQVGA FRAME_240X240
40 # FRAME_QVGA FRAME_CIF FRAME_HVGA FRAME_VGA FRAME_SVGA
41 # FRAME_XGA FRAME_HD FRAME_SXGA FRAME_UXGA
42 # Note: The higher the resolution, the more memory is used.
43 # Note: And too much memory may cause the program to fail.

```

The following functions can modify the image information obtained by the camera.

```

45 camera.flip(1) # Flip up and down window: 0-1
46 camera.mirror(1) # Flip window left and right: 0-1
47 camera.saturation(0) # saturation: -2,2 (default 0). -2 grayscale
48 camera.brightness(0) # brightness: -2,2 (default 0). 2 brightness
49 camera.contrast(0) # contrast: -2,2 (default 0). 2 highcontrast
50 camera.quality(10) # quality: # 10-63 lower number means higher quality
51 # Note: The smaller the number, the sharper the image. The larger the number, the more
    blurry the image
52
53 camera.speffect(camera.EFFECT_NONE) # special effects:
54 # EFFECT_NONE (default) EFFECT_NEG EFFECT_BW EFFECT_RED EFFECT_GREEN EFFECT_BLUE
    EFFECT_RETRO
55 camera.whitebalance(camera.WB_NONE) # white balance
56 # WB_NONE (default) WB_SUNNY WB_CLOUDY WB_OFFICE WB_HOME

```

This is the code for a simple web interface, used here as an example.

```

59 index_web="""
60 HTTP/1.0 200 OK\r\n
61 <html>
62   <head>
63     <title>Video Streaming</title>
64   </head>
65   <body>
66     <h1>Video Streaming Demonstration</h1>
67     
68   </body>
69 </html>
70 """

```

Web page response function. When a user visits the webpage "/" built by ESP32, ESP32 calls this function, allowing the user to observe a display interface in the browser.

```
73 def index(req, resp):
74     # You can construct an HTTP response completely yourself, having
75     yield from resp.awrite(index_web)
```

send_frame() can send the image obtained by ESP32 in web page format. When someone visits the webpage "/video" built by the ESP32, the video(req, resp) function is used to continuously fetch images and send them to the browser.

```
77 # Send camera pictures
78 def send_frame():
79     buf = camera.capture()
80     yield (b'--frame\r\n'
81           b'Content-Type: image/jpeg\r\n\r\n'
82           + buf + b'\r\n')
83     del buf
84     gc.collect()
85
86 # Video transmission
87 def video(req, resp):
88     yield from picoweb.start_response(resp, content_type="multipart/x-mixed-replace;
89     boundary=frame")
90     while True:
91         yield from resp.awrite(next(send_frame()))
92         gc.collect()
```

Create two route decorators and declare their listening strings and corresponding response handlers respectively.

```
94 ROUTES = [
95     # You can specify exact URI string matches...
96     ("/", index),
97     ("/video", video),
98 ]
```

This is the main part of the program. First initialize the ESP32 camera, and then configure WiFi to connect the ESP32 to the network. Call the picoweb library, build a webserver, and run it.

```
103 import ulogging as logging
104 logging.basicConfig(level=logging.INFO)
105 camera_init()
106 wifi_connect()
107
108 #Create an app object that contains two decorators
109 app = picoweb.WebApp(__name__, ROUTES)
110
111 app.run(debug=1, port=80, host="0.0.0.0")
```

Reference

Image resolution	Sharpness	Image resolution	Sharpness
FRAMESIZE_96x96	96x96	FRAMESIZE_HVGA	480x320
FRAMESIZE_QQVGA	160x120	FRAMESIZE_VGA	640x480
FRAMESIZE_QCIF	176x144	FRAMESIZE_SVGA	800x600
FRAMESIZE_HQVGA	240x176	FRAMESIZE_XGA	1024x768
FRAMESIZE_240x240	240x240	FRAMESIZE_HD	1280x720
FRAMESIZE_QVGA	320x240	FRAMESIZE_SXGA	1280x1024
FRAMESIZE_CIF	400x296	FRAMESIZE_UXGA	1600x1200

We recommend that the resolution not exceed VGA(640x480).

What's next?

Thanks for your reading. This tutorial is all over here. If you find any mistakes, omissions or you have other ideas and questions about contents of this tutorial or the kit and etc., please feel free to contact us:

support@freenove.com

We will check and correct it as soon as possible.

If you want learn more about ESP32, you view our ultimate tutorial:

https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_ESP32/archive/master.zip

If you want to learn more about Arduino, Raspberry Pi, smart cars, robots and other interesting products in science and technology, please continue to focus on our website. We will continue to launch cost-effective, innovative and exciting products.

<http://www.freenove.com/>

End of the Tutorial

Thank you again for choosing Freenove products.